UOW MALAYSIA

PART OF THE UNIVERSITY OF WOLLONGONG AUSTRALIA GLOBAL NETWORK

Assessment Coversheet

Please complete all details required clearly. For softcopy submissions, please ensure this cover sheet is included/attached at the start of your document or in the file folder.

Please tick ✓ or click if using MS WORD								
	☑	UNIVERSITY	☐	UNIVERSITY COLLEGE	☐	COLLEGE		
CAMPUS	☑	UTROPOLIS GLENMARIE	☐	BATU KAWAN				
PROGRAM	☐	FOUNDATION/CERTIFICATE	☐	DIPLOMA	☑	DEGREE	☐	POST-GRADUATE

Assessment & Course Details:

Course Code: XBCS2183N		**Course Name: Operating System**	
Program Name: Bachelor of Computer Science (Hons)			
Lecturer/s Name: Ts. Aidora Abdullah			
Assessment Due Date: *(dd/mm/yy)*	07 / 08 / 2026	**Assessment Title:**	Final Report

Declaration:

- I/we have read and understood the rules on academic dishonesty and plagiarism in the University/University College/College Handbook listed here - https://www.uow.edu.my/current-students/handbook/
- I/we understand that plagiarism is the presentation of the work, idea or creation of another person as though it is your own. It is a form of cheating and a very serious academic offence. Plagiarised material can be drawn from, and presented in, written, graphic and visual form, including electronic data, and oral presentations. Plagiarism also occurs when the origin of the material used is not appropriately cited.
- I/we aware that electronic copy submissions may be submitted to a database for plagiarism detection.
- I/we have taken proper care of safeguarding this work and make all reasonable effort to ensure it could not be copied.
- I/we understand the rules and am aware of the consequences which may include failure in the course or part of the course and/or exclusion from the program.

Declaration on the Use of Generative AI:

Option A – AI Use Allowed (with disclosure)

- I/we confirm that I/we have used generative AI tools ethically and responsibly, within the limits permitted for this course.
- Any AI tools used (e.g. ChatGPT, Copilot, Grammarly, Bing AI) are listed below with their specific purpose.
- The final ideas, analysis, and conclusions reflect my/our own independent work.

Generative AI Tool Used	**Purpose of Use**	**Briefly Explain the Extent of Use**
Eg. ChatGPT	*Idea generation for introduction and structure*	*Minor – used only for brainstorming, all writing done independently*

Please add more rows to the table as needed so that each tool used in the creation of your assessment submission are included in the declaration.

Intellectual Property (IP) Ownership

- In accordance with the Institution's Intellectual Property Policy, I/we agree that upon submission, the intellectual property in this work shall be assigned to and vest in the Institution.
- I/we acknowledge that I/we shall not have the right to initiate or pursue any legal or other claims against the Institution in respect of this ownership.
- Notwithstanding the above, I/we shall be automatically granted, without further action, a perpetual, non-exclusive, non-transferable, irrevocable, royalty-free licence to use this work for my/our own research and educational purposes worldwide.
- The Institution may allow co-ownership of the intellectual property contained in this work with collaborators, in accordance with the provisions stipulated in relevant agreements.

By filling in your particulars below, you will be acknowledging this declaration.

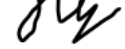
Name	Dihrran Chong Yun Song	**Name**	Li Ruochen
Student ID	0139028	**Student ID**	M44100654
Email	0139028@student.uow.edu.my	**Email**	M44100654@student.uow.edu.my
Mobile No:	016-317-8437	**Mobile No**	01153421675
Date:	06 / 07 / 2026	**Date**	06/07/2026
Signature		**Signature**	
Name	Zhu Wenxi	**Name**	Zhang Hejunjie
Student ID	M44100644	**Student ID**	0137675
Email	M44100644@student.uow.edu.my	**Email**	0137675@student.uow.edu.my
Mobile No	01133413920	**Mobile No**	01163212800
Date	06/07/2026	**Date**	06/07/2026
Signature		**Signature**	

Table of Contents

1.0 Introduction

This project is part of our Operating System XBCS2183N coursework to learn and investigate operating systems behaviour under different circumstances.

The goal is mainly focused on investigating, analyses and evaluating Oracle Linux Server's file system behavior and I/O performance behavior by deploying Apache and MariaDB services directly on Oracle Linux and we also will use Minikube to host Prometheus and Grafana for implement monitoring services such as visualization and real time metric collection. The behavior will be observed when subjected to a combined workload of concurrent HTTP web requests through Apache and database read/write transaction through MariaDB, it will also focus on how Linux operating system manages disk I/O, file system operations under concurrent workloads.

Furthermore, in this investigation, we also aim to reflect real world environment that will affect the operating system such as large media uploads and log churn, I/O latency with persistence guarantees, metadata heavy workloads and fully combined mixed workloads to represent the worse case scenario. This investigation will provide us better understanding on how server perform as user traffic increases.

Moreover, for file system behaviour, the investigation also observes on how the Linux kernel's file system handle creation, access, modification, and deletion operations Apache web requests and MariaDB database operations both running concurrently on Oracle Linux. Including how the kernel allocates inodes and how file metadata is updated as the workload increases.

On the other hand, for I/O performance behavior, this investigation also will focus on how the Linux kernel's I/O scheduler handle the disk read and write requests from both Apache and MariaDB running concurrently on the same oracle Linux node. This will include measure in disk performance, such as throughput and I/O waiting time under different levels of workload, to find out when performance starts to drop.

2.0 Objectives

Investigative Objective

The objective of this investigation will be focused on:

- Investigate and evaluate Oracle Linux Server's file system behaviour and I/O performance behaviour by deploying Apache and MariaDB services in Oracle Linux.

- Observe system behaviour under combined concurrent workloads of HTTP web requests through Apache and database read/write transactions through MariaDB in Oracle Linux.

- Investigate how the Linux kernel's file system handles file operation under increasing workload using stress-ng.

- Investigate and analysis how the file system responds to disk space pre-allocation pressure using stress-ng --fallocate under low, medium, and high stress conditions by measuring disk utilization and write latency.

- Investigate and analysis how forced disk write commits using stress-ng --fsync affect fsync latency, disk utilization, and I/O queue depth as concurrency increases from low to high stress levels.

- Investigate and analysis how concurrent directory creation and deletion operations using stress-ng --dir will affect the file system metadata processing, CPU usage, and I/O queue length when increase the thread counts under low, medium, and high stress conditions.

- Investigate and analysis how concurrent file system operation such as creation, reading, writing, and deletion operations using stress-ng --file affect XFS file system inode allocation, disk read and write throughput, and file operation latency under low, medium, and high stress levels..

- Simulate and analyse real world environments such as large media uploads and log churn, I/O latency with persistence guarantees, metadata heavy workloads, and fully combined mixed workloads to observe how the operating system perform.

3.0 System Architecture

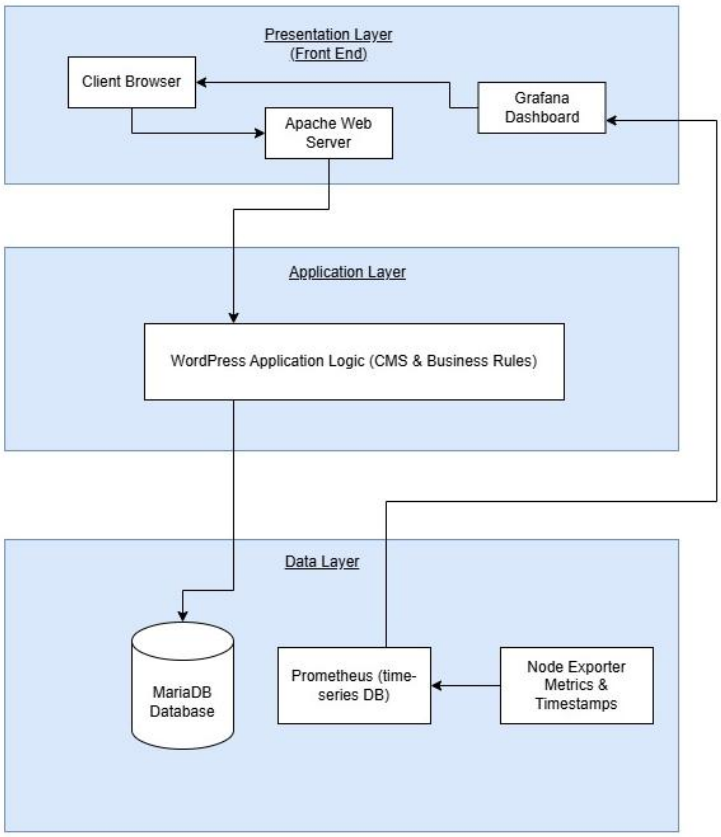


Figure x.x: 3-tier Client Architecture Diagram

The diagram above shows the 3-tier Client Architecture for our project. Presentation layer includes the front end which users will interact with. Application layer includes all the business rules and content management system. Then, data layer is where all the data is stored and accessed.

For presentation layer, we will use Grafana dashboard to visually represent data of the system metrics, focusing on showing file system behaviour while also includes some supporting information such as I/O performance to enhance our understanding of this project. We will also use Apache Web Server for users to access and interact with. This is important since we need actions from end users to stress the system.

Second, for the application layer, this is where the business rules take over. For example, we use Fakerpress to generate fake content, users, posts, and comments. This plugin will modify the

business logic of Wordpress because Fakerpress will add new logic which is content auto generation.

Lastly, the data layer will store every data involved in this project. It will store data from Fakerpress, Wordpress, and the system log. In order for the end user to fetch these data, the application layer will act as a connector between presentation layer and data layer. Hence, allowing the data from data layer to be accessed.

4.0 Hypothesis

4.0.1 File Allocation:

4.0.1.1 Low stress test:

Under low load conditions, Apache uses 10 concurrent requests, MariaDB uses 5 database threads, and it runs on Oracle Linux. After executing stress-ng --fallocate 1, it is expected that the disk utilization will remain below 20%, the write latency will be approximately 1–2 ms, and there will be no significant I/O queue waiting as the pressure on the storage system is relatively low. This assumption is intended to verify that the Linux operating system can efficiently handle low I/O workloads through the file system and I/O scheduling mechanism.

4.0.1.2 Medium stress test:

Under moderate load conditions, Apache handles 50 concurrent requests, MariaDB uses 25 database threads, and it runs on Oracle Linux. After conducting the same stress test, it is expected that the disk utilization will increase to approximately 40% - 60%, write latency will increase by about 10% - 20%, and there may be shorter I/O queue waits as more write requests are sent to the storage device. This assumption is intended to verify that Linux can still maintain stable I/O performance under moderate workloads.

4.0.1.3 High stress test:

Under high load conditions, Apache handles 100 concurrent requests, MariaDB uses 50 database threads, and it runs on Oracle Linux. After executing stress-ng --fallocate 1, it is expected that the disk utilization might exceed 80%, the write latency increases by approximately 30% - 60%, and there might be longer I/O queue waiting times as the storage system is under greater pressure. This assumption is intended to verify how Linux handles high I/O workloads when system resources are gradually approaching saturation.

4.0.2 Flushes data from a file to memory:

4.0.2.1 Low stress test:

Under low fsync concurrency (1–8 concurrent file writers), fsync latency is expected to remain below 5 ms, while disk utilization stays below 30%. Since only a small number of dirty pages are

generated, the Linux page cache can buffer data efficiently before background write-back occurs. As a result, I/O queue waiting time remains minimal, allowing Apache and MariaDB to maintain stable response times.

4.0.2.2 Medium stress test:

When the concurrent pressure is at a moderate level (16–32 fsync threads), the average delay of fsync will rise to 10–20 milliseconds, and the disk occupancy rate will synchronize to 60%–75%. A large number of synchronous write requests compete for the same storage hardware, causing an intensification of IO resource in the Linux kernel, an increase in the waiting time of the IO queue, and thereby prolonging the response time for Apache log writing and MariaDB transaction commit.

4.0.2.3 High stress test:

When the number of concurrent fsync threads exceeds 64, the disk occupancy rate almost reaches full capacity. The average latency for a single synchronous write operation will exceed 50 milliseconds, and the system's IO waiting overhead will significantly increase. Once the total amount of dirty pages in the system reaches the upper limit set by the kernel, the kernel's background disk flushing process and the fsync operations initiated by programs will mutually compete for disk resources, causing severe storage blocking problems. Eventually, this leads to a decline in database processing efficiency and an increased likelihood of timeout occurrences in web service requests.

4.0.3 Directory I/O stress testing (dir)

4.0.3.1 Low stress test:

When at low concurrency which is 4 directory threads, average operation latency will be below 5ms, disk utilization will be below 5%, and I/O queue length will be below 2 because low caching and overhead prevent backlog. This hypothesis is focusing on proving that the operating system can efficiently manage low concurrency requests through I/O scheduling.

4.0.3.2 Medium stress test:

When at medium concurrency which is 24 directory threads, average operation latency will be below 20ms, disk utilization will not be below 5%, and I/O queue length will be below 10. This is

due to the increased threads competition to perform operations such as create or delete in the same directory. Hence, lock contention increased. This hypothesis is focusing on

4.0.3.3 High stress test:

When at high concurrency which is 80 directory threads, average operation latency will increase, disk utilization will be below 10%, and I/O queue length will be lower than medium load. This is due a very high contention formed by many threads competition to perform directory operations. This hypothesis is trying to prove that having 80 threads will lead to a saturation points, where the system will not have any increase in performance.

4.0.4 File I/O stress testing:

4.0.4.1 Low stress test:

When at low concurrency, the system runs 4 concurrent file operation processes simultaneously. The disk read and write throughput will remind in low range of MB/s around 4 - 6 MB/s, file operation latency will reach under 20ms and Prometheus and Grafana are expected to show disk utilization below 40%, it might be due to 4 concurrent file operation processes creating, deleting, reading and writing files simultaneously causes the XFS file system to handle more inode and metadata updates than usual. This hypothesis is trying to prove that the Linux I/O scheduler can manage low concurrency file I/O without notice the performance had drop.

4.0.4.2 Medium stress test:

When at medium concurrency, the system runs 16 concurrent file operation simultaneously. The disk read and write throughput will reduce by 35%, file operation latency will increase by 50% compared to low stress test and Prometheus and Grafana are expected to show disk utilization below 25%, it might be due to 16 concurrent file operation processes creating, deleting, reading and writing files simultaneously causes the XFS file system to handle more inode and metadata updates than usual. This hypothesis tests whether increasing the number of file I/O operations causes more competition for file system resources.

4.0.4.3 High stress test:

When at high concurrency, the system runs 64 concurrent file operation simultaneously. The disk read and write throughput will reduce by around 70%, file operation latency will increase by around 200% compared to low stress test and Prometheus and Grafana are expected to show disk utilization below 15%, it might be due to 64 concurrent file operation processes creating, deleting, reading and writing files simultaneously causes the XFS file system to handle more inode and metadata updates than usual. This hypothesis is trying to prove that 64 concurrent file operations will overload the system and cause performance to drop.

4.0.5 Real world scenarios

4.0.5.1 Large media uploads + log churn:

4.0.5.1.1 Low stress test:

Under low load conditions, fallocate and file run together to simulate a light mixed I/O scenario. Apache and MariaDB traffic on Oracle Linux remains at a low level, generating only a small number of file allocation and log writing operations. Prometheus and Grafana are expected to show disk utilization below 30%, write latency below 5 ms, and little or no I/O wait. This is because the Linux file system can process file allocation and metadata updates efficiently when storage resource contention is low.

4.0.5.1.2 Medium stress test:

As the workload increases to a medium level, Apache and MariaDB generate more file system activity while fallocate and file continue running. Prometheus and Grafana are expected to show disk utilization increasing to about 40%–60%, write latency increasing by about 10%–20%, and I/O wait starting to appear. This is because the Linux kernel must process more file allocation requests, metadata updates, and disk scheduling operations.

4.0.5.1.3 High stress test:

At high load, continuous file allocation and log writing create much heavier pressure on the file system. Prometheus and Grafana are expected to show disk utilization exceeding 80%, write latency increasing by about 30%–60%, and high I/O wait. The increased number of inode updates

and disk I/O requests is expected to cause greater storage resource contention and reduce overall file system performance.

4.0.5.2 I/O latency + persistence guarantees:

4.0.5.2.1 Low stress test:

In the mixed low-concurrency scenario configuration, 4 synchronous disk-writing threads and 4 file reading and writing processes are set up. Apache and MariaDB only handle the minimum basic business traffic. The amount of data synchronized to the disk is very small, and the phenomenon of disk resource contention is weak. The average IO delay is expected to be less than 10 milliseconds; the performance loss caused by the synchronization operation is limited, and the overall system throughput is reduced by less than 10% compared to the asynchronous reading and writing mode.

4.0.5.2.2 Medium stress test:

The mixed medium concurrency includes 32 fsync threads and 16 file processing processes. At the same time, the web and database services handle medium traffic. The combination of synchronous disk writing and file read/write tasks will intensify the competition for disk IO resources. A large number of synchronous write requests competing for the same storage hardware will cause the average fsync delay to rise to 20 to 40 milliseconds, resulting in a total system throughput decrease of approximately 20% to 30%.

4.0.5.2.3 High stress test:

Enabling mixed high concurrency involves using more than 64 fsync threads and 64 file processes. Web and database services can handle peak traffic. Frequent forced disk writing operations will fill up the shared disk, causing a serious IO performance bottleneck. The average system IO latency exceeds 50 milliseconds. Compared to tasks without a forced persistence mechanism, the overall throughput drops by approximately 60%. Under high write pressure, the synchronous disk writing operation will limit the optimization ability of the Linux kernel for IO scheduling, thereby significantly slowing down the system performance.

4.0.5.3 Metadata heavy workloads:

4.0.5.3.1 Low stress test:

Testing low workload which is 4 directory operations and 4 file operation using stress-ng file and directory stressors will increase the disk I/O per second but maintain below 1.3k operations, latency below 10 ms and disk utilization will be below 50%.

4.0.5.3.2 Medium stress test:

Testing medium workload which is 24 directory operations and 16 file operation using stress-ng file and directory stressors will increase the disk I/O per second but maintain below 1.5k operations, latency below 15 ms and disk utilization will be below 70%.

4.0.5.3.3 High stress test:

Testing high workload which is 80 directory operations and 64 file operation using stress-ng file and directory stressors will increase the disk I/O per second to > 3.0k operations, latency below over 50ms and disk utilization will be below 70%.

4.0.5.4 Everything is combined:

4.0.5.4.1 Low stress test:

When at low load condition, all the three real world scenarios run together (Large media uploads with log churn + I/O latency with persistence guarantees + Metadata heavy workloads), which is 4 fallocate operation, 4 fsync operations, 4 directory operations, and 4 file operations using stress-ng running simultaneously. Prometheus and Grafana are expected to show disk throughput remaining high, in the range of 400 – 450MB/s, and the disk utilization rising to around 60% - 75%, write latency increasing to 5 ms, and minor I/O waiting queue beginning to appear (around 1 – 5 disk request), it might be due to the file system must process file space pre-allocation, continuous file creation and deletion, forced disk write commits, and continuously directory operations all at the same time even through it is low stress test only.

4.0.5.4.2 Medium stress test:

When at medium load condition, all the three real world scenarios run together. Which is 16 fallocate operations, 16 fsync operations, 16 directory operations, and 16 file operations using stress-ng running simultaneously. Prometheus and Grafana are expected to show throughput reducing by 35% compared to the low stress test and the disk utilization rising to around 70%–80%, read and write latency reach around 10 – 12 ms, and 15 disk requests waiting to be processed in the I/O queue compared to low stress test, it might be due to the file system must process file space pre-allocation, continuous file creation and deletion, forced disk write commits, and continuous directory operations all at the same time under medium load, causing more competition for file system resources than using only one type of operation.

4.0.5.4.3 High stress test:

When at high load condition, all the three real world scenarios run together, which is 64 fallocate operations, 64 fsync operations, 64 directory operations, and 64 file operations using stress-ng running simultaneously. Prometheus and Grafana are expected to show disk throughput reducing by 55% compared to the low stress test and the disk utilization approaching over 100%, read and write latency reaches around 20 – 24 ms, and 30 disk requests will be waiting in the I/O queue compared to low stress test. It will make the file system completely overwhelming, lose the ability to process inode allocations and metadata updates simultaneously, causing the Linux I/O scheduler queue to saturate and reach the bottleneck point.

5.0 Experimental Design

5.0.1 File allocation

Workload Definition:

This experiment simulates a mixed I/O workload on Oracle Linux running native Apache and MariaDB, combined with file system stress testing. And there are two stress-ng workloads used:

- fallocate: simulate large file uploads and space allocation like video files.
- file: simulate continuous log generation and small file writes.

Apache and MariaDB workloads on Oracle Linux are increased from medium to high concurrency to simulate web and database activity under pressure.

3 test conditions are defined:

- low load: Apache runs with 10 concurrent requests, and MariaDB runs with 5 database threads.
- medium load: Apache runs with 50 concurrent requests, and MariaDB runs with 25 database threads
- high load: Apache runs with 100 concurrent requests, and MariaDB runs with 50 database threads.

Variables:

Type	Variables	Purpose
Independent Variables	Apache request concurrency, MariaDB transaction workload, stress-ng (fallocate + file), workload increasing from medium to high	Control overall system I/O pressure
Dependent Variables	• Disk I/O throughput • %iowait (values obtained via iostat) • write latency • disk utilization • bogo ops • bogo ops/s (usr+sys)	Measure system performance changes

	• bogo ops/s (real time) • real time • usr time • sys time	
Controlled Variables	Virtual machine configuration (CPU, memory, disk), Oracle Linux verison, Kubernetes environment, test duration	Keep other factors unchanged and remove interference factors

Observation Tools:

iostat: used to monitor disk I/O performance and changes in %iowait

Prometheus: used to collect usage data on CPU, memory and disk resources

Grafana: used to visualize disk utilization, latency changes, and I/O queue trends

Procedure:

1. Ensure Apache and MariaDB services running natively on Oracle Linux.
2. Confirm that the system operates normally under low load conditions.
3. Run the workload under the low-load condition and collect system metrics.
4. Increase Apache and MariaDB workloads to the medium load condition.
5. Run stress-ng --fallocate and stress-ng --file to simulate large media uploads and continous log generation.
6. Increase the workloads to the high load condition and run again.
7. Use iostat, Prometheus, and Grafana to collect the running data.
8. Compare the system performance between the low, medium and high conditions.

Evidence Collection:

- iostat output, such as %iowait, disk utilization, and throughput changes.
- Grafana dashboard screenshots, showing disk latency and I/O queue status.
- Prometheus system metrics, including CPU, memory, and disk usage.
- stress-ng execution logs (records of fallocate and file workloads).

- Apache and MariaDB running logs.
- Performance comparison screenshots under different workload levels (low → medium → high).

5.0.2 Flushes data from a file to memory

Workload Definition:

This test utilizes the fsync function of stress-ng to analyze the operating system's behavior during forced page cache flushing. When running Apache and MariaDB as background services on Oracle Linux, the overhead caused by data synchronization barriers can be quantified.

By implementing three levels of fsync synchronization concurrency, performance thresholds for the XFS file system can be identified:

- low load: 1–8 concurrent fsync threads, with the lowest background traffic of Apache/MariaDB
- medium load: 16–32 concurrent fsync threads, with moderate web and database transaction volume
- high load: ≥64 concurrent fsync threads, with the maximum configured concurrent number of Apache and MariaDBL

Variables:

Type	Variables	Purpose
Independent Variables	Number of stress-ng fsync threads (8, 32, 64), Apache request rate, MariaDB transaction volume	Adjust the cache refresh pressure and the background business load
Dependent Variables	• Average fsync latency • disk utilization • I/O queue depth • MariaDB commit response time • bogo ops • bogo ops/s (usr+sys) • bogo ops/s (real time) • real time • usr time • sys time	Measure filesystem degradation, sync overhead and service performance impact

Controlled Variables	VM hardware configuration, Oracle Linux version, Minikube monitoring stack - all these have fixed the 60-second test duration for each load level.	Standardized testing environment, eliminating interference, ensuring reliable data indicators

Observation Tools:

iostat: Monitors real-time disk usage, including storage utilization, written sectors, and dynamic %iowait values.

Prometheus: A time series data collector used for collecting host dirty page write-back logs and core storage performance data.

Grafana: A visualization dashboard tool used to display performance degradation, I/O queue bottlenecks, and transaction delay anomalies.

Procedure:

1. Verify the stable baseline operation status of Apache and MariaDB on Oracle Linux.
2. Confirm that Node Exporter can successfully transfer host metrics to Prometheus in Minikube.
3. Run stress-ng --fsync with 8 worker threads for 60 seconds to conduct low-concurrency testing.
4. Pause for 60 seconds, complete the write-back of dirty pages and restore the system to the baseline state.
5. Start a stress-ng --fsync test with 32 worker threads at medium load, observe the I/O queue accumulation situation.
6. Perform a 60-second cooldown recovery period.
7. Run a high-load test with 64 fsync threads to trigger large-scale hardware synchronization blocking and page refreshing.
8. Export timeline metrics from the Grafana dashboard and compare the performance thresholds at each level.

Evidence Collection:

- Stress-ng terminal log: Raw performance metrics for each fsync synchronization concurrency level.
- Grafana dashboard record: Visualized data on I/O queue peaks and latency fluctuations.
- iostat output log: Historical iowait percentage values and hardware disk response limit information.
- MariaDB transaction slow operation log: Records of timeouts caused by shared disk blocking competition.

5.0.3 Directory I/O stress testing

Workload Definition:

This experiment focuses on observing the server conditions when there are many concurrent creations, deletion, and opening of directories of the webpress. The stressing stimulus is done by using these command tools:

- --dir: numbers of concurrent directory activities
- --timeout: time for the test to finish

To get more variations of data, there are 3 tests that will be conducted which is the low, medium, and high stress test. This ensures that we can get a more detailed insight during analysis. Hence, improving the quality of our project outcomes.

- Low load: 4 directory threads, 60 seconds duration, low directory pressure
- Medium Load: 24 directory threads, 60 seconds duration, medium directory pressure
- High Load: 80 directory threads, 60 seconds duration, medium directory pressure

Variables:

Type	Variables	Purpose
Independent Variables	• Numbers of workers	Testing different concurrent directory stress level
Dependent Variables	• Disk I/O Operations • Disk throughput • Disk Wait Time • Disk I/o Utilization • Average queue size • Root File Nodes free • Root Filesystem Used • bogo ops • bogo ops/s (usr+sys) • bogo ops/s (real time)	Measurable outcome for observation

	• real time • usr time	
Controlled Variables	• Same Hardware: CPU, RAM and Memory • Same Server: Apache and MariaDB	Allows a fixed environment condition for a more precise outcome

Observation Tools:

Grafana dashboard: Viewing the disk and inode usage and free space in real time

Prometheus metrics data

Node Exporter scrapped metrics

Procedure:

1. Deploy Apache and MariaDB services running natively on Oracle Linux.
2. Deploy Prometheus + Grafana services in Minikube.
3. Confirm Node Exporter is running on Oracle Linux and successfully visible in the Grafana dashboard before proceeding with any stress testing.
4. Run watch and du commands to record the initial directory size.
5. Run stress-ng –dir with 4 workers under low stress conditions to observe initial directory behavior in 60 seconds.
6. Allow system to recover in 60 seconds.
7. Run stress-ng –dir with 24 workers under low stress conditions to observe initial directory behavior in 60 seconds.
8. Allow system to recover in 60 seconds.
9. Run stress-ng –dir with 80 workers under low stress conditions to observe initial directory behavior in 60 seconds.
10. Observe Grafana dashboard and collect the result.
11. Compare the system performance between low, medium and high then analyze it.

Evidence Collection:

- Screenshot of the Grafana dashboard will be collected to provide a visual representation of the result.
- The result that are inserted in the performance metrics table will be used as evidence as well to provide clear comparison between baseline and workload performance.
- Stress-ng output log will also be collected as evidence to complement the result in the dashboard. It provides evidence of Prometheus scraping activities and kernel messages.

5.0.4 File I/O stress testing

Workload Definition:

This experiment focuses on observing the server conditions when there are many concurrent file operations happening simultaneously on the Oracle Linux XFS file system. Simulate file I/O pressure by continuously creating, reading, writing and deleting files on the Oracle Linux XFS file system while Apache and MariaDB run together on Oracle Linux. Node exporter will collect the result for performance metrics from Oracle Linux and export them to Prometheus running inside the Minikube, and the Grafana will visualize it in real time. The stressing stimulus is done by using these command tools:

- --filename: continuous log file creation, reading, writing, and deletion
- --timeout: duration of each stress test
- --metrics-brief: displays a summary of operations completed after each test

There are 3 tests that will be conducted which are the low, medium, and high stress test. This ensures that we can get a more detailed insight during analysis. Hence, improving the quality of our project outcomes:

- Low load: 4 concurrent file operation processes, 60 seconds duration, low file I/O pressure.
- Medium load: 16 concurrent file operation processes, 60 seconds duration, medium file I/O pressure.
- High load: 64 concurrent file operation processes, 60 seconds duration, heavy file I/O pressure.

Variables:

Type	Variables	Purpose
Independent Variables	Number of stress-ng file stressor workers: 4 (low stress), 16 (medium stress), 64 (high stress)	Control file I/O pressure
Dependent Variables	• Disk read and write throughput	Measure the performance change and the time taken

	• File operation latency • Average queue size • disk utilization • bogo ops • bogo ops/s (usr+sys) • bogo ops/s (real time) • real time • usr time • sys time	
Controlled Variables	• Virtual machine configuration • Oracle Linux version • Test duration • Stress-ng version • Same Server: Apache and MariaDB	Make those factors unchanged to make the observation fair

Observation Tools:

Prometheus: used to collect usage data on CPU, memory and disk resources.

Grafana: used to visualize disk utilization, latency changes, and I/O queue trends.

Iostat: It is used to observe real-time disk performance.

Procedure:

1. Deploy Apache and MariaDB services running natively on Oracle Linux.
2. Deploy Prometheus + Grafana services in Minikube
3. Confirm that the system operates normally under baseline conditions.
4. Confirm Node Exporter is running on Oracle Linux and successfully visible in the Grafana dashboard before proceeding with any stress testing.

5. Run stress-ng --file with 4 workers under low stress conditions to observe initial file I/O behavior in 60 seconds.
6. Allow system to recover in 60 seconds.
7. Run stress-ng --file with 16 workers under medium stress conditions to observe file I/O behavior in 60 seconds.
9. Allow system to recover in 60 seconds.
10. Run stress-ng --file with 64 workers under high stress conditions to observe file I/O behavior in 60 seconds.
11. Use iostat, vmstat, Prometheus, and Grafana to continuously collect runtime performance data throughout each stress level.
12. Compare the performance between each of the stress levels and analyse it.

Evidence Collection:

- Video recording for Grafana dashboard during stress test.
- screenshots the Grafana dashboard to show disk utilization, read and write latency, and I/O queue depth at each stress level.
- The result that are inserted in the performance metrics table will be used as evidence as well to provide clear comparison between benchmark test and load test performance.
- stress-ng terminal logs tells will be collected as evidence to complement the result in the dashboard. It provides evidence of Prometheus scraping activities and kernel messages.

5.0.5 Large media uploads + log churn

Workload Definition:

This experiment simulates the problem of large media uploads and log churn. The workload combines stress-ng --fallocate and stress-ng --file, while Apache and MariaDB run together on Oracle Linux and export metrics to Minikube.

3 workload conditions are used:

- Low load: Apache and MariaDB run with low workload levels on Oracle Linux. Only a small number of file operations are generated, representing normal system activity.
- Medium load: Apache and MariaDB run with moderate workload. stress-ng --fallocate and stress-ng --file run together to simulate media uploads and continuous log generation.
- High load: Apache and MariaDB run under high load. By combining the use of stress-ng --fallocate and stress-ng --file workloads, higher I/O pressure can be generated to simulate a situation where the system is approaching a resource bottleneck.

Variables:

Type	Variables	Purpose
Independent Variables	Apache request rate, MariaDB workload, combined stress-ng --fallocate and stress-ng --file workload	Simulate the real-world workload
Dependent Variables	• Disk I/O throughput • %iowait (iostat) • write latency • disk utilization • bogo ops • bogo ops/s (usr+sys) • bogo ops/s (real time) • real time • usr time	Measure system performance changes

	• sys time	
Controlled Variables	Virtual machine configuration (CPU, memory, disk), Oracle Linux verison, Kubernetes environment, test duration	Keep other factors unchanged and remove interference factors

Observation Tools:

iostat: measure disk I/O throughput and %iowait.

Prometheus: collect CPU, memory, and disk usage.

Grafana: visualize disk utilization, latency, and I/O performance.

Procedure:

1. Ensure Apache and MariaDB services are running on Oracle Linux.
2. Verify both services run normally under the low load condition.
3. Run the workload under the low load condition and collect system metrics.
4. Increase Apache and MariaDB to the medium workload on Oracle Linux.
5. Run stress-ng --fallocate and stress-ng --file together to simulate the real-world scenario.
6. Increase the workload to the high load condition and collect data again.
7. Collect data using iostat, Prometheus, and Grafana for at least 60 seconds.
8. Compare the low, medium and high workload conditions,

Evidence Collection:

- iostat output showing %iowait, disk utilization, and throughput.
- Grafana dashboard screenshots showing disk latency and I/O performance.
- Prometheus metrics for CPU, memory, and disk usage.
- stress-ng execution logs.
- Apache and MariaDB running logs.
- Performance comparison between low, medium and high workload conditions

5.0.6 I/O latency + persistence guarantees

Workload Definition:

This experiment simulates the competition situation of storage resources in a production environment by combining sequential file creation (--file) and forced disk flushing (--fsync). It reproduces the resource contention between Apache access logs and MariaDB transaction commits on Oracle Linux, and quantifies the performance loss caused by hardware synchronization barriers and write amplification.

Three standardized test load levels are adopted instead of the original benchmark divisions:

- Medium load: Simulate medium-scale production Web and database traffic. 16 parallel --file and --fsync worker processes are enabled, which continuously trigger synchronous write blocking to reproduce real-world disk I/O bottlenecks.

Variables:

Type	Variables	Purpose
Independent Variables	Mixed pressure test load (executed in parallel --file and --fsync tasks); Apache/MariaDB client transaction volume	Simulate a multi-tenant high-load storage and resource competition scenario
Dependent Variables	• Disk read/write throughput • write amplification factor • p99 latency peak • transaction data packet loss rate • bogo ops • bogo ops/s (usr+sys) • bogo ops/s (real time) • real time • usr time • sys time	The decline in storage performance when implementing quantitative forced hardware synchronization (disabling page caching)

Controlled Variables	Fixed system configuration, XFS block configuration, Minikube metric collection rules, ≥ 60 seconds stable test duration	Eliminate external environmental interference to isolate the performance of the storage engine

Observation Tools:

iostat: Used to observe changes in critical read/write response times and storage saturation curves.

vmstat: Used to check the kernel's active block layer execution queue to identify processes in the blocked D state (non-interruptible sleep).

Prometheus and Grafana: Used to record and plot the situations of service request interruptions and performance degradation of the storage subsystem during benchmark tests.

Procedure:

1. Deploy the integrated web and database business platform to the Oracle Linux nodes
2. Configure the Minikube metrics and log collection pipeline
3. Collect the benchmark performance of a 60-second low-load system
4. Generate a medium-scale continuous business traffic
5. Concurrently execute the stress-ng file and forced disk writing pressure to simulate the multi-application write amplification scenario
6. Steadily conduct stress testing for ≥ 60 seconds to obtain an effective performance curve
7. Observe the kernel IO blocking status through vmstat and iostat
8. Stop the load and compare the differences between the baseline and the stress test indicators

Evidence Collection:

- Grafana dashboard screenshot: Trends of system throughput and storage utilization
- iostat output: Decrease in disk write performance under compound load
- vmstat indicators: Status of process queue and data on memory buffer exhaustion
- stress-ng log: Record of test execution parameters and timeout
- Apache and MariaDB logs: Failure of application requests due to storage blocking

5.0.7 Metadata heavy workloads

Workload Definition:

This experiment focuses on concurrent file and directory operations. By using stress-ng tool, using two stressors at the same time is possible by combining (-- fallocate) and (-- dir) commands. This allows the experiment to be more realistic since the file and directory operations complement each other.

Two test scenarios have been defined:

- Low Load: Testing with medium workload which is 4 directory workers and 4 file workers.
- Medium Load: Testing with medium workload which is 24 directory workers and 16 file workers.
- High load: Testing with medium workload which is 80 directory workers and 64 file workers.

Variables:

Type	Variables	Purpose
Independent Variables	• 24 directory operations • 16 file operation workers	To test medium workload for file system metadata
Dependent Variables	• Disk I/O Operations • Disk throughput • Disk Wait Time • Disk I/o Utilization • Average queue size • Root File Nodes free • Root Filesystem Used • bogo ops • bogo ops/s (usr+sys) • bogo ops/s (real time) • real time • usr time	To measure the result of the file system performance and metadata operations on operating system.

	• sys time	
Controlled Variables	• Fixed Oracle Linux configurations	To ensure the testing environment is not biased.

Observation Tools:

Prometheus metrics data

Node Exporter scrapped metrics

Grafana dashboard: seeing the real time changes (every 5 seconds) for the dependent variables.

Procedure:

1. Deploy Apache and MariaDB services running natively on Oracle Linux.
2. Deploy Prometheus + Grafana services in Minikube.
3. Confirm that the system operates normally under baseline conditions.
4. Confirm Node Exporter is running on Oracle Linux and successfully visible in the Grafana dashboard before proceeding with any stress testing.
5. Run find, watch and du commands to record the initial directory size.
6. Test each of the stress test in 60 seconds
7. Gradually increase the combined workload from low to medium to high concurrency
8. Give 60 seconds of recover time after each test
9. Observe Prometheus, Node Exporter and Grafana dashboard and collect the result.
10. Compare the system performance between baseline and load test then analyze it.

Evidence Collection:

- Screenshot of the Grafana dashboard will be collected to provide a visual representation of the result.
- The result that are inserted in the performance metrics table will be used as evidence as well to provide clear comparison between benchmark test and load test performance.

- Stress-ng output log will also be collected as evidence to complement the result in the dashboard. It provides evidence of Prometheus scraping activities and kernel messages

5.0.8 Everything is combined

Workload Definition:

This experiment simulates the real world system stress scenario by running all three real world scenarios simultaneously at different condition (Large media uploads with log churn (fallocate + file), I/O latency with persistence guarantees (fsync + file), and Metadata heavy workloads (dir + file)), This will create combined workload where the Oracle Linux XFS file system must handle all the scenario at the same time while Apache and MariaDB run together on Oracle Linux. The Node Exporter will collect performance metrics from Oracle Linux and export them to Minikube, and Grafana will visualize the impact in real time.

The stressing stimulus is done by using these command tools:

- --filename: simulates continuous log file creation, reading, writing, and deletion
- --timeout: duration of stress test
- --fallocate: simulates media uploads
- --fsync: simulates guarantee data persistence
- --dir: simulates rapid directory creation and deletion for metadata heavy workloads

To get more data to prove the hypothesis, there are 3 tests will be apply in this experiment, which are low, medium and high stress test:

- Low load: 4 fallocate operations, 4 fsync operations, 4 directory operations, and 4 file operations running simultaneously, 60 seconds duration
- Medium load: 16 fallocate operations, 32 fsync operations, 24 directory operations, and 16 file operations running simultaneously, 60 seconds duration
- High load: 64 fallocate operations, 64 fsync operations, 80 directory operations, and 64 file operations running simultaneously, 60 seconds duration

Variables:

Type	Variables	Purpose
Independent Variables	Number of concurrent processes for each command (fallocate, file, fsync, dir)	Control file system pressure

Dependent Variables	• Disk utilization • Disk throughput • Read and write latency • Average queue size • bogo ops • bogo ops/s (usr+sys) • bogo ops/s (real time) • real time • usr time • sys time	Measure the performance change and the time taken
Controlled Variables	• Virtual machine configuration • Oracle Linux version • Test duration • Stress-ng version • Same Server: Apache and MariaDB	Make those factors unchanged to make the observation fair

Observation Tools:

Prometheus: used to collect usage data on CPU, memory and disk resources.

Grafana: used to visualize disk utilization, latency changes, and I/O queue.

vmstat: It is used to show I/O queue depth.

Procedure:

1. Deploy Apache and MariaDB services running natively on Oracle Linux.

2. Deploy Prometheus + Grafana services in Minikube.

3. Make the connection to Grafana dashboard and make sure it is able to observe.

4. Confirm that the system operates normally under baseline conditions.

5. Run stress-ng for all four commands simultaneously.

6. Test each of the stress test in 60 seconds

7. Gradually increase the combined workload from low to medium to high concurrency

8. Give 60 seconds of recover time after each test

9. Observe Grafana dashboard and collect the results at each stress level.

10. Compare the system performance between the each of the stress condition and start to analysis.

Evidence Collection:

- Video recording for Grafana dashboard during stress test.
- screenshots the Grafana dashboard to show disk utilization, read and write latency, and I/O queue depth at each stress level.
- The result that are inserted in the performance metrics table will be used as evidence as well to provide clear comparison between benchmark test and load test performance.
- stress-ng terminal logs tells will be collected as evidence to complement the result in the dashboard. It provides evidence of Prometheus scraping activities and kernel messages.

6.0 Runtime Evidence

The runtime evidence is used to prove that our experiment is working and ensuring the data can be collected or observed.

Note to lecturer:

We are unable to test and evaluate MariaDB performance for every experiment due to time constraints. However, Apache (web http requests) and filesystem are tested, evaluated and recorded. The result of our findings is unaffected even without testing MariaDB for every experiment, and the outcome reflects our learning of this subject. Nonetheless, we are willing to include a dedicated MariaDB performance testing in the future as part of our learning process.

Below is the function of each runtime evidence:

1. **System Logs:**

   Proof of command invocation using "sudo journalctl". This shows the system state, user, timestamp and how workloads are executed.

2. **Node exporter metrics:**

   More detailed information about the low level system statistics, used as supporting evidence to evaluate the trends as workloads are increasing.

3. **Prometheus data:**

   Time-series visualisation for specific metrics, used as supporting evidence as well

4. **Grafana Dashboard:**

   Interactive metrics presentation in real time. It provides immediate visibility to any changes, making it easier to spot outliers or anomaly during experiment.

5. **Stress-ng logs**:

   Direct output from the stress-ng tool to provide primary evidence of workload intensity and the stressor behavior.

6. **Apachebench logs:**

   Apachebench log provides the information about Wordpress performance. The log will show the changes in metrics like throughput and requests per second for different levels of stress.

6.1 File allocation

System Logs

```
Jul 31 18:14:42 localhost.localdomain stress-ng[43104]: invoked with 'stress-
ng --fallocate 1 --timeout 20s' by user 1000 'liruochen'
Jul 31 18:14:42 localhost.localdomain stress-ng[43104]: system: 'localhost.lo
caldomain' Linux 6.12.0-203.76.7.5.el9uek.x86_64 #2 SMP PREEMPT_DYNAMIC Mon J
un 15 22:50:26 PDT 2026 x86_64
Jul 31 18:14:42 localhost.localdomain stress-ng[43104]: memory (MB): total 34
37.13, free 1040.72, shared 10.31, buffer 0.02, swap 4020.00, free swap 3663.
46
Jul 31 18:16:02 localhost.localdomain stress-ng[43193]: invoked with 'stress-
ng --fallocate 1 --timeout 20s' by user 1000 'liruochen'
Jul 31 18:16:02 localhost.localdomain stress-ng[43193]: system: 'localhost.lo
caldomain' Linux 6.12.0-203.76.7.5.el9uek.x86_64 #2 SMP PREEMPT_DYNAMIC Mon J
un 15 22:50:26 PDT 2026 x86_64
Jul 31 18:16:02 localhost.localdomain stress-ng[43193]: memory (MB): total 34
37.13, free 1130.40, shared 10.32, buffer 0.01, swap 4020.00, free swap 3663.
46
Jul 31 18:17:22 localhost.localdomain stress-ng[43198]: invoked with 'stress-
ng --fallocate 1 --timeout 20s' by user 1000 'liruochen'
Jul 31 18:17:22 localhost.localdomain stress-ng[43198]: system: 'localhost.lo
caldomain' Linux 6.12.0-203.76.7.5.el9uek.x86_64 #2 SMP PREEMPT_DYNAMIC Mon J
un 15 22:50:26 PDT 2026 x86_64
Jul 31 18:17:22 localhost.localdomain stress-ng[43198]: memory (MB): total 34
37.13, free 1136.12, shared 10.32, buffer 0.01, swap 4020.00, free swap 3663.
46
```

Diagram 6.1-1

Node exporter metrics:

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Curre
nt
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--
# HELP node_disk_written_bytes_total The total number of bytes written succes
sfully.
# TYPE node_disk_written_bytes_total counter
node_disk_written_bytes_total{device="dm-0"} 1.387395072e+09
node_disk_written_bytes_total{device="dm-1"} 3.82500864e+08
node_disk_written_bytes_total{device="dm-2"} 1.065894819328e+12
node_disk_written_bytes_total{device="sda"} 1.067666917888e+12
node_disk_written_bytes_total{device="sr0"} 0
100 88786    0 88786    0     0  2223k      0 --:--:-- --:--:-- --:--:-- 2343
k
```

Diagram 6.1-2: Low disks write byte total

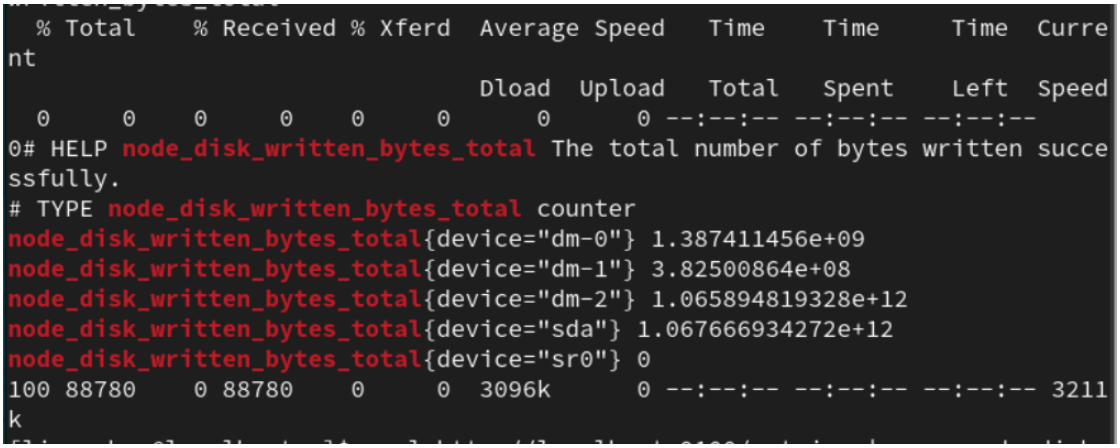


Diagram 6.1-3: Medium disks write byte total

Disk written bytes total: High:

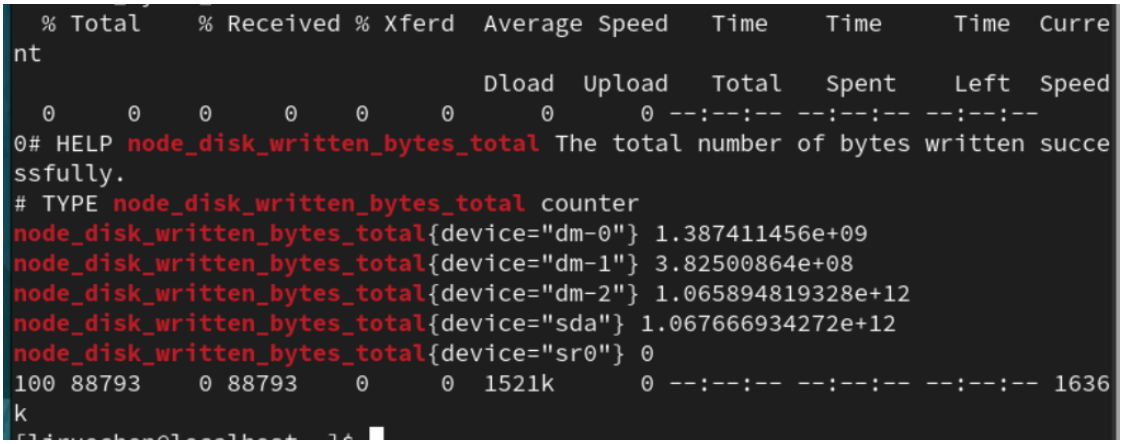


Diagram 6.1-3: High disk write byte total

Prometheus Data

Query:
rate(node_disk_written_bytes_totals[1m])

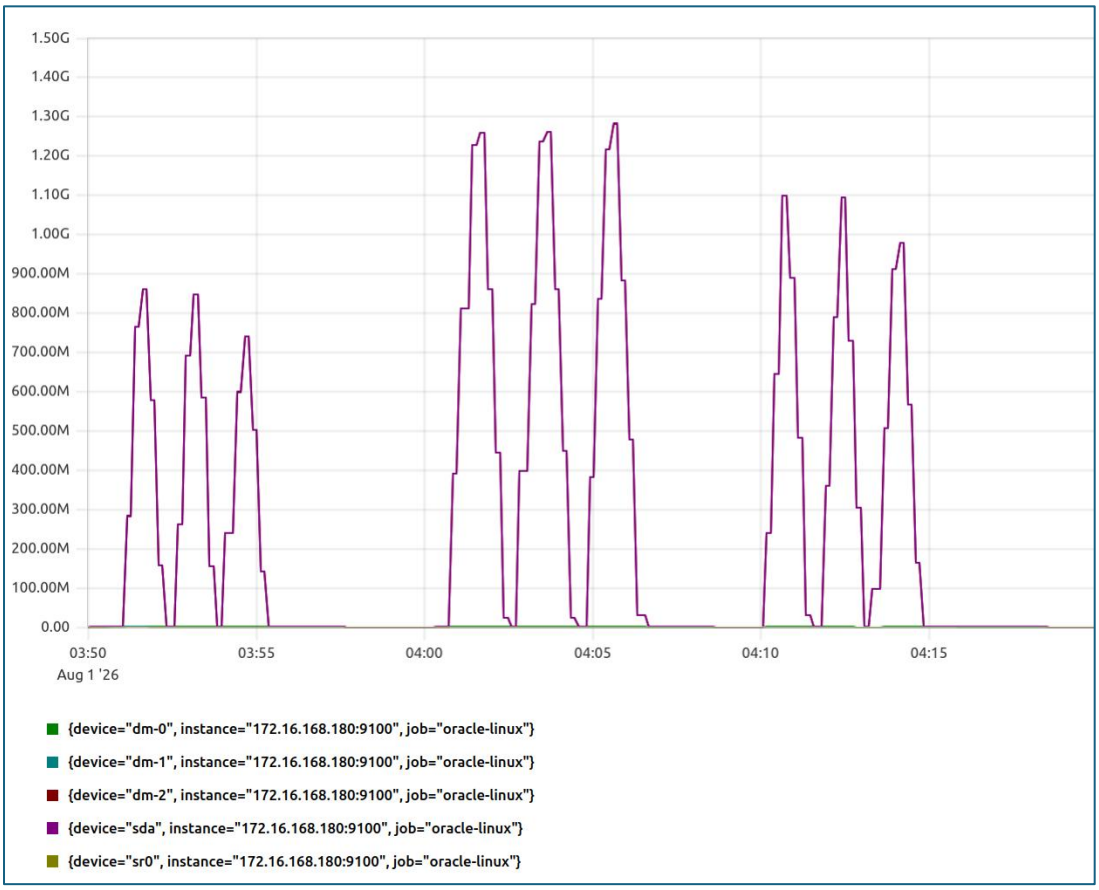


Diagram 6.1-4: Graph for node disk write bytes total

Query:
rate(node_disk_io_time_seconds_total{device=”sda”}[1m])

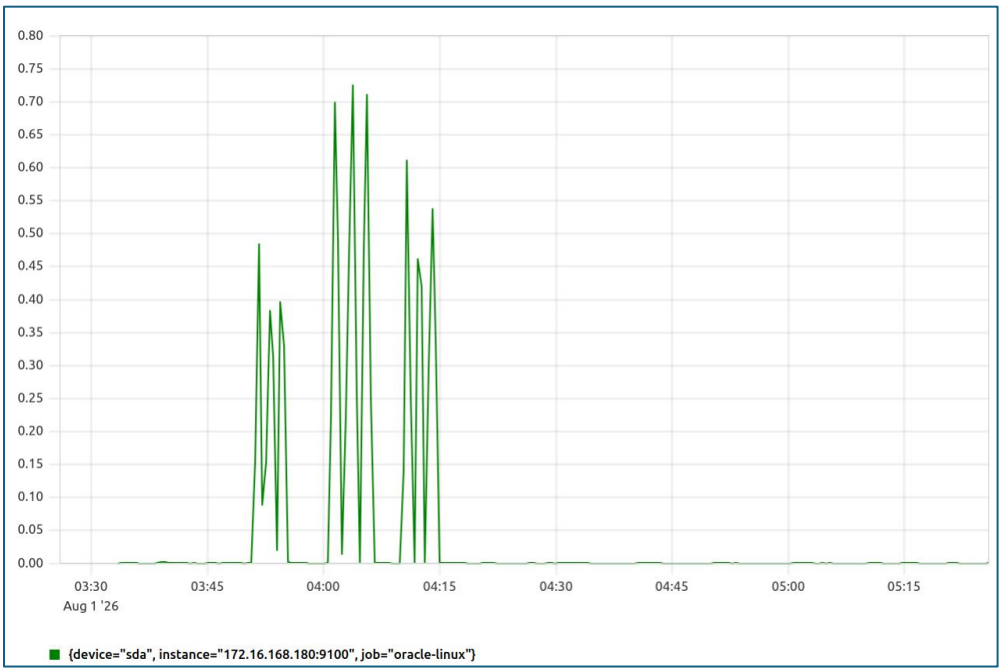


Diagram 6.1-5: Graph for node disk io time seconds total

Grafana Dashboard

Diagram 6.1-6: Disk Write Rate

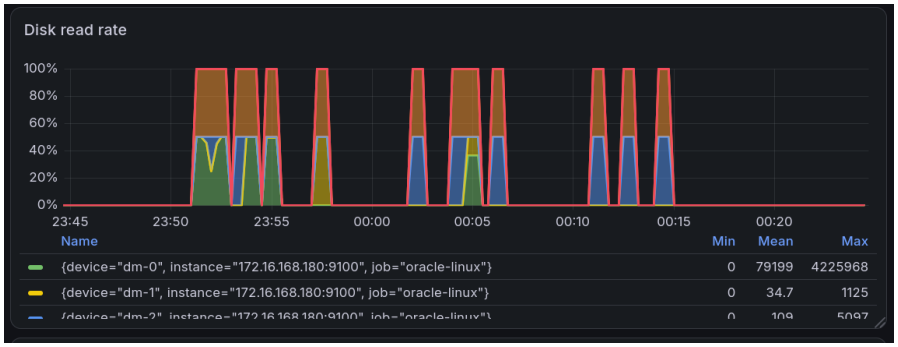


Diagram 6.1-7: Disk Read Rate

Diagram 6.1-8: Disk Latency

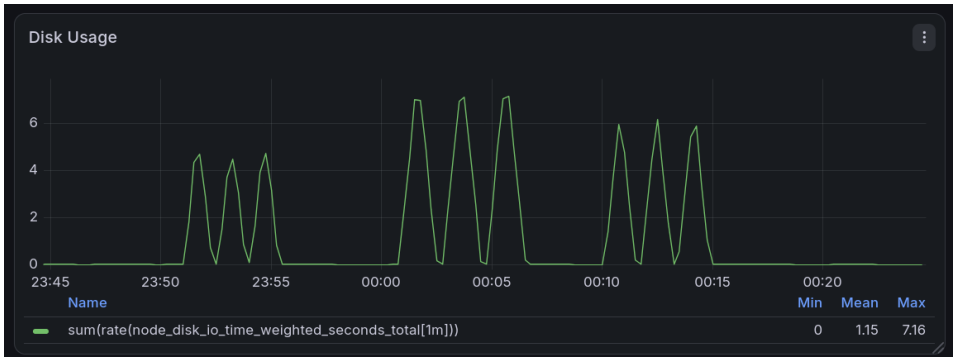


Diagram 6.1-9: Disk Usage

Stress-ng Output Logs

```
stress-ng: info:  [44641] setting to a 1 min run per stressor
stress-ng: info:  [44641] dispatching hogs: 1 fallocate
stress-ng: info:  [44643] fallocate: using 1G file system space per stressor
instance (total 1G of 17.45G available file system space)
stress-ng: metrc: [44641] stressor       bogo ops real time  usr time  sys ti
me   bogo ops/s     bogo ops/s
stress-ng: metrc: [44641]                           (secs)    (secs)    (secs
)   (real time) (usr+sys time)
stress-ng: metrc: [44641] fallocate            71     60.03      0.23     23.
44         1.18           3.00
stress-ng: info:  [44641] skipped: 0
stress-ng: info:  [44641] passed: 1: fallocate (1)
stress-ng: info:  [44641] failed: 0
stress-ng: info:  [44641] metrics untrustworthy: 0
stress-ng: info:  [44641] successful run completed in 1 min
iostat after test
Linux 6.12.0-203.76.7.5.el9uek.x86_64 (localhost.localdomain)   08/01/2026  _
x86_64_ (2 CPU)

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           0.33    0.00    5.22    0.33    0.00   94.12
```

Diagram 6.1-10: Stress-ng log for low stress (fallocate)

```
stress-ng: info:  [44663] setting to a 1 min run per stressor
stress-ng: info:  [44663] dispatching hogs: 1 fallocate
stress-ng: info:  [44665] fallocate: using 1G file system space per stressor
instance (total 1G of 17.45G available file system space)
stress-ng: metrc: [44663] stressor       bogo ops real time  usr time  sys ti
me   bogo ops/s     bogo ops/s
stress-ng: metrc: [44663]                           (secs)    (secs)    (secs
)   (real time) (usr+sys time)
stress-ng: metrc: [44663] fallocate            71     60.04      0.30     23.
48         1.18           2.99
stress-ng: info:  [44663] skipped: 0
stress-ng: info:  [44663] passed: 1: fallocate (1)
stress-ng: info:  [44663] failed: 0
stress-ng: info:  [44663] metrics untrustworthy: 0
stress-ng: info:  [44663] successful run completed in 1 min
iostat after test
Linux 6.12.0-203.76.7.5.el9uek.x86_64 (localhost.localdomain)   08/01/2026  _
x86_64_ (2 CPU)

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           0.33    0.00    5.28    0.35    0.00   94.04
```

Diagram 6.1-11: Stress-ng log for medium stress (fallocate)

```
stress-ng: info:  [44672] setting to a 1 min run per stressor
stress-ng: info:  [44672] dispatching hogs: 1 fallocate
stress-ng: info:  [44674] fallocate: using 1G file system space per stressor
instance (total 1G of 17.45G available file system space)
stress-ng: metrc: [44672] stressor       bogo ops real time  usr time  sys ti
me   bogo ops/s     bogo ops/s
stress-ng: metrc: [44672]                           (secs)    (secs)    (secs
)   (real time) (usr+sys time)
stress-ng: metrc: [44672] fallocate            71     60.07      0.22     23.
70         1.18           2.97
stress-ng: info:  [44672] skipped: 0
stress-ng: info:  [44672] passed: 1: fallocate (1)
stress-ng: info:  [44672] failed: 0
stress-ng: info:  [44672] metrics untrustworthy: 0
stress-ng: info:  [44672] successful run completed in 1 min
iostat after test
Linux 6.12.0-203.76.7.5.el9uek.x86_64 (localhost.localdomain)   08/01/2026  _
x86_64_ (2 CPU)

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           0.33    0.00    5.34    0.36    0.00   93.97
```

Diagram 6.1-12: Stress-ng log for high stress (fallocate)

ApacheBench Logs:

```
Server Software:        Apache/2.4.62
Server Hostname:        localhost
Server Port:            80

Document Path:          /
Document Length:        3539 bytes

Concurrency Level:      10
Time taken for tests:   0.482 seconds
Complete requests:      1000
Failed requests:        0
Non-2xx responses:      1000
Total transferred:      3823000 bytes
HTML transferred:       3539000 bytes
Requests per second:    2074.84 [#/sec] (mean)
Time per request:       4.820 [ms] (mean)
Time per request:       0.482 [ms] (mean, across all concurrent requests)
Transfer rate:          7746.20 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    1   1.5      1      14
Processing:     0    3   2.5      3      24
Waiting:        0    2   2.2      2      22
Total:          1    5   3.0      4      28

Percentage of the requests served within a certain time (ms)
  50%      4
  66%      5
  75%      5
  80%      6
  90%      8
  95%     10
  98%     13
  99%     18
 100%     28 (longest request)
```

Diagram 6.1-13: Benchmarking result (low)

```
Server Software:        Apache/2.4.62
Server Hostname:        localhost
Server Port:            80

Document Path:          /
Document Length:        3539 bytes

Concurrency Level:      50
Time taken for tests:   1.786 seconds
Complete requests:      5000
Failed requests:        0
Non-2xx responses:      5000
Total transferred:      19115000 bytes
HTML transferred:       17695000 bytes
Requests per second:    2799.51 [#/sec] (mean)
Time per request:       17.860 [ms] (mean)
Time per request:       0.357 [ms] (mean, across all concurrent requests)
Transfer rate:          10451.69 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    6   4.3      6      23
Processing:     0   12   5.9     11      52
Waiting:        0    9   4.8      8      46
Total:          1   18   6.9     17      62

Percentage of the requests served within a certain time (ms)
  50%     17
  66%     20
  75%     21
  80%     23
  90%     26
  95%     30
  98%     36
  99%     40
 100%     62 (longest request)
```

Diagram 6.1-14: Benchmarking result (medium)

```
Server Software:        Apache/2.4.62
Server Hostname:        localhost
Server Port:            80

Document Path:          /
Document Length:        3539 bytes

Concurrency Level:      100
Time taken for tests:   3.033 seconds
Complete requests:      10000
Failed requests:        0
Non-2xx responses:      10000
Total transferred:      38230000 bytes
HTML transferred:       35390000 bytes
Requests per second:    3297.15 [#/sec] (mean)
Time per request:       30.329 [ms] (mean)
Time per request:       0.303 [ms] (mean, across all concurrent requests)
Transfer rate:          12309.57 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0   12   5.5     12      37
Processing:     2   18   7.6     17      57
Waiting:        0   14   6.3     13      53
Total:          6   30   8.5     30      72

Percentage of the requests served within a certain time (ms)
  50%     30
  66%     32
  75%     35
  80%     37
  90%     41
  95%     45
  98%     50
  99%     54
 100%     72 (longest request)
```

Diagram 6.1-15: Benchmarking result (high)

6.2 Flushes data from a file to a memory

System Logs

```
Jul 30 05:45:15 VM stress-ng: invoked with 'stress-n' by user 0
Jul 30 05:45:15 VM stress-ng: system: 'VM' Linux 5.15.0-139-generic #149~20.04.
1-Ubuntu SMP Wed Apr 16 08:29:56 UTC 2025 x86_64
Jul 30 05:45:15 VM stress-ng: memory (MB): total 7937.56, free 253.07, shared 3
1.55, buffer 378.29, swap 2048.00, free swap 2046.73
Jul 30 05:47:15 VM stress-ng: invoked with 'stress-n' by user 0
Jul 30 05:47:15 VM stress-ng: system: 'VM' Linux 5.15.0-139-generic #149~20.04.
1-Ubuntu SMP Wed Apr 16 08:29:56 UTC 2025 x86_64
Jul 30 05:47:15 VM stress-ng: memory (MB): total 7937.56, free 219.52, shared 3
1.55, buffer 429.33, swap 2048.00, free swap 2046.73
Jul 30 05:49:15 VM stress-ng: invoked with 'stress-n' by user 0
Jul 30 05:49:15 VM stress-ng: system: 'VM' Linux 5.15.0-139-generic #149~20.04.
1-Ubuntu SMP Wed Apr 16 08:29:56 UTC 2025 x86_64
Jul 30 05:49:15 VM stress-ng: memory (MB): total 7937.56, free 147.76, shared 3
1.55, buffer 458.22, swap 2048.00, free swap 2046.73
```

Diagram 6.2-1

Node Exporter Metrics

```
node_disk_io_now{device="sda"} 0
node_disk_io_now{device="sr0"} 0
node_disk_io_time_seconds_total{device="sda"} 61.672000000000004
node_disk_io_time_seconds_total{device="sr0"} 0.1
node_disk_write_time_seconds_total{device="sda"} 57.106
node_disk_write_time_seconds_total{device="sr0"} 0
node_disk_writes_completed_total{device="sda"} 18964
node_disk_writes_completed_total{device="sr0"} 0
```

Diagram 6.2-2

Prometheus Data

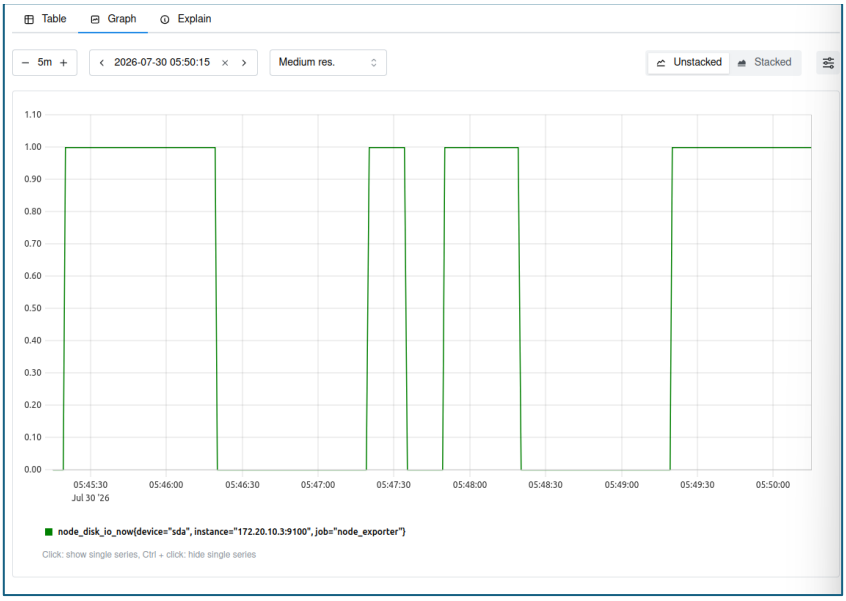


Diagram 6.2-3: Prometheus query results

Grafana Dashboard

Diagram 6.2-4: Disk IOps

Diagram 6.2-5: Disk Throughput

Diagram 6.2-6: Filesystem Space Available

Diagram 6.2-7: Filesystem Used

Diagram 6.2-8: Disk I/O Utilization

Diagram 6.2-9: Pressure Stall Information

Stress-ng Output Logs

```
>>> [LOW LOAD] Running stress-ng --sync-file with 8 threads for 60s
stress-ng: info:  [4476] dispatching hogs: 8 sync-file
stress-ng: info:  [4476] successful run completed in 60.01s (1 min, 0.01 secs)
stress-ng: info:  [4476] stressor       bogo ops real time  usr time  sys time
 bogo ops/s   bogo ops/s
stress-ng: info:  [4476]                           (secs)    (secs)    (secs)
(real time) (usr+sys time)
stress-ng: info:  [4476] sync-file         15712     60.00      4.94     16.21
     261.85       742.88
>>> [LOW LOAD] Test finished. Recovering for 60s...
```

Diagram 6.2-10: Low Stress Test (8 sync-file)

```
>>> [MEDIUM LOAD] Running stress-ng --sync-file with 32 threads for 60s
stress-ng: info:  [4510] dispatching hogs: 32 sync-file
stress-ng: info:  [4510] successful run completed in 60.02s (1 min, 0.02 secs)
stress-ng: info:  [4510] stressor       bogo ops real time  usr time  sys time
 bogo ops/s   bogo ops/s
stress-ng: info:  [4510]                           (secs)    (secs)    (secs)
(real time) (usr+sys time)
stress-ng: info:  [4510] sync-file         58620     60.01      4.16     16.62
     976.88      2820.98
>>> [MEDIUM LOAD] Test finished. Recovering for 60s...
```

Diagram 6.2-11: Medium Stress Test (32 sync-file)

```
>>> [HIGH LOAD] Running stress-ng --sync-file with 64 threads for 60s
stress-ng: info:  [4581] dispatching hogs: 64 sync-file
stress-ng: info:  [4581] successful run completed in 60.02s (1 min, 0.02 secs)
stress-ng: info:  [4581] stressor       bogo ops real time  usr time  sys time
 bogo ops/s   bogo ops/s
stress-ng: info:  [4581]                           (secs)    (secs)    (secs)
(real time) (usr+sys time)
stress-ng: info:  [4581] sync-file        123784     60.01      3.75     20.28
    2062.81      5151.23
>>> [HIGH LOAD] Test finished. Recovering for 60s...
===== All fsync stress tests completed. Logs saved in ./stress-ng-logs/fsync ===
__
```

Diagram 6.2-12: High Stress Test (64 sync-file)

Apachebench logs

```
tee: logs/apache_low.log: No such file or directory
This is ApacheBench, Version 2.3 <$Revision: 1843412 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient)
Completed 100 requests
Completed 200 requests
Completed 300 requests
Completed 400 requests
Completed 500 requests
Completed 600 requests
Completed 700 requests
Completed 800 requests
Completed 900 requests
Completed 1000 requests
Finished 1000 requests


Server Software:        Apache/2.4.41
Server Hostname:        localhost
Server Port:            80

Document Path:          /
Document Length:        10918 bytes

Concurrency Level:      10
Time taken for tests:   0.171 seconds
Complete requests:      1000
Failed requests:        0
Total transferred:      11192000 bytes
HTML transferred:       10918000 bytes
Requests per second:    5835.50 [#/sec] (mean)
Time per request:       1.714 [ms] (mean)
Time per request:       0.171 [ms] (mean, across all concurrent requests)
Transfer rate:          63780.16 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.2      0       3
Processing:     0    2   1.4      1       9
Waiting:        0    1   1.3      1       7
Total:          0    2   1.4      1       9

Percentage of the requests served within a certain time (ms)
  50%      1
  66%      2
  75%      2
  80%      2
  90%      4
  95%      5
  98%      6
  99%      6
 100%      9 (longest request)
```

Diagram 6.2-13: Benchmarking result (low)

```
tee: logs/apache_medium.log: No such file or directory
This is ApacheBench, Version 2.3 <$Revision: 1843412 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient)
Completed 500 requests
Completed 1000 requests
Completed 1500 requests
Completed 2000 requests
Completed 2500 requests
Completed 3000 requests
Completed 3500 requests
Completed 4000 requests
Completed 4500 requests
Completed 5000 requests
Finished 5000 requests


Server Software:        Apache/2.4.41
Server Hostname:        localhost
Server Port:            80

Document Path:          /
Document Length:        10918 bytes

Concurrency Level:      50
Time taken for tests:   0.621 seconds
Complete requests:      5000
Failed requests:        0
Total transferred:      55960000 bytes
HTML transferred:       54590000 bytes
Requests per second:    8049.02 [#/sec] (mean)
Time per request:       6.212 [ms] (mean)
Time per request:       0.124 [ms] (mean, across all concurrent requests)
Transfer rate:          87973.22 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.1      0       2
Processing:     0    6   3.6      5      28
Waiting:        0    5   3.1      5      23
Total:          0    6   3.6      5      28

Percentage of the requests served within a certain time (ms)
  50%      5
  66%      7
  75%      8
  80%      8
  90%     11
  95%     13
  98%     16
  99%     19
 100%     28 (longest request)
```

Diagram 6.2-14: Benchmarking result (medium)

```
tee: logs/apache_high.log: No such file or directory
This is ApacheBench, Version 2.3 <$Revision: 1843412 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient)
Completed 1000 requests
Completed 2000 requests
Completed 3000 requests
Completed 4000 requests
Completed 5000 requests
Completed 6000 requests
Completed 7000 requests
Completed 8000 requests
Completed 9000 requests
Completed 10000 requests
Finished 10000 requests


Server Software:        Apache/2.4.41
Server Hostname:        localhost
Server Port:            80

Document Path:          /
Document Length:        10918 bytes

Concurrency Level:      100
Time taken for tests:   1.099 seconds
Complete requests:      10000
Failed requests:        0
Total transferred:      111920000 bytes
HTML transferred:       109180000 bytes
Requests per second:    9095.31 [#/sec] (mean)
Time per request:       10.995 [ms] (mean)
Time per request:       0.110 [ms] (mean, across all concurrent requests)
Transfer rate:          99408.87 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.2      0       3
Processing:     0   11   5.3     10      59
Waiting:        0   10   4.7      9      48
Total:          1   11   5.3     10      59

Percentage of the requests served within a certain time (ms)
  50%     10
  66%     12
  75%     13
  80%     14
  90%     17
  95%     21
  98%     26
  99%     30
 100%     59 (longest request)
```

Diagram 6.2-15: Benchmarking result (high)

6.3 Directory I/O stress testing

System Logs

```
Aug 01 07:24:34 localhost.localdomain stress-ng[4980]: invoked with 'stress-ng --dir 4 --timeout 60s --metrics-brief --temp-path /var/www/html/dummydirectory' by user 1000 'dihrran'
Aug 01 07:24:34 localhost.localdomain stress-ng[4980]: system: 'localhost.localdomain' Linux 6.12.0-203.76.7.1.el10uek.x86_64 #1 SMP PREEMPT_DYNAMIC Mon Jun  1 14:32:12 PDT 2026 x86_64
Aug 01 07:24:34 localhost.localdomain stress-ng[4980]: memory (MB): total 3457.36, free 1359.61, shared 21.20, buffer 4.43, swap 4040.00, free swap 4040.00
Aug 01 07:26:58 localhost.localdomain stress-ng[4998]: invoked with 'stress-ng --dir 24 --timeout 60s --metrics-brief --temp-path /var/www/html/dummydirectory' by user 1000 'dihrran'
Aug 01 07:26:58 localhost.localdomain stress-ng[4998]: system: 'localhost.localdomain' Linux 6.12.0-203.76.7.1.el10uek.x86_64 #1 SMP PREEMPT_DYNAMIC Mon Jun  1 14:32:12 PDT 2026 x86_64
Aug 01 07:26:58 localhost.localdomain stress-ng[4998]: memory (MB): total 3457.36, free 1398.34, shared 21.20, buffer 4.43, swap 4040.00, free swap 4040.00
Aug 01 07:29:10 localhost.localdomain stress-ng[5053]: invoked with 'stress-ng --dir 80 --timeout 60s --metrics-brief --temp-path /var/www/html/dummydirectory' by user 1000 'dihrran'
Aug 01 07:29:10 localhost.localdomain stress-ng[5053]: system: 'localhost.localdomain' Linux 6.12.0-203.76.7.1.el10uek.x86_64 #1 SMP PREEMPT_DYNAMIC Mon Jun  1 14:32:12 PDT 2026 x86_64
Aug 01 07:29:10 localhost.localdomain stress-ng[5053]: memory (MB): total 3457.36, free 1394.19, shared 21.20, buffer 4.43, swap 4040.00, free swap 4040.00
Aug 01 07:31:52 localhost.localdomain kernel: 23:31:51.786165 Timer    VBoxDRMClient: push screen layout data of 1 display(s) to DRM stack, fPartialLayout=false, rc=VINF_SUC
Aug 01 07:31:52 localhost.localdomain kernel: 23:31:52.284623 Timer    VBoxDRMClient: push screen layout data of 1 display(s) to DRM stack, fPartialLayout=false, rc=VINF_SUC
~
```

Diagram 6.3-1

Node Exporter Metrics:

The Node Exporter Metrics for this experiment is not included since the result is recorded at the end of the experiment, making the metrics accumulated from low stress test to high stress test. As a result the metrics cannot show the changes between increased workloads.

Prometheus Data

Query:

(node_filesystem_size_bytes{job="oracle-linux"} -
node_filesystem_avail_bytes{job="oracle-linux"})

/ node_filesystem_size_bytes{job="oracle-linux"} * 100

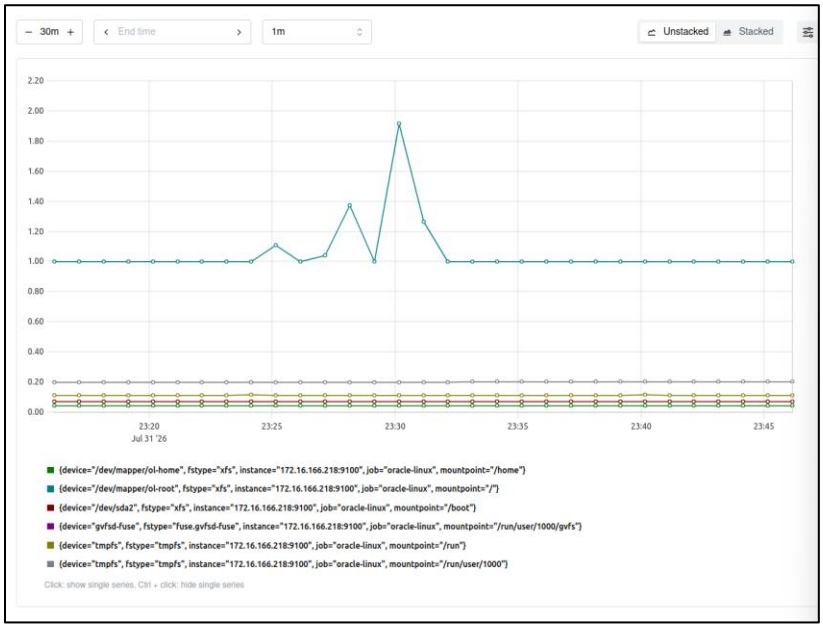


Diagram 6.3-2: File usage percentage by Prometheus (normalized)

Grafana Dashboard

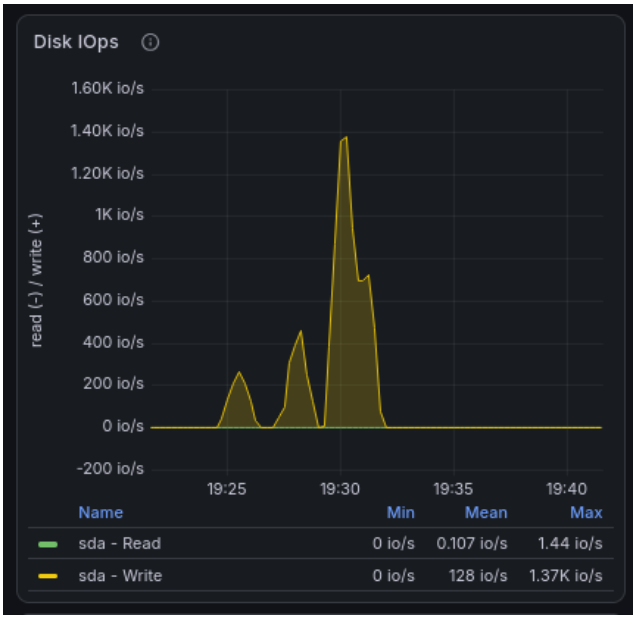


Diagram 6.3-3: Disk IOps

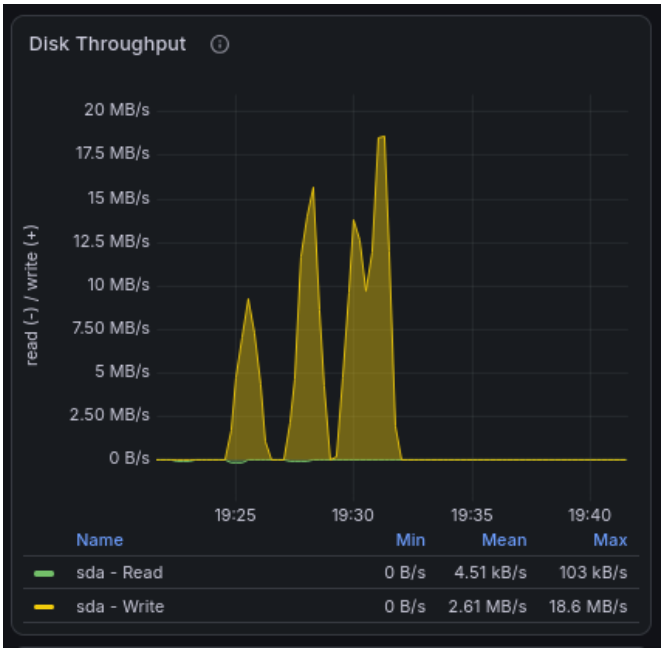


Diagram 6.3-4: Disk Throughput

Diagram 6.3-5: Disk Average Wait Time

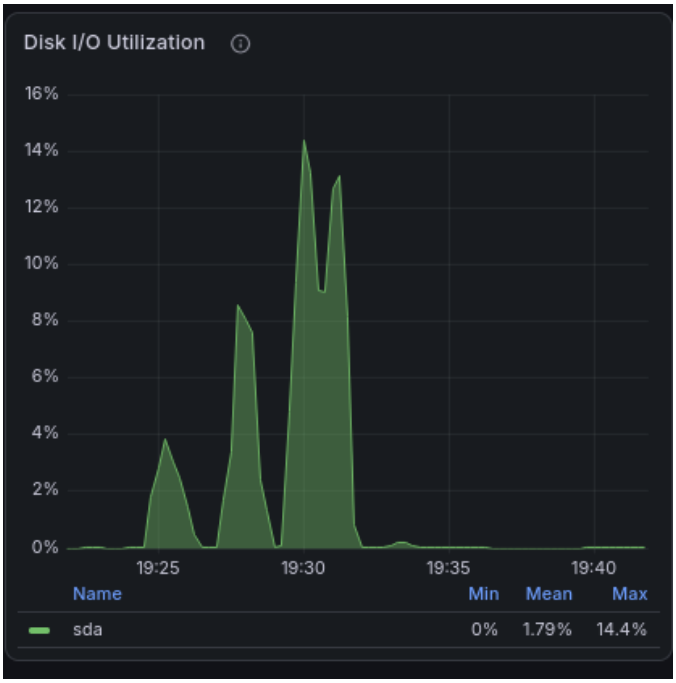


Diagram 6.3-6: Disk I/O Utilization

Diagram 6.3-7: Average Queue Size

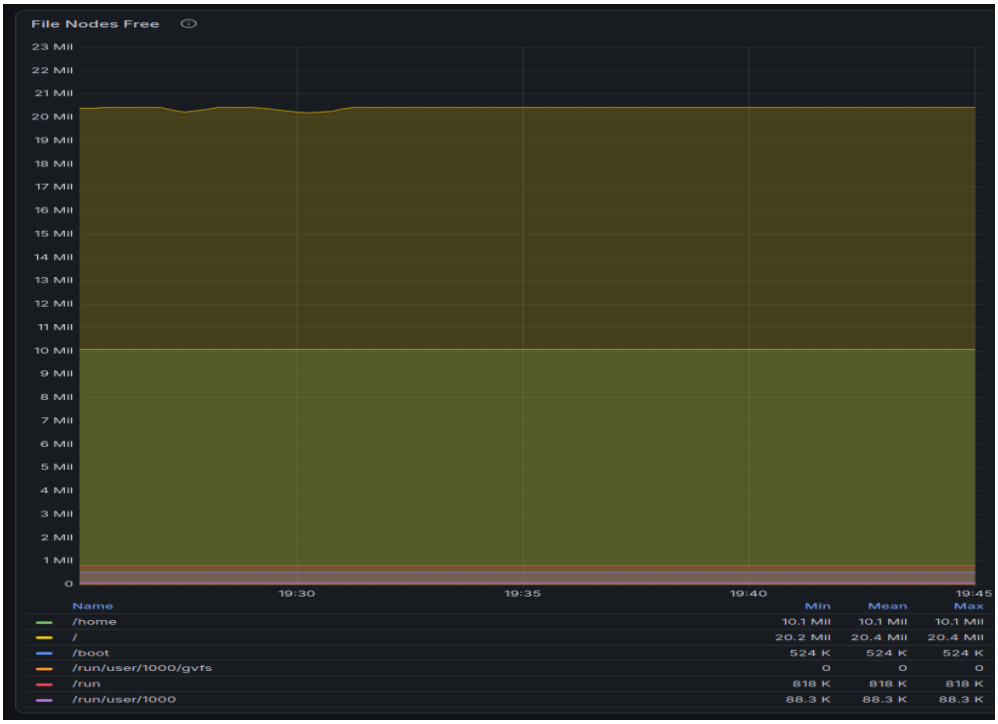


Diagram 6.3-8: File Nodes Free

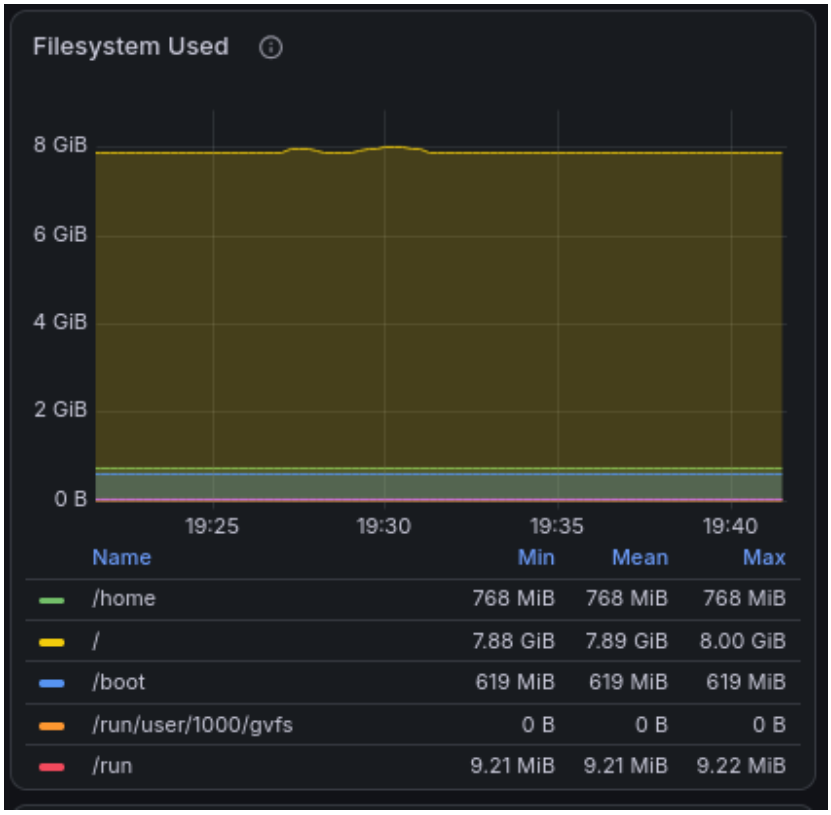


Diagram 6.3-9: Filesystem Used

Stress-ng Output Logs

```
stress-ng: info:  [4980] setting to a 1 min run per stressor
stress-ng: info:  [4980] dispatching hogs: 4 dir
stress-ng: metrc: [4980] stressor       bogo ops real time  usr time  sys time   bogo ops/s     bog
o ops/s
stress-ng: metrc: [4980]                           (secs)    (secs)    (secs)   (real time) (usr+sy
s time)
stress-ng: metrc: [4980] dir              448870     60.84      6.50    101.55      7378.36
4154.24
stress-ng: info:  [4980] skipped: 0
stress-ng: info:  [4980] passed: 4: dir (4)
stress-ng: info:  [4980] failed: 0
stress-ng: info:  [4980] metrics untrustworthy: 0
stress-ng: info:  [4980] successful run completed in 1 min, 1.07 sec
FINIHSED LOW STRESS TEST FOR DIRECTORY --dir
```

Diagram 6.3-10: Low Directory Stress Test (4 dir)

```
stress-ng: info:  [4998] setting to a 1 min run per stressor
stress-ng: info:  [4998] dispatching hogs: 24 dir
stress-ng: metrc: [4998] stressor       bogo ops real time  usr time  sys time   bogo ops/s     bog
o ops/s
stress-ng: metrc: [4998]                           (secs)    (secs)    (secs)   (real time) (usr+sy
s time)
stress-ng: metrc: [4998] dir              204824     68.56      9.46    117.26      2987.52
1616.31
stress-ng: info:  [4998] skipped: 0
stress-ng: info:  [4998] passed: 24: dir (24)
stress-ng: info:  [4998] failed: 0
stress-ng: info:  [4998] metrics untrustworthy: 0
stress-ng: info:  [4998] successful run completed in 1 min, 9.98 secs
FINIHSED MEDIUM STRESS TEST FOR DIRECTORY --dir
```

Diagram 6.3-11: Medium Directory Stress Test (24 dir)

```
stress-ng: info:  [5053] setting to a 1 min run per stressor
stress-ng: info:  [5053] dispatching hogs: 80 dir
stress-ng: metrc: [5053] stressor       bogo ops real time  usr time  sys time   bogo ops/s     bog
o ops/s
stress-ng: metrc: [5053]                           (secs)    (secs)    (secs)   (real time) (usr+sy
s time)
stress-ng: metrc: [5053] dir              221847    105.09     16.08    184.81      2110.92
1104.33
stress-ng: info:  [5053] skipped: 0
stress-ng: info:  [5053] passed: 80: dir (80)
stress-ng: info:  [5053] failed: 0
stress-ng: info:  [5053] metrics untrustworthy: 0
stress-ng: info:  [5053] successful run completed in 1 min, 52.91 secs
FINIHSED HIGH STRESS TEST FOR DIRECTORY --dir
```

Diagram 6.3-12: High Directory Stress Test (80 dir)

ApacheBench Logs:

```
Server Software:        Apache/2.4.63
Server Hostname:        localhost
Server Port:            80

Document Path:          /wordpress/
Document Length:        66517 bytes

Concurrency Level:      10
Time taken for tests:   7.602 seconds
Complete requests:      100
Failed requests:        0
Total transferred:      6678800 bytes
HTML transferred:       6651700 bytes
Requests per second:    13.15 [#/sec] (mean)
Time per request:       760.222 [ms] (mean)
Time per request:       76.022 [ms] (mean, across all concurrent requests)
Transfer rate:          857.94 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.7      0       5
Processing:   131  733 264.1    674    1439
Waiting:      118  710 255.1    651    1368
Total:        131  733 264.1    674    1439

Percentage of the requests served within a certain time (ms)
  50%    674
  66%    756
  75%    912
  80%   1003
  90%   1134
  95%   1256
  98%   1347
  99%   1439
 100%   1439 (longest request)
```

Diagram 6.3-13: Benchmarking result (low)

```
Server Software:        Apache/2.4.63
Server Hostname:        localhost
Server Port:            80

Document Path:          /wordpress/
Document Length:        66517 bytes

Concurrency Level:      10
Time taken for tests:   7.254 seconds
Complete requests:      100
Failed requests:        0
Total transferred:      6678800 bytes
HTML transferred:       6651700 bytes
Requests per second:    13.79 [#/sec] (mean)
Time per request:       725.382 [ms] (mean)
Time per request:       72.538 [ms] (mean, across all concurrent requests)
Transfer rate:          899.15 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    1   3.6      0      13
Processing:    92  708 256.6    619    1238
Waiting:       75  688 250.4    603    1205
Total:         92  709 258.5    619    1251

Percentage of the requests served within a certain time (ms)
  50%    619
  66%    712
  75%   1010
  80%   1031
  90%   1109
  95%   1196
  98%   1225
  99%   1251
 100%   1251 (longest request)
```

Diagram 6.3-14: Benchmarking result (Medium)

```
Server Software:        Apache/2.4.63
Server Hostname:        localhost
Server Port:            80

Document Path:          /wordpress/
Document Length:        66517 bytes

Concurrency Level:      10
Time taken for tests:   7.874 seconds
Complete requests:      100
Failed requests:        0
Total transferred:      6678800 bytes
HTML transferred:       6651700 bytes
Requests per second:    12.70 [#/sec] (mean)
Time per request:       787.390 [ms] (mean)
Time per request:       78.739 [ms] (mean, across all concurrent requests)
Transfer rate:          828.34 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.3      0       1
Processing:   113  760 227.3    812    1100
Waiting:        0  734 222.6    777    1099
Total:        113  760 227.3    812    1100

Percentage of the requests served within a certain time (ms)
  50%    812
  66%    886
  75%    911
  80%    934
  90%    979
  95%   1001
  98%   1098
  99%   1100
 100%   1100 (longest request)
```

Diagram 6.3-15: Benchmarking result (high)

6.4 File I/O stress testing

System Logs

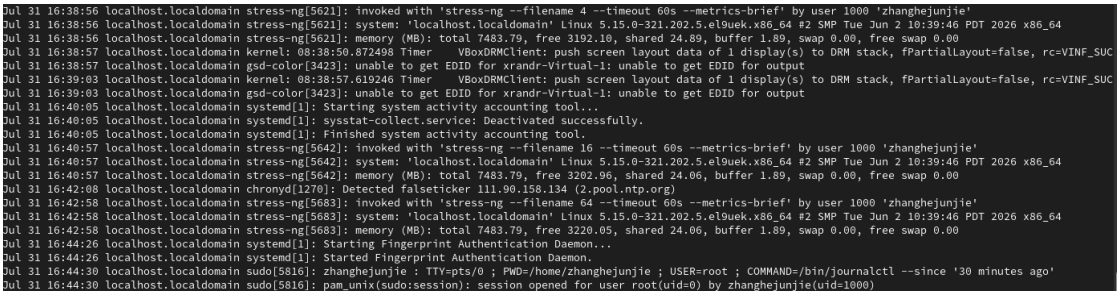


Diagram 6.4-1: system logs

Node Exporter Metrics

```
# TYPE node_xfs_write_calls_total counter
node_xfs_write_calls_total{device="dm-0"} 276216
node_xfs_write_calls_total{device="dm-1"} 29354
node_xfs_write_calls_total{device="sda1"} 1
```

Diagram 6.4-2: Node xfs write calls total (Low stress)

```
# TYPE node_xfs_write_calls_total counter
node_xfs_write_calls_total{device="dm-0"} 276245
node_xfs_write_calls_total{device="dm-1"} 35754
node_xfs_write_calls_total{device="sda1"} 1
```

Diagram 6.4-3: Node xfs write calls total (Medium stress)

```
# TYPE node_xfs_write_calls_total counter
node_xfs_write_calls_total{device="dm-0"} 276247
node_xfs_write_calls_total{device="dm-1"} 43547
node_xfs_write_calls_total{device="sda1"} 1
```

Diagram 6.4-4: Node xfs write calls total (High stress)

Prometheus Data

Query:

(rate(node_disk_io_time_seconds_total{job="oracle-linux"}[9m]))

This graph shows the rate of time disk spent doing I/O work per second

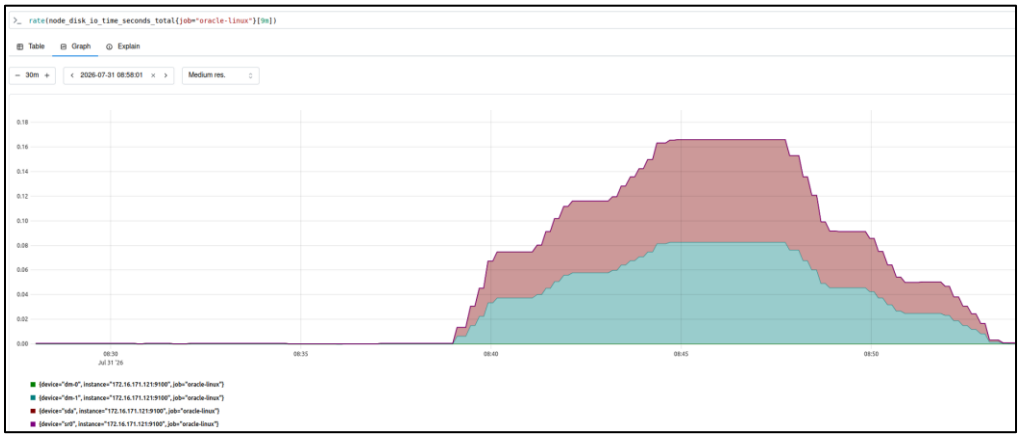


Diagram 6.4-5: Prometheus disk I/O time

Grafana Dashboard *(from 3: 26 to 3.33)*

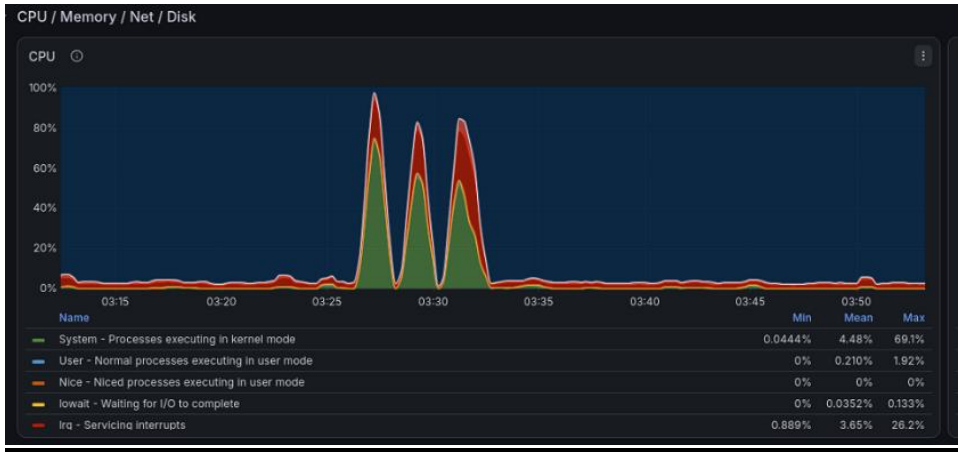


Diagram 6.4-6: CPU usage

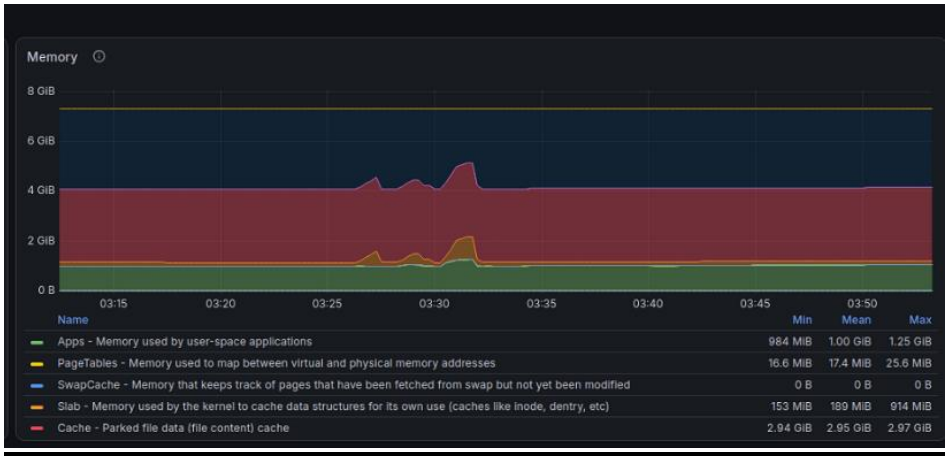


Diagram 6.4-7: Memory usage

Diagram 6.4-8: Disk Throughput

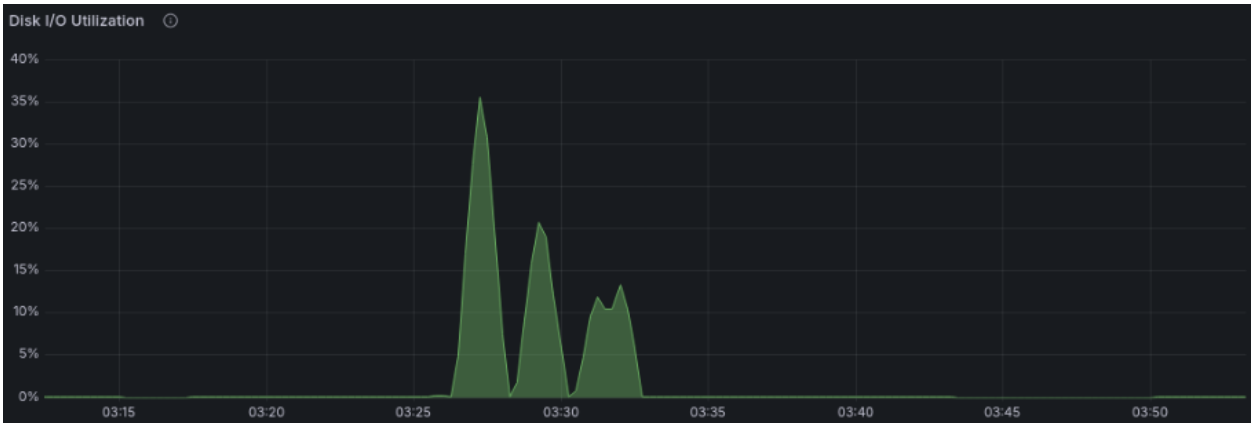


Diagram 6.4-9: Disk I/O Utilization

Diagram 6.4-10: Disk Average Wait Time

Diagram 6.4-11: Average Queue Size

Stress-ng Output Logs

```
Low stress test & result:
stress-ng: info:  [5189] setting to a 1 min run per stressor
stress-ng: info:  [5189] dispatching hogs: 4 filename
stress-ng: metrc: [5189] stressor       bogo ops real time  usr time  sys time   bogo ops/s     bogo ops/s
stress-ng: metrc: [5189]                           (secs)    (secs)    (secs)   (real time) (usr+sys time)
stress-ng: metrc: [5189] filename         160040     60.72      1.59     76.46      2635.88        2050.59
stress-ng: info:  [5189] skipped: 0
stress-ng: info:  [5189] passed: 4: filename (4)
stress-ng: info:  [5189] failed: 0
stress-ng: info:  [5189] metrics untrustworthy: 0
stress-ng: info:  [5189] successful run completed in 1 min, 1.17 sec
End time: Fri Jul 31 03:27:16 PM +08 2026
```

Diagram 6.4-12: Low File Stress Test (4 filename)

```
Start time: Fri Jul 31 03:28:16 PM +08 2026
Medium stress test & result:
stress-ng: info:  [5206] setting to a 1 min run per stressor
stress-ng: info:  [5206] dispatching hogs: 16 filename
stress-ng: metrc: [5206] stressor       bogo ops real time  usr time  sys time   bogo ops/s     bogo ops/s
stress-ng: metrc: [5206]                           (secs)    (secs)    (secs)   (real time) (usr+sys time)
stress-ng: metrc: [5206] filename          48525     61.51      1.97     48.13       788.86         968.53
stress-ng: info:  [5206] skipped: 0
stress-ng: info:  [5206] passed: 16: filename (16)
stress-ng: info:  [5206] failed: 0
stress-ng: info:  [5206] metrics untrustworthy: 0
stress-ng: info:  [5206] successful run completed in 1 min, 1.67 sec
End time: Fri Jul 31 03:29:18 PM +08 2026
```

Diagram 6.4-13: Medium File Stress Test (16 filename)

```
Start time: Fri Jul 31 03:30:18 PM +08 2026
High stress test & result:
stress-ng: info:  [5249] setting to a 1 min run per stressor
stress-ng: info:  [5249] dispatching hogs: 64 filename
stress-ng: metrc: [5249] stressor       bogo ops real time  usr time  sys time   bogo ops/s     bogo ops/s
stress-ng: metrc: [5249]                           (secs)    (secs)    (secs)   (real time) (usr+sys time)
stress-ng: metrc: [5249] filename          29225     87.49      2.36     56.40       334.05         497.41
stress-ng: info:  [5249] skipped: 0
stress-ng: info:  [5249] passed: 64: filename (64)
stress-ng: info:  [5249] failed: 0
stress-ng: info:  [5249] metrics untrustworthy: 0
stress-ng: info:  [5249] successful run completed in 1 min, 29.05 secs
End time: Fri Jul 31 03:31:47 PM +08 2026
```

Diagram 6.4-14: High File Stress Test (64 filename)

Apache Bench Logs:

```
Server Software:        Apache/2.4.62
Server Hostname:        localhost
Server Port:            80

Document Path:          /wordpress/
Document Length:        92427 bytes

Concurrency Level:      10
Time taken for tests:   5.397 seconds
Complete requests:      100
Failed requests:        0
Total transferred:      9269800 bytes
HTML transferred:       9242700 bytes
Requests per second:    18.53 [#/sec] (mean)
Time per request:       539.652 [ms] (mean)
Time per request:       53.965 [ms] (mean, across all concurrent requests)
Transfer rate:          1677.48 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.1      0       1
Processing:   145  514 111.4    537     785
Waiting:       78  491 113.6    509     746
Total:        145  515 111.4    537     785

Percentage of the requests served within a certain time (ms)
  50%    537
  66%    575
  75%    599
  80%    607
  90%    648
  95%    683
  98%    706
  99%    785
 100%    785 (longest request)
```

Diagram 6.4-15: Benchmarking result (low)

```
Server Software:        Apache/2.4.62
Server Hostname:        localhost
Server Port:            80

Document Path:          /wordpress/
Document Length:        92427 bytes

Concurrency Level:      10
Time taken for tests:   11.080 seconds
Complete requests:      100
Failed requests:        0
Total transferred:      9269800 bytes
HTML transferred:       9242700 bytes
Requests per second:    9.02 [#/sec] (mean)
Time per request:       1108.042 [ms] (mean)
Time per request:       110.804 [ms] (mean, across all concurrent requests)
Transfer rate:          816.98 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.1      0       1
Processing:   448 1062 270.1   1042    1699
Waiting:      246 1002 270.6    968    1650
Total:        448 1062 270.1   1042    1699

Percentage of the requests served within a certain time (ms)
  50%   1042
  66%   1152
  75%   1251
  80%   1326
  90%   1484
  95%   1575
  98%   1677
  99%   1699
 100%   1699 (longest request)
```

Diagram 6.4-16: Benchmarking result (medium)

```
Benchmarking localhost (be patient).....done


Server Software:        Apache/2.4.62
Server Hostname:        localhost
Server Port:            80

Document Path:          /wordpress/
Document Length:        92427 bytes

Concurrency Level:      10
Time taken for tests:   23.926 seconds
Complete requests:      100
Failed requests:        0
Total transferred:      9269800 bytes
HTML transferred:       9242700 bytes
Requests per second:    4.18 [#/sec] (mean)
Time per request:       2392.575 [ms] (mean)
Time per request:       239.258 [ms] (mean, across all concurrent requests)
Transfer rate:          378.36 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.4      0       3
Processing:   270 2329 981.5   2437    4322
Waiting:      178 2254 955.4   2360    4190
Total:        270 2330 981.6   2437    4322

Percentage of the requests served within a certain time (ms)
  50%   2437
  66%   2858
  75%   3224
  80%   3360
  90%   3620
  95%   3936
  98%   4146
  99%   4322
 100%   4322 (longest request)
```

Diagram 6.4-17: Benchmarking result (high)

6.5 Large metadata uploads + log churn

System Logs

```
Jul 31 16:56:29 localhost.localdomain stress-ng[42512]: invoked with 'stress-ng --fallocate 1 --timeout 20s' by user 1000 'liruochen'
Jul 31 16:56:29 localhost.localdomain stress-ng[42512]: system: 'localhost.localdomain' Linux 6.12.0-203.76.7.5.el9uek.x86_64 #2 SMP PREEMPT_DYNAMIC Mon Jun 15 22:50:26 PDT 2026 x86_64
Jul 31 16:56:29 localhost.localdomain stress-ng[42512]: memory (MB): total 3437.13, free 1548.65, shared 10.26, buffer 0.02, swap 4020.00, free swap 3641.42
Jul 31 16:59:37 localhost.localdomain stress-ng[42560]: invoked with 'stress-ng --fallocate 1 --timeout 15s' by user 1000 'liruochen'
Jul 31 16:59:37 localhost.localdomain stress-ng[42560]: system: 'localhost.localdomain' Linux 6.12.0-203.76.7.5.el9uek.x86_64 #2 SMP PREEMPT_DYNAMIC Mon Jun 15 22:50:26 PDT 2026 x86_64
Jul 31 16:59:37 localhost.localdomain stress-ng[42560]: memory (MB): total 3437.13, free 1529.74, shared 10.27, buffer 0.02, swap 4020.00, free swap 3641.42
Jul 31 17:00:52 localhost.localdomain stress-ng[42568]: invoked with 'stress-ng --fallocate 1 --timeout 15s' by user 1000 'liruochen'
Jul 31 17:00:52 localhost.localdomain stress-ng[42568]: system: 'localhost.localdomain' Linux 6.12.0-203.76.7.5.el9uek.x86_64 #2 SMP PREEMPT_DYNAMIC Mon Jun 15 22:50:26 PDT 2026 x86_64
Jul 31 17:00:52 localhost.localdomain stress-ng[42568]: memory (MB): total 3437.13, free 1542.07, shared 10.27, buffer 0.02, swap 4020.00, free swap 3641.42
Jul 31 17:02:08 localhost.localdomain stress-ng[42594]: invoked with 'stress-ng --fallocate 1 --timeout 15s' by user 1000 'liruochen'
Jul 31 17:02:08 localhost.localdomain stress-ng[42594]: system: 'localhost.localdomain' Linux 6.12.0-203.76.7.5.el9uek.x86_64 #2 SMP PREEMPT_DYNAMIC Mon Jun 15 22:50:26 PDT 2026 x86_64
Jul 31 17:02:08 localhost.localdomain stress-ng[42594]: memory (MB): total 3437.13, free 1563.76, shared 10.28, buffer 0.02, swap 4020.00, free swap 3641.42
Jul 31 17:05:27 localhost.localdomain stress-ng[42603]: invoked with 'stress-ng --fallocate 1 --timeout 30s' by user 1000 'liruochen'
Jul 31 17:05:27 localhost.localdomain stress-ng[42603]: system: 'localhost.localdomain' Linux 6.12.0-203.76.7.5.el9uek.x86_64 #2 SMP PREEMPT_DYNAMIC Mon Jun 15 22:50:26 PDT 2026 x86_64
Jul 31 17:05:27 localhost.localdomain stress-ng[42603]: memory (MB): total 3437.13, free 1549.00, shared 10.29, buffer 0.02, swap 4020.00, free swap 3641.42
Jul 31 17:06:57 localhost.localdomain stress-ng[42610]: invoked with 'stress-ng --fallocate 1 --timeout 30s' by user 1000 'liruochen'
Jul 31 17:06:57 localhost.localdomain stress-ng[42610]: system: 'localhost.localdomain' Linux 6.12.0-203.76.7.5.el9uek.x86_64 #2 SMP PREEMPT_DYNAMIC Mon Jun 15 22:50:26 PDT 2026 x86_64
Jul 31 17:06:57 localhost.localdomain stress-ng[42610]: memory (MB): total 3437.13, free 1549.09, shared 10.29, buffer 0.02, swap 4020.00, free swap 3641.42
Jul 31 17:08:27 localhost.localdomain stress-ng[42619]: invoked with 'stress-ng --fallocate 1 --timeout 30s' by user 1000 'liruochen'
Jul 31 17:08:27 localhost.localdomain stress-ng[42619]: system: 'localhost.localdomain' Linux 6.12.0-203.76.7.5.el9uek.x86_64 #2 SMP PREEMPT_DYNAMIC Mon Jun 15 22:50:26 PDT 2026 x86_64
Jul 31 17:08:27 localhost.localdomain stress-ng[42619]: memory (MB): total 3437.13, free 1531.21, shared 10.31, buffer 0.02, swap 4020.00, free swap 3641.42
```

Diagram 6.5-1

Node Exporter Metrics

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Curre
nt
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--
0# HELP node_disk_written_bytes_total The total number of bytes written succe
ssfully.
# TYPE node_disk_written_bytes_total counter
node_disk_written_bytes_total{device="dm-0"} 1.377274368e+09
node_disk_written_bytes_total{device="dm-1"} 3.82500864e+08
node_disk_written_bytes_total{device="dm-2"} 1.024597937152e+12
node_disk_written_bytes_total{device="sda"} 1.026359915008e+12
node_disk_written_bytes_total{device="sr0"} 0
100 88752    0 88752    0     0  1926k      0 --:--:-- --:--:-- --:--:-- 1926
k
```

Diagram 6.5-2: Disk written bytes total (low)

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Curre
nt
                                 Dload  Upload   Total   Spent    Left  Speed
1# HELP node_disk_written_bytes_total The total number of bytes written succe
ssfully.
00 887# TYPE node_disk_written_bytes_total counter
28 node_disk_written_bytes_total{device="dm-0"} 1.379547648e+09
  node_disk_written_bytes_total{device="dm-1"} 3.82500864e+08
 0node_disk_written_bytes_total{device="dm-2"} 1.024598629376e+12
 8node_disk_written_bytes_total{device="sda"} 1.026362880512e+12
87node_disk_written_bytes_total{device="sr0"} 0
28    0     0  5776k      0 --:--:-- --:--:-- --:--:-- 6189k
```

Diagram 6.5-3: Disk written bytes total (Medium)

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Curre
nt
                                 Dload  Upload   Total   Spent    Left  Speed
100 88718    0 88718    0     0  9626k      0 --:--:-- --:--:-- --:--:-- 9626
k
# HELP node_disk_written_bytes_total The total number of bytes written succes
sfully.
# TYPE node_disk_written_bytes_total counter
node_disk_written_bytes_total{device="dm-0"} 1.379547648e+09
node_disk_written_bytes_total{device="dm-1"} 3.82500864e+08
node_disk_written_bytes_total{device="dm-2"} 1.024598629376e+12
node_disk_written_bytes_total{device="sda"} 1.026362880512e+12
node_disk_written_bytes_total{device="sr0"} 0
```

Diagram 6.5-4: Disk written bytes total (High)

Prometheus Data

Query:

rate(node_disk_written_bytes_total[1m])

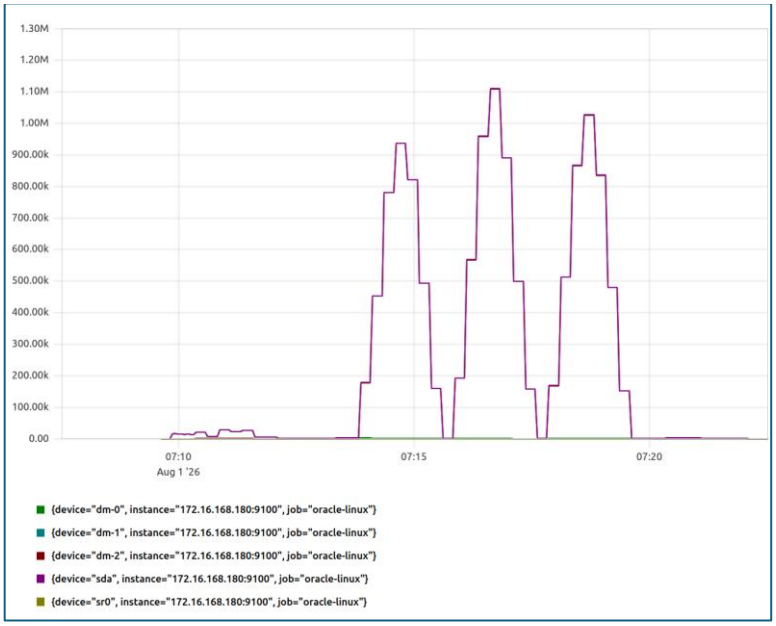


Diagram 6.5-5:node disk written bytes total — Prometheus query result

Query:
rate(node_disk_read_bytes_total[1m])

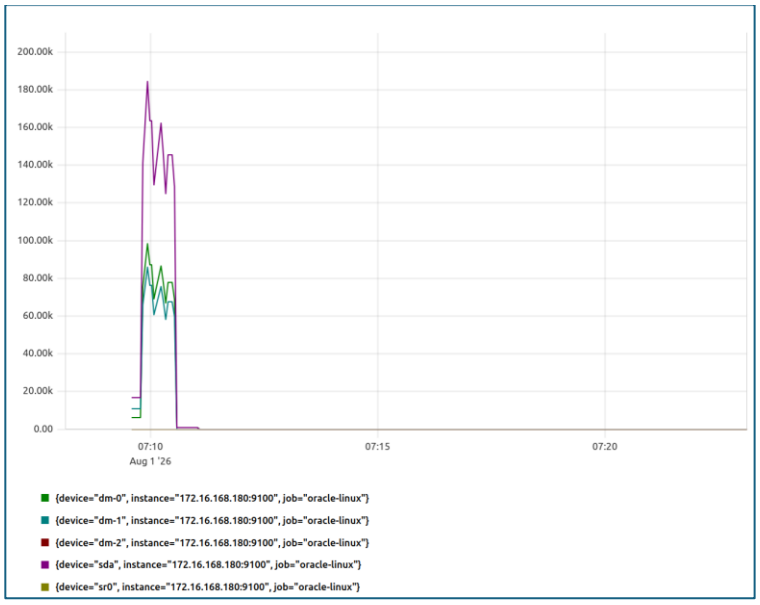


Diagram 6.5-6: node disk read bytes — Prometheus query result

Grafana Dashboard

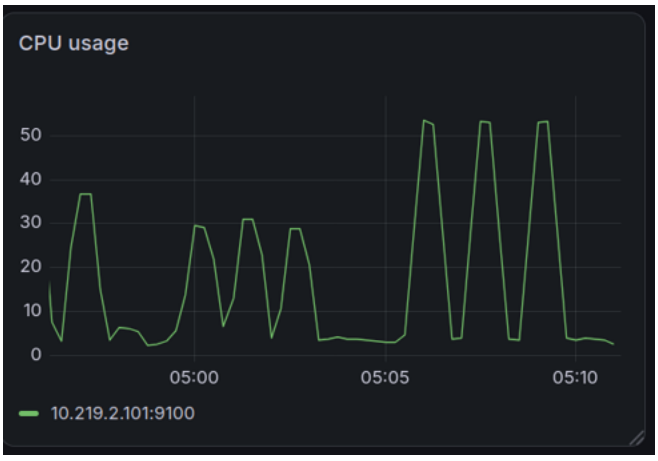


Diagram 6.5-7: CPU Usage

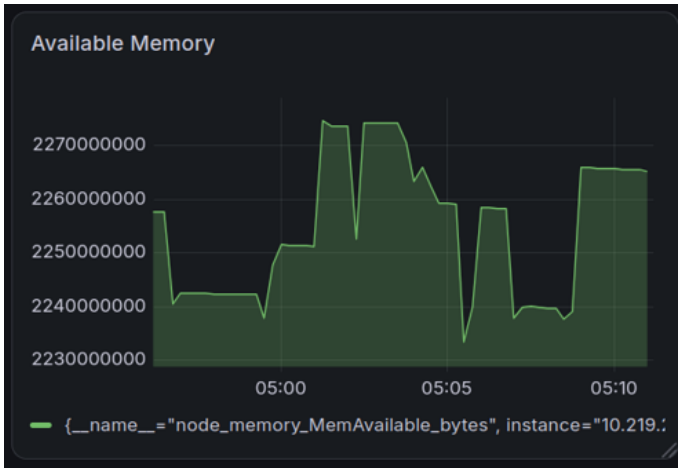


Diagram 6.5-8: Available Memory

Diagram 6.5-9: Disk Write Rate

Diagram 6.5-10: Disk Read Rate

Stress-ng Output Logs

```
Running file workload...
stress-ng: info:  [71655] setting to a 1 min, 0 secs run per stressor
stress-ng: info:  [71655] dispatching hogs: 1 fallocate
stress-ng: metrc: [71655] stressor       bogo ops real time  usr time  sys ti
me   bogo ops/s     bogo ops/s
stress-ng: metrc: [71655]                           (secs)    (secs)    (secs
)   (real time) (usr+sys time)
stress-ng: metrc: [71655] fallocate           357     60.00      0.12      1.
22         5.95         267.07
stress-ng: info:  [71655] skipped: 0
stress-ng: info:  [71655] passed: 1: fallocate (1)
stress-ng: info:  [71655] failed: 0
stress-ng: info:  [71655] metrics untrustworthy: 0
stress-ng: info:  [71655] successful run completed in 1 min, 0.01 secs
iostat after test
Linux 6.12.0-203.76.7.5.el9uek.x86_64 (localhost.localdomain)   08/01/2026  _
x86_64_ (2 CPU)
```

Diagram 6.5-11: Low Stress Test (1 fallocate)

```
Running file workload...
stress-ng: info:  [72241] setting to a 1 min, 0 secs run per stressor
stress-ng: info:  [72241] dispatching hogs: 1 fallocate
stress-ng: metrc: [72241] stressor       bogo ops real time  usr time  sys ti
me   bogo ops/s     bogo ops/s
stress-ng: metrc: [72241]                           (secs)    (secs)    (secs
)   (real time) (usr+sys time)
stress-ng: metrc: [72241] fallocate           245    217.29      0.04      0.
95         1.13         247.75
stress-ng: info:  [72241] skipped: 0
stress-ng: info:  [72241] passed: 1: fallocate (1)
stress-ng: info:  [72241] failed: 0
stress-ng: info:  [72241] metrics untrustworthy: 0
stress-ng: info:  [72241] successful run completed in 3 mins, 37.30 secs
iostat after test
Linux 6.12.0-203.76.7.5.el9uek.x86_64 (localhost.localdomain)   08/01/2026  _
x86_64_ (2 CPU)
```

Diagram 6.5-12: Medium Stress Test

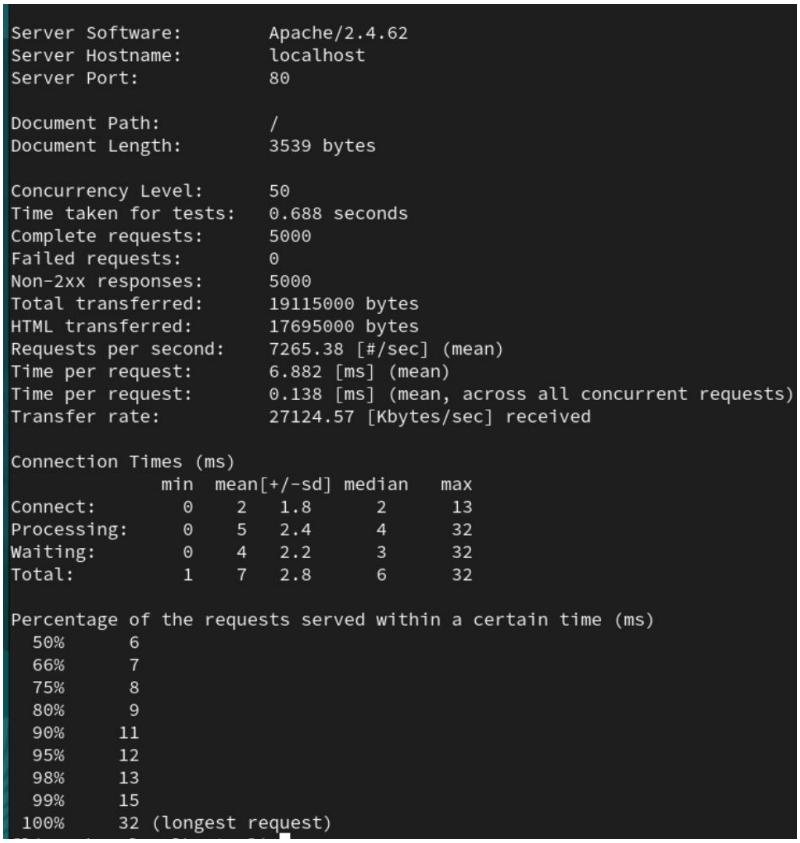
```
Running file workload...
stress-ng: info:  [72712] setting to a 1 min, 0 secs run per stressor
stress-ng: info:  [72712] dispatching hogs: 1 fallocate
stress-ng: metrc: [72712] stressor       bogo ops real time  usr time  sys ti
me   bogo ops/s     bogo ops/s
stress-ng: metrc: [72712]                           (secs)    (secs)    (secs
)   (real time) (usr+sys time)
stress-ng: metrc: [72712] fallocate           368     60.01      0.10      1.
24         6.13         274.59
stress-ng: info:  [72712] skipped: 0
stress-ng: info:  [72712] passed: 1: fallocate (1)
stress-ng: info:  [72712] failed: 0
stress-ng: info:  [72712] metrics untrustworthy: 0
stress-ng: info:  [72712] successful run completed in 1 min, 0.02 secs
iostat after test
Linux 6.12.0-203.76.7.5.el9uek.x86_64 (localhost.localdomain)   08/01/2026  _
x86_64_ (2 CPU)
```

Diagram 6.5-13: High Stress Test

ApacheBench Logs:

```
Server Software:        Apache/2.4.62
Server Hostname:        localhost
Server Port:            80

Document Path:          /
Document Length:        3539 bytes

Concurrency Level:      10
Time taken for tests:   0.353 seconds
Complete requests:      1000
Failed requests:        0
Non-2xx responses:      1000
Total transferred:      3823000 bytes
HTML transferred:       3539000 bytes
Requests per second:    2832.64 [#/sec] (mean)
Time per request:       3.530 [ms] (mean)
Time per request:       0.353 [ms] (mean, across all concurrent requests)
Transfer rate:          10575.36 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    1   1.0      0       7
Processing:     0    3   2.8      2      22
Waiting:        0    2   1.8      1      15
Total:          0    3   3.0      2      23

Percentage of the requests served within a certain time (ms)
  50%      2
  66%      3
  75%      4
  80%      5
  90%      6
  95%      9
  98%     12
  99%     16
 100%     23 (longest request)
```

Diagram 6.5-14: Benchmarking result (low)



```
Server Software:        Apache/2.4.62
Server Hostname:        localhost
Server Port:            80

Document Path:          /
Document Length:        3539 bytes

Concurrency Level:      50
Time taken for tests:   0.688 seconds
Complete requests:      5000
Failed requests:        0
Non-2xx responses:      5000
Total transferred:      19115000 bytes
HTML transferred:       17695000 bytes
Requests per second:    7265.38 [#/sec] (mean)
Time per request:       6.882 [ms] (mean)
Time per request:       0.138 [ms] (mean, across all concurrent requests)
Transfer rate:          27124.57 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    2   1.8      2      13
Processing:     0    5   2.4      4      32
Waiting:        0    4   2.2      3      32
Total:          1    7   2.8      6      32

Percentage of the requests served within a certain time (ms)
  50%      6
  66%      7
  75%      8
  80%      9
  90%     11
  95%     12
  98%     13
  99%     15
 100%     32 (longest request)
```

Diagram 6.5-15: Benchmarking result (medium)

```
Server Software:        Apache/2.4.62
Server Hostname:        localhost
Server Port:            80

Document Path:          /
Document Length:        3539 bytes

Concurrency Level:      100
Time taken for tests:   1.329 seconds
Complete requests:      10000
Failed requests:        0
Non-2xx responses:      10000
Total transferred:      38230000 bytes
HTML transferred:       35390000 bytes
Requests per second:    7521.72 [#/sec] (mean)
Time per request:       13.295 [ms] (mean)
Time per request:       0.133 [ms] (mean, across all concurrent requests)
Transfer rate:          28081.56 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    5   3.1      4      30
Processing:     1    9   4.2      8      39
Waiting:        0    6   3.5      6      33
Total:          2   13   4.9     12      41

Percentage of the requests served within a certain time (ms)
  50%     12
  66%     14
  75%     15
  80%     16
  90%     18
  95%     22
  98%     28
  99%     36
 100%     41 (longest request)
```

Diagram 6.5-16: Benchmarking result (high)

6.6 I/O latency + persistence guarantees

System Logs

```
Jul 31 03:34:41 VM stress-ng[9057]: invoked with 'stress-n' by user 0
Jul 31 03:34:41 VM stress-ng[9057]: system: 'VM' Linux 5.15.0-139-generic #149~2
0.04.1-Ubuntu SMP Wed Apr 16 08:29:56 UTC 2025 x86_64
Jul 31 03:34:41 VM stress-ng[9057]: memory (MB): total 7937.57, free 3009.04, sh
ared 35.87, buffer 91.95, swap 2048.00, free swap 2048.00
```

Diagram 6.6-1

Node Exporter Metrics

```
node_disk_io_now{device="sda"} 0
node_disk_io_now{device="sr0"} 0
node_disk_io_time_seconds_total{device="sda"} 61.672000000000004
node_disk_io_time_seconds_total{device="sr0"} 0.1
node_disk_write_time_seconds_total{device="sda"} 57.106
node_disk_write_time_seconds_total{device="sr0"} 0
node_disk_writes_completed_total{device="sda"} 18964
node_disk_writes_completed_total{device="sr0"} 0
```

Diagram 6.6-2

Prometheus Data

Query: node_disk_io_now{device="sda"}

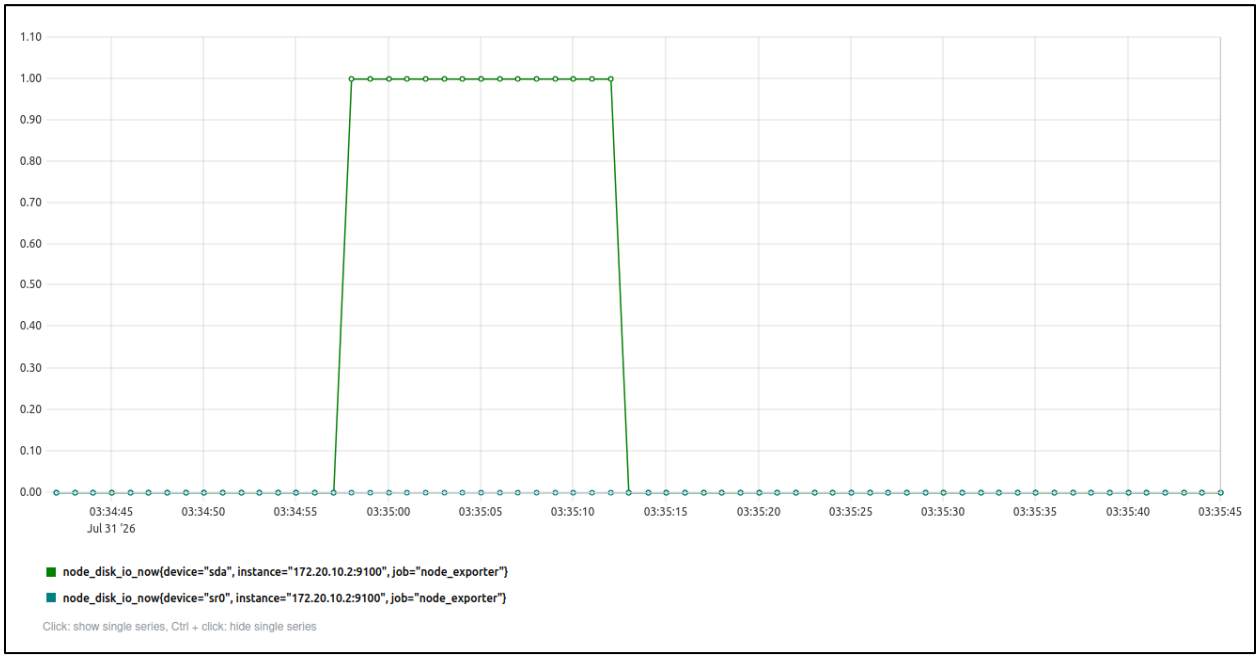


Diagram 6.6-3: node disk io — Prometheus query result

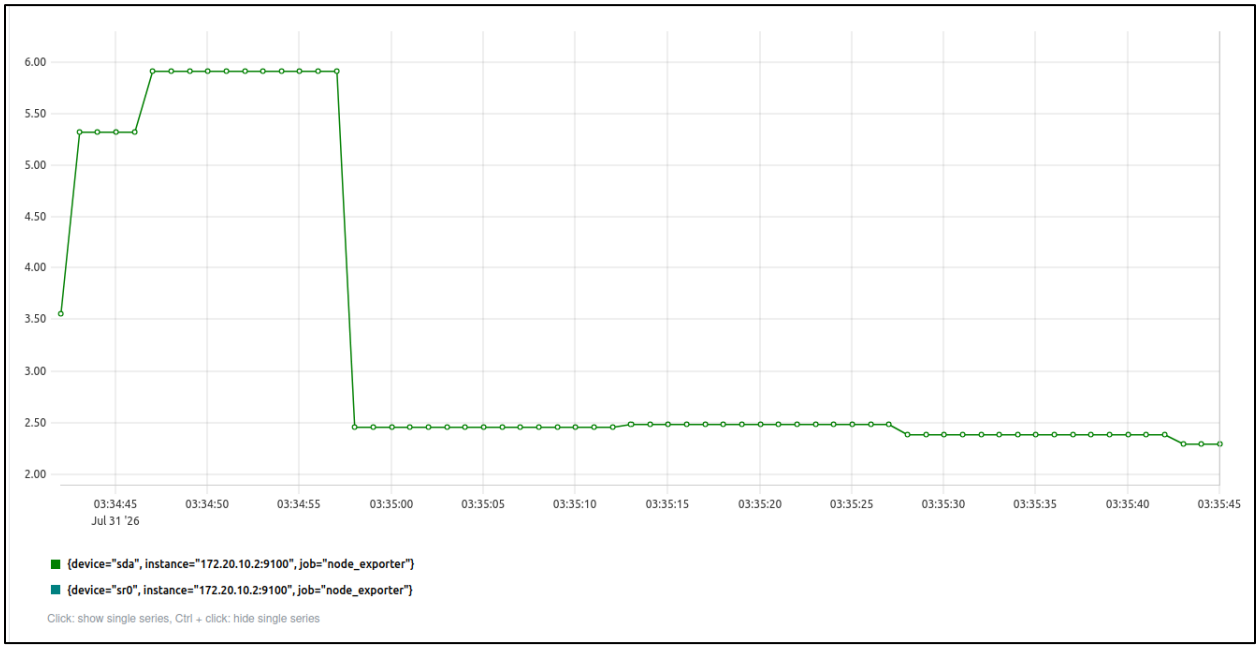


Diagram 6.6-4: Average write latency (ms) — Prometheus query result

Grafana Dashboard

Diagram 6.6-5: Disk IOps

Diagram 6.6-6: Disk Throughput

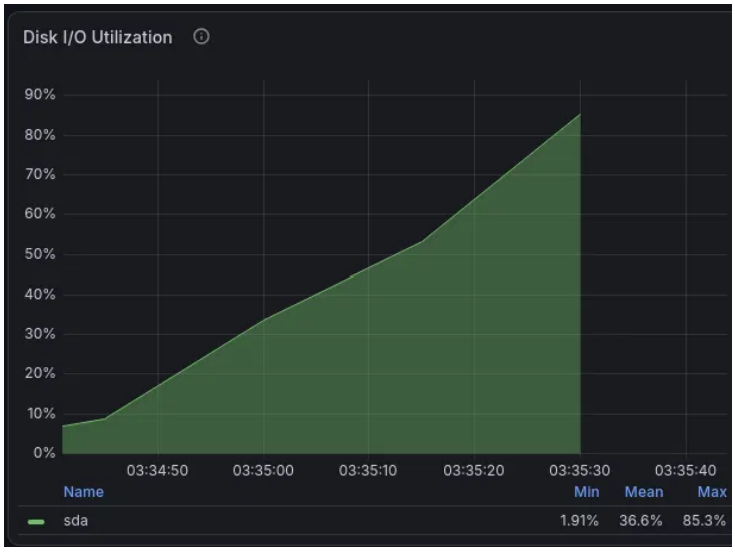


Diagram 6.6-7: Disk I/O Utilization

Stress-ng Output Logs

```
===== Starting 5.0.6 combined file + sync-file stress test (medium load) =====
File workers: 16 | Sync-file workers: 16 | Duration: 60s
stress-ng: info:  [2852] dispatching hogs: 16 filename, 16 sync-file
stress-ng: info:  [2852] successful run completed in 63.70s (1 min, 3.70 secs)
stress-ng: info:  [2852] stressor       bogo ops real time  usr time  sys time
 bogo ops/s   bogo ops/s
stress-ng: info:  [2852]                           (secs)    (secs)    (secs)
(real time) (usr+sys time)
stress-ng: info:  [2852] filename        1534325     62.04      6.14    165.27
   24732.12      8951.20
stress-ng: info:  [2852] sync-file         31235     61.93      3.09     11.25
     504.36      2178.17
===== Test completed. Log saved in ./stress-ng-logs/io-latency-persistence =====
```

Diagram 6.6-8: Medium Stress Test

Apachebench logs

```
tee: logs/apache_low.log: No such file or directory
This is ApacheBench, Version 2.3 <$Revision: 1843412 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient)
Completed 100 requests
Completed 200 requests
Completed 300 requests
Completed 400 requests
Completed 500 requests
Completed 600 requests
Completed 700 requests
Completed 800 requests
Completed 900 requests
Completed 1000 requests
Finished 1000 requests


Server Software:        Apache/2.4.41
Server Hostname:        localhost
Server Port:            80

Document Path:          /
Document Length:        10918 bytes

Concurrency Level:      10
Time taken for tests:   0.177 seconds
Complete requests:      1000
Failed requests:        0
Total transferred:      11192000 bytes
HTML transferred:       10918000 bytes
Requests per second:    5658.73 [#/sec] (mean)
Time per request:       1.767 [ms] (mean)
Time per request:       0.177 [ms] (mean, across all concurrent requests)
Transfer rate:          61848.18 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.2      0       1
Processing:     0    2   1.5      1       9
Waiting:        0    1   1.3      1       9
Total:          0    2   1.5      1       9

Percentage of the requests served within a certain time (ms)
  50%      1
  66%      2
  75%      2
  80%      3
  90%      4
  95%      5
  98%      6
  99%      8
 100%      9 (longest request)
```

Diagram 6.6-9: Benchmarking result (low)

```
tee: logs/apache_medium.log: No such file or directory
This is ApacheBench, Version 2.3 <$Revision: 1843412 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient)
Completed 500 requests
Completed 1000 requests
Completed 1500 requests
Completed 2000 requests
Completed 2500 requests
Completed 3000 requests
Completed 3500 requests
Completed 4000 requests
Completed 4500 requests
Completed 5000 requests
Finished 5000 requests


Server Software:        Apache/2.4.41
Server Hostname:        localhost
Server Port:            80

Document Path:          /
Document Length:        10918 bytes

Concurrency Level:      50
Time taken for tests:   0.738 seconds
Complete requests:      5000
Failed requests:        0
Total transferred:      55960000 bytes
HTML transferred:       54590000 bytes
Requests per second:    6772.19 [#/sec] (mean)
Time per request:       7.383 [ms] (mean)
Time per request:       0.148 [ms] (mean, across all concurrent requests)
Transfer rate:          74017.88 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.6      0       5
Processing:     0    7   7.6      5      67
Waiting:        0    7   7.3      5      66
Total:          0    7   7.6      6      67

Percentage of the requests served within a certain time (ms)
  50%      6
  66%      7
  75%      8
  80%      9
  90%     12
  95%     15
  98%     47
  99%     52
 100%     67 (longest request)
```

Diagram 6.6-10: Benchmarking result (medium)

```
tee: logs/apache_high.log: No such file or directory
This is ApacheBench, Version 2.3 <$Revision: 1843412 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient)
Completed 1000 requests
Completed 2000 requests
Completed 3000 requests
Completed 4000 requests
Completed 5000 requests
Completed 6000 requests
Completed 7000 requests
Completed 8000 requests
Completed 9000 requests
Completed 10000 requests
Finished 10000 requests


Server Software:        Apache/2.4.41
Server Hostname:        localhost
Server Port:            80

Document Path:          /
Document Length:        10918 bytes

Concurrency Level:      100
Time taken for tests:   0.567 seconds
Complete requests:      10000
Failed requests:        0
Total transferred:      111920000 bytes
HTML transferred:       109180000 bytes
Requests per second:    17624.50 [#/sec] (mean)
Time per request:       5.674 [ms] (mean)
Time per request:       0.057 [ms] (mean, across all concurrent requests)
Transfer rate:          192630.27 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.7      0       8
Processing:     1    5   4.1      4      34
Waiting:        0    5   3.8      4      33
Total:          1    6   4.1      4      34

Percentage of the requests served within a certain time (ms)
  50%      4
  66%      5
  75%      6
  80%      7
  90%     11
  95%     13
  98%     18
  99%     23
 100%     34 (longest request)
```

Diagram 6.6-11: Benchmarking result (high)

6.7 Metadata heavy workloads

System Logs

```
Aug 01 15:28:27 localhost.localdomain stress-ng[6476]: invoked with 'stress-ng --dir 4 --fallocate 4 --timeout 60s --metrics-brief --temp-path
Aug 01 15:28:27 localhost.localdomain stress-ng[6476]: system: 'localhost.localdomain' Linux 6.12.0-203.76.7.1.el10uek.x86_64 #1 SMP PREEMPT_D
Aug 01 15:28:27 localhost.localdomain stress-ng[6476]: memory (MB): total 3457.36, free 1196.69, shared 22.36, buffer 0.32, swap 4040.00, free
Aug 01 15:30:41 localhost.localdomain stress-ng[6503]: invoked with 'stress-ng --dir 24 --fallocate 16 --timeout 60s --metrics-brief --temp-pa
Aug 01 15:30:41 localhost.localdomain stress-ng[6503]: system: 'localhost.localdomain' Linux 6.12.0-203.76.7.1.el10uek.x86_64 #1 SMP PREEMPT_D
Aug 01 15:30:41 localhost.localdomain stress-ng[6503]: memory (MB): total 3457.36, free 1211.06, shared 22.36, buffer 0.32, swap 4040.00, free
Aug 01 15:33:19 localhost.localdomain stress-ng[6575]: invoked with 'stress-ng --dir 80 --fallocate 64 --timeout 60s --metrics-brief --temp-pa
Aug 01 15:33:19 localhost.localdomain stress-ng[6575]: system: 'localhost.localdomain' Linux 6.12.0-203.76.7.1.el10uek.x86_64 #1 SMP PREEMPT_D
Aug 01 15:33:19 localhost.localdomain stress-ng[6575]: memory (MB): total 3457.36, free 1213.96, shared 22.36, buffer 0.32, swap 4040.00, free
Aug 01 15:39:38 localhost.localdomain kernel: 07:39:38.801965 SHCLX11   Shared Clipboard: Converting X11 format index 0x3 to VBox format 0x1 f
Aug 01 15:39:39 localhost.localdomain gnome-shell[3735]: Cursor update failed: drmModeAtomicCommit: Invalid argument
```

Diagram 6.7-1

Node Exporter Metrics

Note: This data is combined with experiment 5.0.3 because the Prometheus scrapping accumulates the metrics from the NodeExporter.

```
# HELP node_xfs_directory_operation_create_total Number of times a new directory entry was created for a filesystem.
# TYPE node_xfs_directory_operation_create_total counter
node_xfs_directory_operation_create_total{device="dm-0"} 5.222048e+06
node_xfs_directory_operation_create_total{device="dm-2"} 89
node_xfs_directory_operation_create_total{device="sda2"} 1
# HELP node_xfs_directory_operation_getdents_total Number of times the directory getdents operation was performed for a filesystem.
# TYPE node_xfs_directory_operation_getdents_total counter
node_xfs_directory_operation_getdents_total{device="dm-0"} 4.296221e+06
node_xfs_directory_operation_getdents_total{device="dm-2"} 196
node_xfs_directory_operation_getdents_total{device="sda2"} 0
# HELP node_xfs_directory_operation_lookup_total Number of file name directory lookups which miss the operating systems directory name lookup cache.
# TYPE node_xfs_directory_operation_lookup_total counter
node_xfs_directory_operation_lookup_total{device="dm-0"} 4.198171e+06
node_xfs_directory_operation_lookup_total{device="dm-2"} 886
node_xfs_directory_operation_lookup_total{device="sda2"} 13
# HELP node_xfs_directory_operation_remove_total Number of times an existing directory entry was created for a filesystem.
# TYPE node_xfs_directory_operation_remove_total counter
node_xfs_directory_operation_remove_total{device="dm-0"} 5.222048e+06
node_xfs_directory_operation_remove_total{device="dm-2"} 88
node_xfs_directory_operation_remove_total{device="sda2"} 1
```

Diagram 6.7-2: XFS directory operation counters by NodeExporter

Prometheus Data

Query: rate(node_xfs_inode_operation_attempts_total{device="dm-0"}[5m])

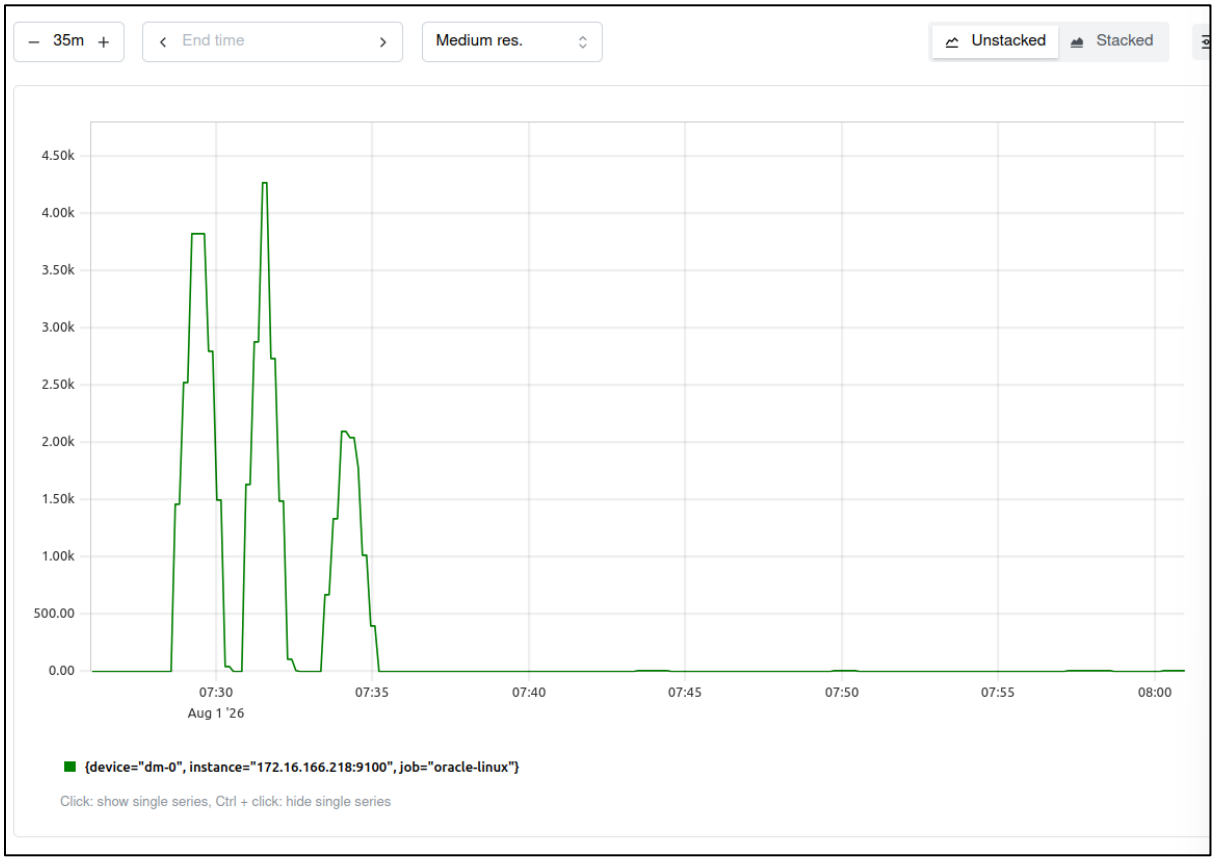


Diagram 6.7-3: Inode Attempts per second

Query: increase(node_xfs_inode_operation_missed_total{device="dm-0"}[5m])

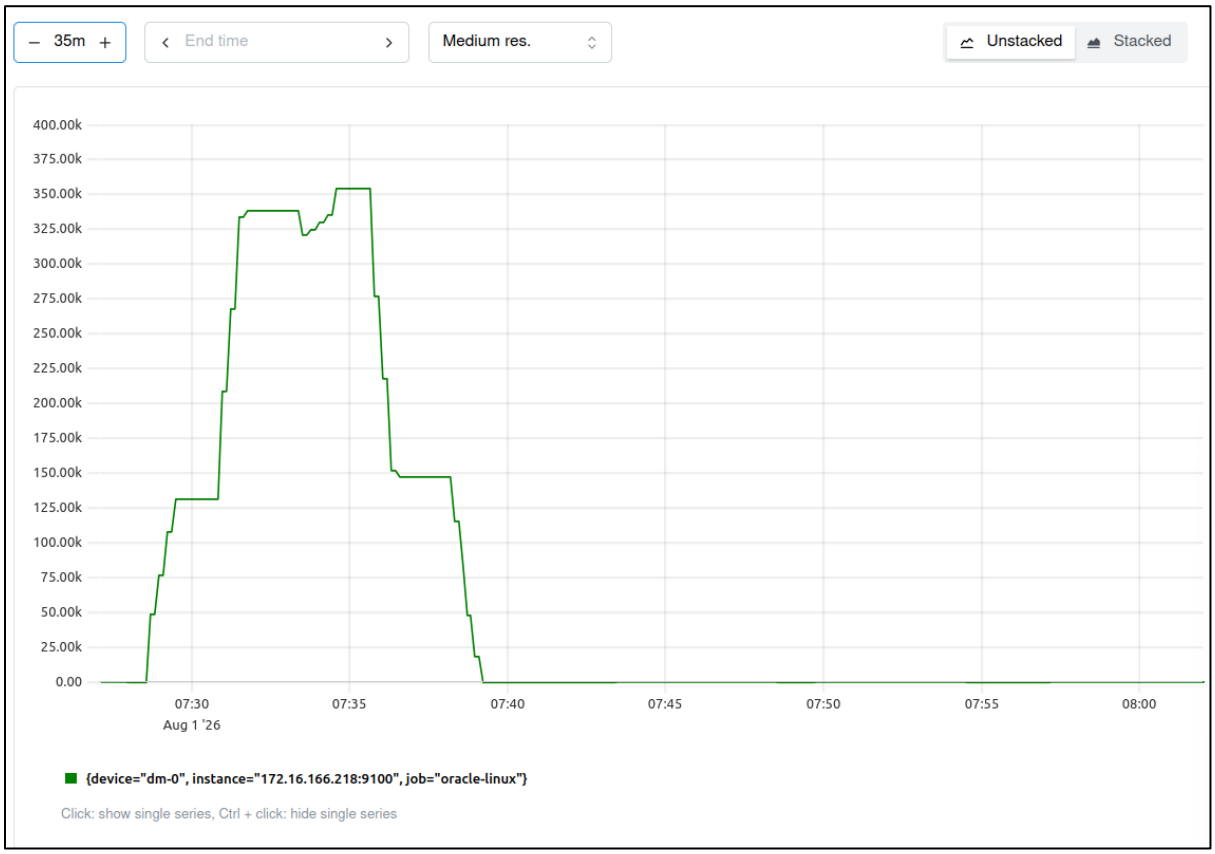


Diagram 6.7-4: Inode misses during a time window

Grafana Dashboard

Diagram 6.7-5: Disk Read/Write IOps

Diagram 6.7-6: Disk Read/Write Data

Diagram 6.7-7: Disk Average Wait Time

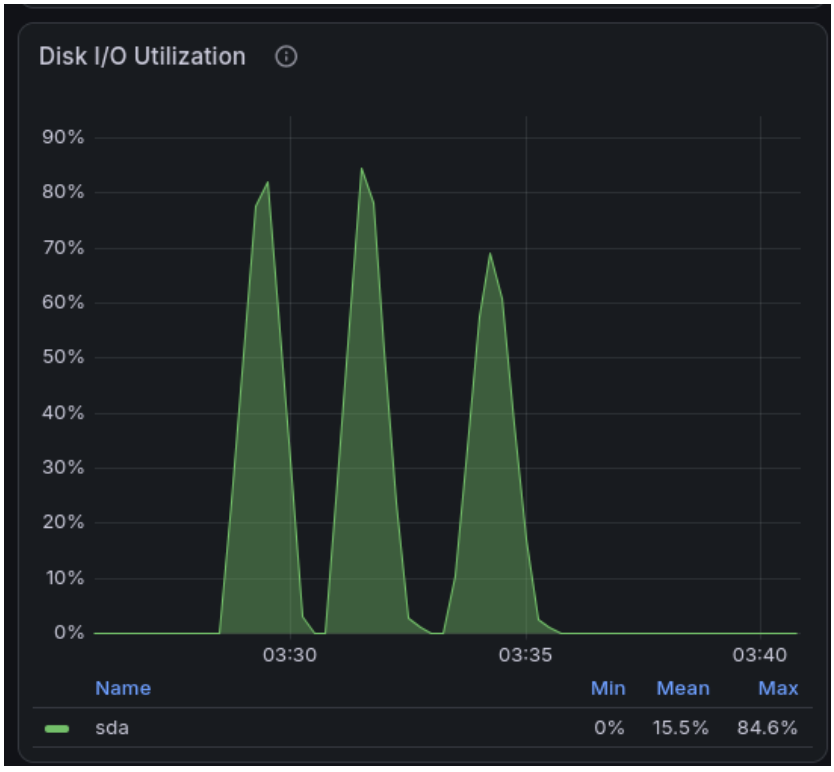


Diagram 6.7-8: Disk I/O Utilization

Diagram 6.7-9: Average Queue Size

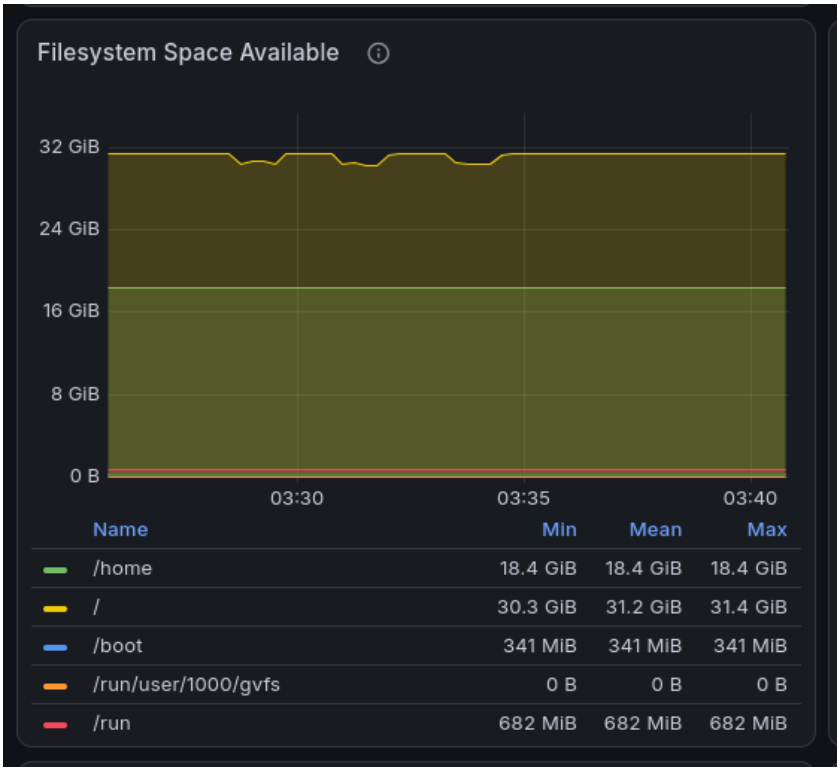


Diagram 6.7-10: Filesystem Space Available

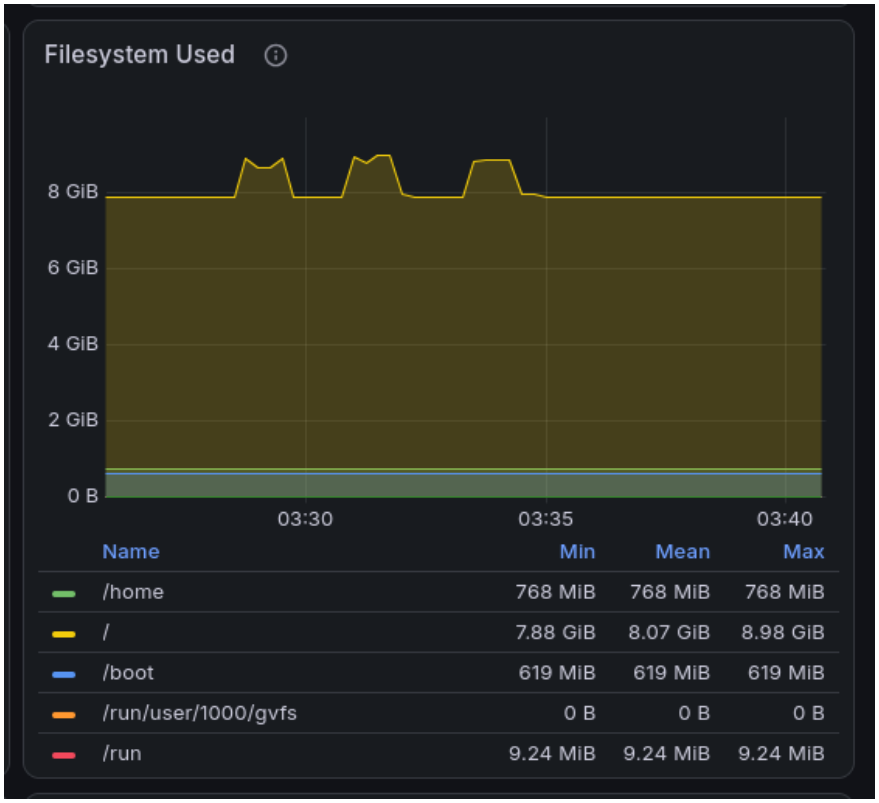


Diagram 6.7-11: Filesystem Used

Diagram 6.7-12: File Nodes Free

Stress-ng Output Logs

```
stress-ng: info:  [6476] setting to a 1 min run per stressor
stress-ng: info:  [6476] dispatching hogs: 4 dir, 4 fallocate
stress-ng: metrc: [6476] stressor       bogo ops real time  usr time  sys time   bogo ops/s     bogo ops/s
stress-ng: metrc: [6476]                           (secs)    (secs)    (secs)   (real time) (usr+sys time)
stress-ng: metrc: [6476] dir              239571     60.66      4.71     63.59      3949.16        3507.38
stress-ng: metrc: [6476] fallocate           135     60.04      0.61     26.32         2.25           5.01
stress-ng: info:  [6476] skipped: 0
stress-ng: info:  [6476] passed: 8: dir (4) fallocate (4)
stress-ng: info:  [6476] failed: 0
stress-ng: info:  [6476] metrics untrustworthy: 0
stress-ng: info:  [6476] successful run completed in 1 min
FINIHSED LOW STRESS TEST FOR DIRECTORY --dir
```

Diagram 6.7-13: Low Stress Test (4 dir + 4 fallocate)

```
stress-ng: info:  [6503] setting to a 1 min run per stressor
stress-ng: info:  [6503] dispatching hogs: 24 dir, 16 fallocate
stress-ng: metrc: [6503] stressor       bogo ops real time  usr time  sys time   bogo ops/s     bogo ops/s
stress-ng: metrc: [6503]                           (secs)    (secs)    (secs)   (real time) (usr+sys time)
stress-ng: metrc: [6503] dir              196623     85.48      9.88    121.88      2300.21        1492.34
stress-ng: metrc: [6503] fallocate           348     60.06      0.49     20.05         5.79          16.94
stress-ng: info:  [6503] skipped: 0
stress-ng: info:  [6503] passed: 40: dir (24) fallocate (16)
stress-ng: info:  [6503] failed: 0
stress-ng: info:  [6503] metrics untrustworthy: 0
stress-ng: info:  [6503] successful run completed in 1 min, 26.59 secs
FINIHSED MEDIUM STRESS TEST FOR DIRECTORY --dir
```

Diagram 6.7-14: Medium Stress Test (24 dir + 16 fallocate)

```
stress-ng: info:  [6575] setting to a 1 min run per stressor
stress-ng: info:  [6575] dispatching hogs: 80 dir, 64 fallocate
stress-ng: metrc: [6575] stressor       bogo ops real time  usr time  sys time   bogo ops/s     bogo ops/s
stress-ng: metrc: [6575]                           (secs)    (secs)    (secs)   (real time) (usr+sys time)
stress-ng: metrc: [6575] dir              139902     82.11     10.56    123.12      1703.78        1046.53
stress-ng: metrc: [6575] fallocate           522     65.74      0.31     15.56         7.94          32.90
stress-ng: info:  [6575] skipped: 0
stress-ng: info:  [6575] passed: 144: dir (80) fallocate (64)
stress-ng: info:  [6575] failed: 0
stress-ng: info:  [6575] metrics untrustworthy: 0
stress-ng: info:  [6575] successful run completed in 1 min, 26.86 secs
FINIHSED HIGH STRESS TEST FOR DIRECTORY --dir
```

Diagram 6.7-15: High Stress Test (80 dir + 64 fallocate)

ApacheBench logs:

```
Server Software:        Apache/2.4.63
Server Hostname:        localhost
Server Port:            80

Document Path:          /wordpress/
Document Length:        66517 bytes

Concurrency Level:      10
Time taken for tests:   4.380 seconds
Complete requests:      100
Failed requests:        0
Total transferred:      6678800 bytes
HTML transferred:       6651700 bytes
Requests per second:    22.83 [#/sec] (mean)
Time per request:       438.019 [ms] (mean)
Time per request:       43.802 [ms] (mean, across all concurrent requests)
Transfer rate:          1489.04 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.1      0       1
Processing:   153  368 128.1    323     742
Waiting:      151  358 126.8    312     718
Total:        153  368 128.1    323     742

Percentage of the requests served within a certain time (ms)
  50%    323
  66%    390
  75%    490
  80%    520
  90%    557
  95%    605
  98%    643
  99%    742
 100%    742 (longest request)
```

Diagram 6.7-16: Benchmarking result (Idle)

```
Server Software:        Apache/2.4.63
Server Hostname:        localhost
Server Port:            80

Document Path:          /wordpress/
Document Length:        66517 bytes

Concurrency Level:      10
Time taken for tests:   7.276 seconds
Complete requests:      100
Failed requests:        0
Total transferred:      6678800 bytes
HTML transferred:       6651700 bytes
Requests per second:    13.74 [#/sec] (mean)
Time per request:       727.552 [ms] (mean)
Time per request:       72.755 [ms] (mean, across all concurrent requests)
Transfer rate:          896.47 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.4      0       2
Processing:    70  679 245.8    718    1092
Waiting:       67  663 236.9    701    1053
Total:         70  679 245.8    718    1092

Percentage of the requests served within a certain time (ms)
  50%    718
  66%    807
  75%    874
  80%    915
  90%    997
  95%   1043
  98%   1086
  99%   1092
 100%   1092 (longest request)
```

Diagram 6.7-17: Benchmarking result (low)

```
Server Software:        Apache/2.4.63
Server Hostname:        localhost
Server Port:            80

Document Path:          /wordpress/
Document Length:        66517 bytes

Concurrency Level:      10
Time taken for tests:   10.719 seconds
Complete requests:      100
Failed requests:        0
Total transferred:      6678800 bytes
HTML transferred:       6651700 bytes
Requests per second:    9.33 [#/sec] (mean)
Time per request:       1071.950 [ms] (mean)
Time per request:       107.195 [ms] (mean, across all concurrent requests)
Transfer rate:          608.45 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.5      0       5
Processing:    67 1042 587.8    946    2340
Waiting:       67  976 571.2    861    2231
Total:         67 1042 587.8    946    2340

Percentage of the requests served within a certain time (ms)
  50%    946
  66%   1253
  75%   1478
  80%   1688
  90%   1952
  95%   2119
  98%   2267
  99%   2340
 100%   2340 (longest request)
```

Diagram 6.7-18: Benchmarking result (medium)

```
Server Software:        Apache/2.4.63
Server Hostname:        localhost
Server Port:            80

Document Path:          /wordpress/
Document Length:        66517 bytes

Concurrency Level:      10
Time taken for tests:   12.072 seconds
Complete requests:      100
Failed requests:        0
Total transferred:      6678800 bytes
HTML transferred:       6651700 bytes
Requests per second:    8.28 [#/sec] (mean)
Time per request:       1207.202 [ms] (mean)
Time per request:       120.720 [ms] (mean, across all concurrent requests)
Transfer rate:          540.28 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    1   6.2      0      37
Processing:    85 1184 545.3   1137    2317
Waiting:       80 1097 551.5   1073    2258
Total:         85 1185 545.7   1137    2317

Percentage of the requests served within a certain time (ms)
  50%   1137
  66%   1257
  75%   1493
  80%   1671
  90%   2026
  95%   2193
  98%   2248
  99%   2317
 100%   2317 (longest request)
```

Diagram 6.7-18: Benchmarking result (high)

6.8 Everything is combined

System Logs

```
Jul 31 19:54:48 localhost.localdomain stress-ng[8606]: invoked with 'stress-ng --fallocate 4 --filename 4 --sync-file 4 --dir 4 --timeout 60s --metrics-brief' by user 1000 'zhanghejunjie'
Jul 31 19:54:48 localhost.localdomain stress-ng[8606]: system: 'localhost.localdomain' Linux 5.15.0-321.202.5.el9uek.x86_64 #2 SMP Tue Jun 2 10:39:46 PDT 2026 x86_64
Jul 31 19:54:48 localhost.localdomain stress-ng[8606]: memory (MB): total 7483.79, free 3623.02, shared 24.03, buffer 1.86, swap 0.00, free swap 0.00
Jul 31 19:56:49 localhost.localdomain stress-ng[8640]: invoked with 'stress-ng --fallocate 16 --filename 16 --sync-file 16 --dir 16 --timeout 60s --metrics-brief' by user 1000 'zhanghejunjie'
Jul 31 19:56:49 localhost.localdomain stress-ng[8640]: system: 'localhost.localdomain' Linux 5.15.0-321.202.5.el9uek.x86_64 #2 SMP Tue Jun 2 10:39:46 PDT 2026 x86_64
Jul 31 19:56:49 localhost.localdomain stress-ng[8640]: memory (MB): total 7483.79, free 3646.67, shared 24.03, buffer 1.86, swap 0.00, free swap 0.00
Jul 31 19:59:33 localhost.localdomain stress-ng[8748]: invoked with 'stress-ng --fallocate 64 --filename 64 --sync-file 64 --dir 64 --timeout 60s --metrics-brief' by user 1000 'zhanghejunjie'
Jul 31 19:59:33 localhost.localdomain stress-ng[8748]: system: 'localhost.localdomain' Linux 5.15.0-321.202.5.el9uek.x86_64 #2 SMP Tue Jun 2 10:39:46 PDT 2026 x86_64
Jul 31 19:59:33 localhost.localdomain stress-ng[8748]: memory (MB): total 7483.79, free 3642.03, shared 24.03, buffer 1.86, swap 0.00, free swap 0.00
Jul 31 19:59:58 localhost.localdomain rtkit-daemon[746]: The canary thread is apparently starving. Taking action.
```

Diagram 6.8-1: System Logs

Node Exporter Metrics

```
# TYPE node_xfs_write_calls_total counter
node_xfs_write_calls_total{device="dm-0"} 277061
node_xfs_write_calls_total{device="dm-1"} 5.131257e+06
node_xfs_write_calls_total{device="sda1"} 1
```

Diagram 6.8-2: Node xfs write calls total (Low stress)

```
# TYPE node_xfs_write_calls_total counter
node_xfs_write_calls_total{device="dm-0"} 277101
node_xfs_write_calls_total{device="dm-1"} 5.82549e+06
node_xfs_write_calls_total{device="sda1"} 1
```

Diagram 6.8-3: Node xfs write calls total (Medium stress)

```
# TYPE node_xfs_write_calls_total counter
node_xfs_write_calls_total{device="dm-0"} 277119
node_xfs_write_calls_total{device="dm-1"} 6.360217e+06
node_xfs_write_calls_total{device="sda1"} 1
```

Diagram 6.8-4: Node xfs write calls total (High stress)

Prometheus Data

Query:
(rate(node_disk_io_time_seconds_total{job="oracle-linux"}[9m]))
(rate(node_disk_written_bytes_total{job="oracle-linux"}[9m]))

This graph shows that disk I/O time spike under everything is combine condition is higher than all other conditions.

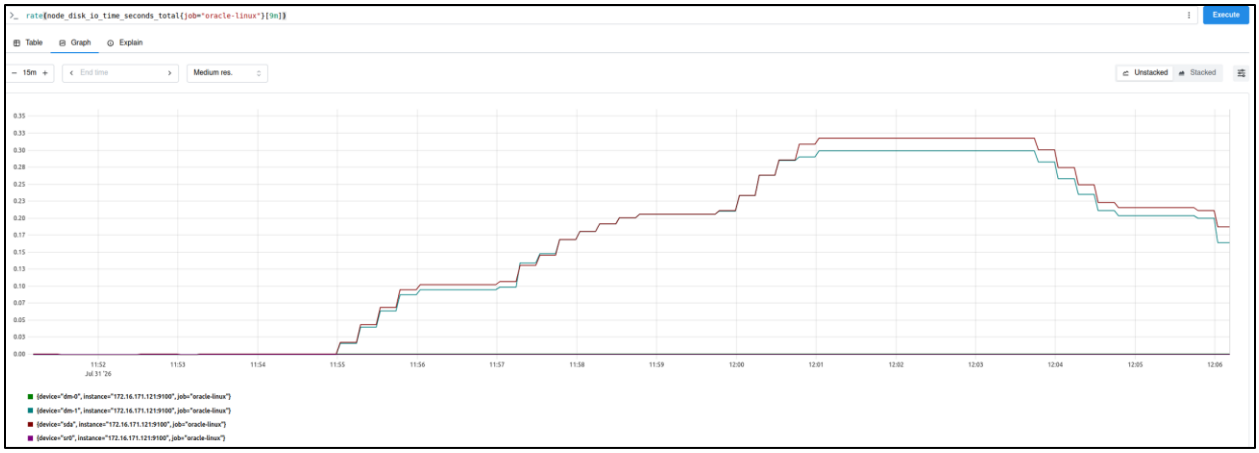


Diagram 6.8-5: Total disk I/O time

This graph shows that the throughput spike higher is higher than all other conditions.

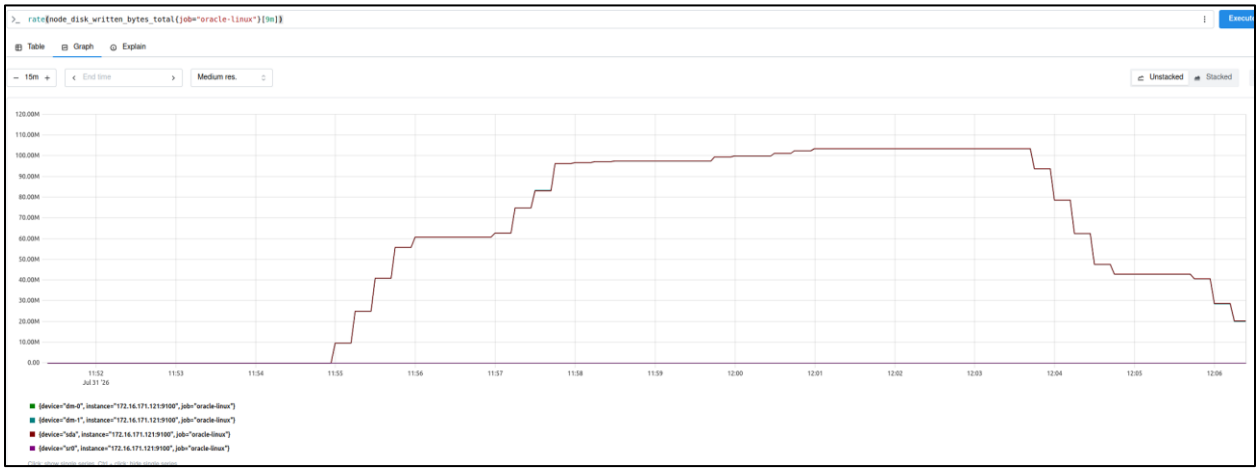


Diagram 6.8-6: Total disk written byte

Grafana Dashboard (7:31 – 7.40)

Diagram 6.8-7: CPU usage

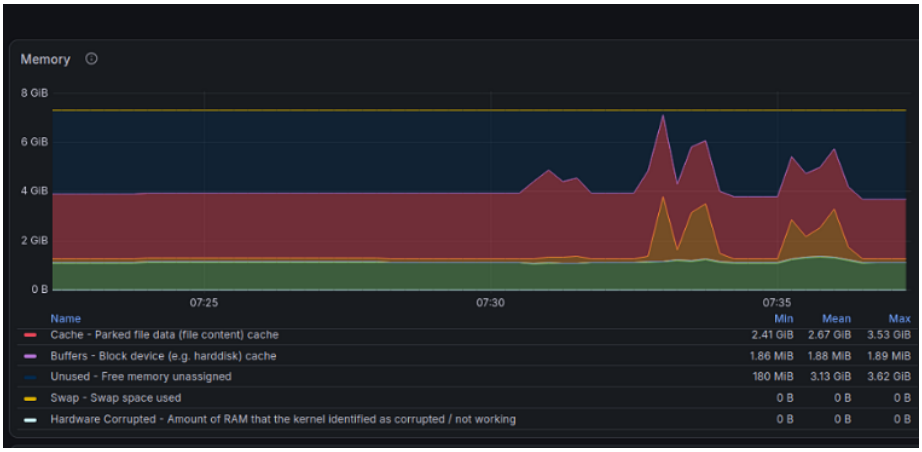


Diagram 6.8-8: Memory usage

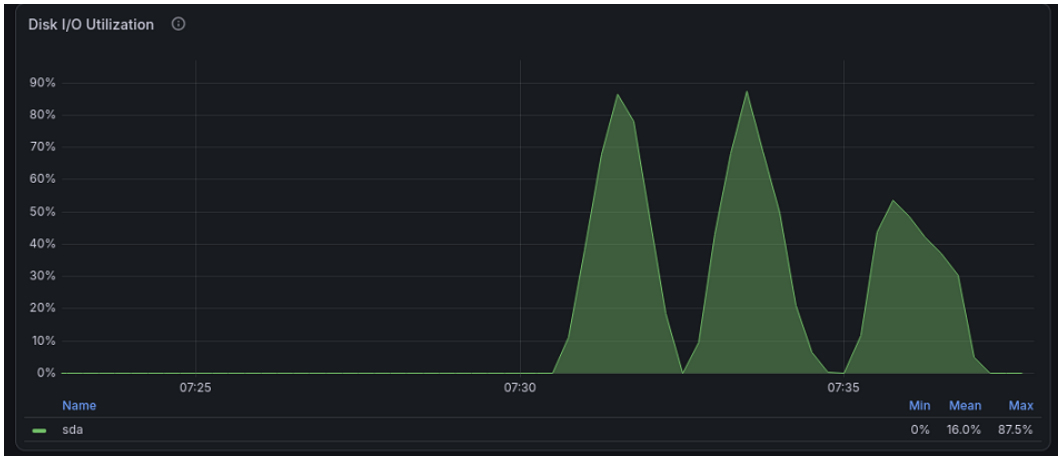


Diagram 6.8-9: Disk I/O Utilization

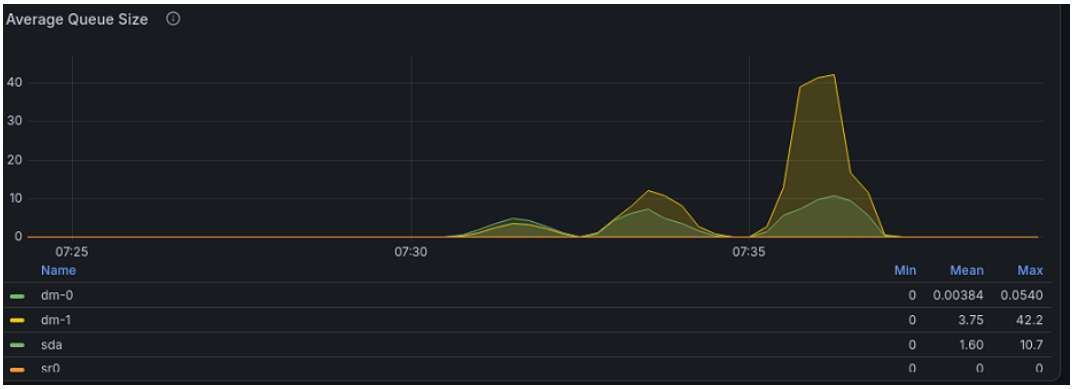


Diagram 6.8-10: Average Queue Size

Diagram 6.8-11: Disk Average Wait Time

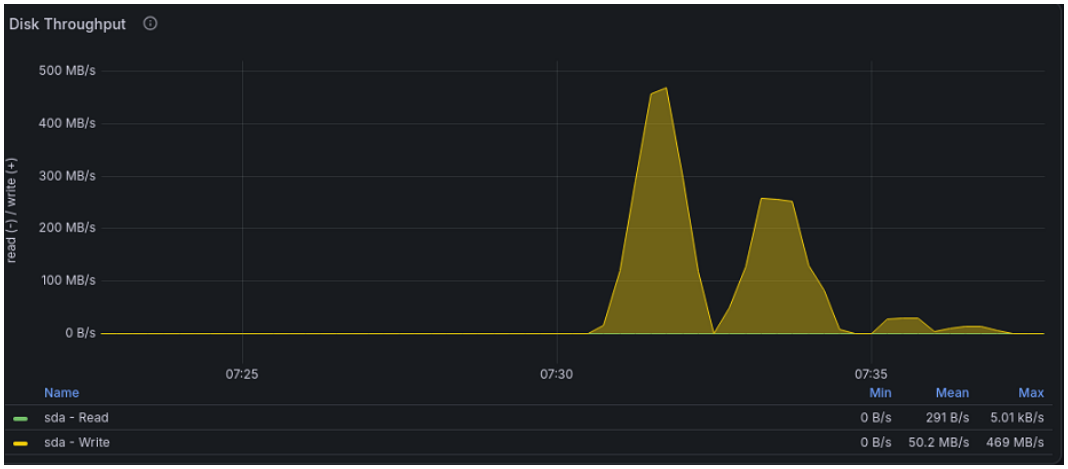


Diagram 6.8-12: Disk Throughput

Apache Bench logs:

```
Server Software:        Apache/2.4.62
Server Hostname:        localhost
Server Port:            80

Document Path:          /wordpress/
Document Length:        92427 bytes

Concurrency Level:      10
Time taken for tests:   15.488 seconds
Complete requests:      100
Failed requests:        0
Total transferred:      9269800 bytes
HTML transferred:       9242700 bytes
Requests per second:    6.46 [#/sec] (mean)
Time per request:       1548.820 [ms] (mean)
Time per request:       154.882 [ms] (mean, across all concurrent requests)
Transfer rate:          584.48 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.5      0       3
Processing:   749 1395 254.6   1400    2143
Waiting:      606 1329 255.3   1361    1852
Total:        749 1395 254.7   1400    2144

Percentage of the requests served within a certain time (ms)
  50%   1400
  66%   1487
  75%   1551
  80%   1585
  90%   1718
  95%   1755
  98%   1991
  99%   2144
 100%   2144 (longest request)
```

Diagram 6.8-13: Benchmarking result (low)

```
Server Software:        Apache/2.4.62
Server Hostname:        localhost
Server Port:            80

Document Path:          /wordpress/
Document Length:        92427 bytes

Concurrency Level:      10
Time taken for tests:   30.106 seconds
Complete requests:      100
Failed requests:        0
Total transferred:      9269800 bytes
HTML transferred:       9242700 bytes
Requests per second:    3.32 [#/sec] (mean)
Time per request:       3010.589 [ms] (mean)
Time per request:       301.059 [ms] (mean, across all concurrent requests)
Transfer rate:          300.69 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    5  12.2      0      83
Processing:   902 2710 1071.1   2606    5420
Waiting:      900 2622 1067.7   2501    5335
Total:        903 2714 1066.4   2607    5420

Percentage of the requests served within a certain time (ms)
  50%   2607
  66%   3018
  75%   3222
  80%   3455
  90%   4683
  95%   4938
  98%   5109
  99%   5420
 100%   5420 (longest request)
```

Diagram 6.8-14: Benchmarking result (medium)

```
Benchmarking localhost (be patient).....done


Server Software:        Apache/2.4.62
Server Hostname:        localhost
Server Port:            80

Document Path:          /wordpress/
Document Length:        92427 bytes

Concurrency Level:      10
Time taken for tests:   45.366 seconds
Complete requests:      100
Failed requests:        0
Total transferred:      9269800 bytes
HTML transferred:       9242700 bytes
Requests per second:    2.20 [#/sec] (mean)
Time per request:       4536.552 [ms] (mean)
Time per request:       453.655 [ms] (mean, across all concurrent requests)
Transfer rate:          199.55 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0   39 101.5      4     501
Processing:   239 4357 2176.8   4364    8655
Waiting:      230 4095 2035.7   4042    8514
Total:        242 4397 2176.7   4367    8655

Percentage of the requests served within a certain time (ms)
  50%   4367
  66%   5332
  75%   6065
  80%   6704
  90%   7534
  95%   7822
  98%   7931
  99%   8655
 100%   8655 (longest request)
```

Diagram 6.8-15: Benchmarking result (high)

Stress-ng Output Logs

```
Start time: Fri Jul 31 07:54:48 PM +08 2026
Low stress test & result:
stress-ng: info:  [8606] setting to a 1 min run per stressor
stress-ng: info:  [8606] dispatching hogs: 4 fallocate, 4 filename, 4 sync-file, 4 dir
stress-ng: info:  [8607] fallocate: using 256M file system space per stressor instance (total 1G of 18.90G available file system space)
stress-ng: info:  [8615] sync-file: using 256M file system space per stressor instance (total 1G of 18.75G available file system space)
stress-ng: metrc: [8606] stressor       bogo ops real time  usr time  sys time   bogo ops/s     bogo ops/s
stress-ng: metrc: [8606]                           (secs)    (secs)    (secs)   (real time) (usr+sys time)
stress-ng: metrc: [8606] fallocate           115     60.25      0.19     14.90         1.91           7.62
stress-ng: metrc: [8606] filename          20251     60.17      0.55     11.79       336.59        1640.59
stress-ng: metrc: [8606] sync-file          4972     60.09      1.85     10.27        82.75         410.20
stress-ng: metrc: [8606] dir              119109     60.82      1.37     18.73      1958.24        5927.70
stress-ng: info:  [8606] skipped: 0
stress-ng: info:  [8606] passed: 16: fallocate (4) filename (4) sync-file (4) dir (4)
stress-ng: info:  [8606] failed: 0
stress-ng: info:  [8606] metrics untrustworthy: 0
stress-ng: info:  [8606] successful run completed in 1 min
End time: Fri Jul 31 07:55:49 PM +08 2026
```

Diagram 6.8-16: Low Stress Test (Everything is combined)

```
Start time: Fri Jul 31 07:56:49 PM +08 2026
Medium stress test & result:
stress-ng: info:  [8640] setting to a 1 min run per stressor
stress-ng: info:  [8640] dispatching hogs: 16 fallocate, 16 filename, 16 sync-file, 16 dir
stress-ng: info:  [8641] fallocate: using 64M file system space per stressor instance (total 1G of 18.91G available file system space)
stress-ng: info:  [8673] sync-file: using 64M file system space per stressor instance (total 1G of 18.73G available file system space)
stress-ng: metrc: [8640] stressor       bogo ops real time  usr time  sys time   bogo ops/s     bogo ops/s
stress-ng: metrc: [8640]                           (secs)    (secs)    (secs)   (real time) (usr+sys time)
stress-ng: metrc: [8640] fallocate           280     60.55      0.19      9.65         4.62          28.46
stress-ng: metrc: [8640] filename           7415     63.48      0.40      7.22       116.81         972.09
stress-ng: metrc: [8640] sync-file          9690     60.41      1.08      6.23       160.41        1325.19
stress-ng: metrc: [8640] dir              109944     95.87      3.55     67.67      1146.82        1543.80
stress-ng: info:  [8640] skipped: 0
stress-ng: info:  [8640] passed: 64: fallocate (16) filename (16) sync-file (16) dir (16)
stress-ng: info:  [8640] failed: 0
stress-ng: info:  [8640] metrics untrustworthy: 0
stress-ng: info:  [8640] successful run completed in 1 min, 44.28 secs
End time: Fri Jul 31 07:58:33 PM +08 2026
```

Diagram 6.8-17: Medium Stress Test (Everything is combined)

```
Start time: Fri Jul 31 07:59:33 PM +08 2026
High stress test & result:
stress-ng: info:  [8748] setting to a 1 min run per stressor
stress-ng: info:  [8748] dispatching hogs: 64 fallocate, 64 filename, 64 sync-file, 64 dir
stress-ng: info:  [8749] fallocate: using 16M file system space per stressor instance (total 1G of 18.71G available file system space)
stress-ng: info:  [8877] sync-file: using 16M file system space per stressor instance (total 1G of 18.83G available file system space)
stress-ng: metrc: [8748] stressor       bogo ops real time  usr time  sys time   bogo ops/s     bogo ops/s
stress-ng: metrc: [8748]                           (secs)    (secs)    (secs)   (real time) (usr+sys time)
stress-ng: metrc: [8748] fallocate           126     67.61      0.10      1.75         1.86          68.25
stress-ng: metrc: [8748] filename           4577     70.45      0.28      8.56        64.97         517.62
stress-ng: metrc: [8748] sync-file          2274     67.14      0.09      0.97        33.87        2146.17
stress-ng: metrc: [8748] dir              110270     77.98      6.56    102.75      1414.15        1008.79
stress-ng: info:  [8748] skipped: 0
stress-ng: info:  [8748] passed: 256: fallocate (64) filename (64) sync-file (64) dir (64)
stress-ng: info:  [8748] failed: 0
stress-ng: info:  [8748] metrics untrustworthy: 0
stress-ng: info:  [8748] successful run completed in 1 min, 21.10 secs
End time: Fri Jul 31 08:01:06 PM +08 2026
```

Diagram 6.8-18: High Stress Test (Everything is combined)

7.0 Result and Analysis

7.1 File allocation

Stress-ng Log:

Stress Level	Low	Medium	High
Bogo ops	71	71	71
Real time (s)	60.03	60.04	60.07
Usr time (s)	0.23	0.30	0.22
Sys time (s)	23.44	23.48	23.70

ApacheBench Logs:

Stress Level	Low	Medium	High
Throughput (Requests per second)	2074.84	2799.51	3297.15
Latency (Time per request)	4.820	17.860	30.329

Grafana Result:

Stress Level	Low	Medium	High
Disk iops (io/s)	38.76	40.63	42.49
Disk throughput (MB/s)	0.359	0.413	0.469
Disk Max wait time (ms)	13.96	13.49	13.07
Disk I/O utilization (peak)	4.40%	4.50%	4.60%
Average queue size (peak)	0.56	0.56	0.57

Analysis:

The result shows that file allocation activity increased as the stress level increased. Write requests increased from 38.76 w/s to 42.49 w/s. Write throughput also increased from 367.74 kB/s to 480.15 kB/s. This means that the system completed more write operations under higher stress.

However, disk utilization only increased from 4.40% to 4.60%. The average queue size also stayed almost the same, from 0.56 to 0.57. This shows that the disk was not close to saturation. Write await time decreased from 13.96 ms to 13.07 ms instead of increasing. This is different from the original hypothesis. The higher workload did not create longer disk waiting time.

Bogo operations stayed at 71 for all three stress levels. The real time also stayed close to 60 seconds. This shows that all tests finished successfully. System time increased slightly from 23.44 seconds to 23.70 seconds. The kernel spent a little more time handling the file system work, but the increase was small.

Overall, the file allocation workload increased the amount of writing, but it did not create a clear I/O bottleneck. The disk utilization, queue size and write await time stayed stable. This means that the tested fallocate workload did not put as much pressure on the storage system as expected.

Hypothesis Evaluation:

Low stress: Partially Accepted

The result shows low disk utilization and a small I/O queue. These two results match the hypothesis. However, the write await time was 13.96 ms instead of the predicted 1–2 ms. The system was stable, but the latency prediction was not correct.

Medium stress: Rejected

The hypothesis is rejected. The hypothesis predicted 40%–60% disk utilization and a 10%–20% increase in write latency. The actual disk utilization was only 4.50% and write await time decreased to 13.49 ms. The queue size also stayed at 0.56. The system handled more write requests without showing the expected increase in pressure.

High stress: Rejected

The hypothesis is rejected. Disk utilization was only 4.60%, which was far below the predicted value of more than 80%. Write await time also decreased to 13.07 ms, and the queue size stayed very small. The system did not reach the saturation point predicted in the hypothesis.

7.2 Flushes data from a file to a memory

Stress-ng Log:

Stress Level	Low	Medium	High
fsync Threads	8	32	64
Bogo Ops	16887	65885	128854
Bogo Ops/s (usr+sys)	1075.61	3688.97	5127.50
Bogo Ops/s (real time)	281.43	1097.97	2147.39
Real Time (s)	60.00	60.01	60.00
Usr Time (s)	3.14	3.76	3.96
Sys Time (s)	12.56	14.10	21.17

ApacheBench Logs:

Stress Level	Low	Medium	High
Throughput (Requests per second)	5835.50	8049.02	9095.31
Latency (Time per request)	1.714	6.212	10.995

Grafana Result:

Stress Level	Low	Medium	High
fsync Threads	8	32	64
Avg fsync Latency (ms)	2.24	2.24	2.29
Disk Utilization (%)	46.6	46.9	46.6
I/O Queue Depth	~1 (constant)	0-1 (fluctuating)	~1 (constant)
Average queue size (peak)	N/A	N/A	N/A

Analysis

The test results show that as the number of concurrent threads (8, 32, 64) of fsync gradually increases, the total operation volume, the operation rate per second, and the system time all increase simultaneously. High concurrency increases the kernel synchronization overhead. However, the average fsync delay and the average disk usage rate of the three groups of tests are basically the same. The peak disk usage rate is all close to 99.9%, which does not match the assumption in version 4.0.2 that the higher the load, the greater the delay and occupation. The reason is that the single disk queue quickly fills up, and the number of concurrent I/O operations is mostly 0 or 1; further increasing the fsync threads does not change the average delay and the average disk usage rate. The performance bottleneck is the single queue depth of the disk, rather than the number of concurrent fsync threads.

Hypothesis evaluation:

Low Load: Partially Accepted

The measured delay was 2.24ms, which met the expectation in the assumption that "the delay under low load should be lower than 5ms". However, the average disk utilization was 46.6%, which was higher than the expected "lower than 30%". This indicates that the disk pressure under low load was actually greater than expected.

Medium Load: Rejected

The assumption expected the delay to increase to 10-20ms and the utilization to rise to 60%-75%, but the measured delay remained at 2.24ms and the average utilization was only 46.9%, both significantly lower than the expected range.

High Load: Rejected

The assumption expected the delay to exceed 50ms and the utilization to be close to full load, but the measured delay was only 2.29ms and the average utilization was only 46.6%, far lower than expected. This indicates that the system's actual performance in this testing environment was better than the assumption.

7.3 Directory I/O stress testing

Stress-ng Log:

Stress Level	Low	Medium	High
Bogo ops	448870	204824	221847
Real time (s)	60.84	68.56	105.09
Usr time (s)	6.50	9.46	16.08
Sys time (s)	101.55	117.26	184.81

ApacheBench Log:

Stress Level	Low	Medium	High
Throughput (Requests per second)	13.15	13.79	12.70
Latency (Time per request)	760.222	725.382	787.390

Grafana Result:

Stress Level	Low	Medium	High
Disk iops (io/s)	260	460	1400
Disk throughput (MB/s)	9.00	16.00	18.50
Disk Max wait time (ms)	19ms	30ms	Negative value due to scrape mismatch under extreme load
Disk I/O utilization (peak)	4%	8.5%	9%
Average queue size (peak)	0.3	3.9	2.1

Root File Nodes Free (Million)	Mean = 20.4 Million
Root Filesystem used (GiB)	Mean = 7.89 GiB

Analysis:

The table above shows the result for directory stress testing using stress-ng --dir tool. By increasing (4, 24, 80) workers gradually, we can obtain a detailed result for each stress levels.

Based on the stress-ng log, the bogo operations decrease as we test from low to high concurrency. This means that the filesystem can handle small numbers of directory operations but struggle to handle higher concurrency (>20 workers). Starting at medium concurrency (20 workers), this shows that the threads are competing for directory locks and metadata structures which makes the system time to increase as we proceed to test with high concurrency (80 workers).

Then, for the Grafana result:

At low load test, the OS handled the stress level efficiently due to low concurrency (4 workers). This can be proven by the low disk I/O, low wait time and low queue size. This shows that the situation is cache-dominated, which means that most of the directory operations are handled by the kernel's in-memory caches and not hitting the physical disk. Hence, most of the lookups, create or remove operations can be performed with the cache memory.

At medium load test, the OS is facing a bad contention period and lots of cache misses. This can be proven by the disk increase in throughput, higher wait time, high disk utilization and high average queue size.

Cache misses:

As concurrency increases, the tendency for cache to miss is increased as well. This makes the OS cannot rely on its cache memory. Instead, it will request for data from the disk which involves data transfer. This is the reason why the data transfer rate is higher in medium load than in low load.

Bad contention:

The increase in number of threads will increase the competition in accessing the same resources. This will lead to higher waiting time because the threads spend more time waiting than working.

At high load test, we can see that the OS performance dropped because it reaches the saturation point where it starts to limit performance to keep system stable. This can be proven that when the disk IOPS spikes, the throughput and disk utilization no longer spikes but it will increase slowly.

Hypothesis evaluation:

Low stress: Accepted

The result shows low utilization, low wait time, and low queues. This represents the efficient scheduling of the OS and matches the scenario stated in the hypothesis. Therefore, the hypothesis is accepted.

Medium stress: Accepted

The result shows a spike in every metrics, for example throughput, disk utilization, wait time, and queue size. This means that the OS is stressed heavily due to lock contention and frequent cache misses. The result matches the scenario stated in the hypothesis. Therefore, hypothesis is accepted.

High stress: Accepted

The result shows a decrease for average queue size and a very slow increase for other metrics. This proves that the OS is in saturation point where it starts limiting performance to stabilize the system. It matches the scenario stated in hypothesis. Therefore, hypothesis is accepted.

7.4 File I/O stress testing

Stress-ng Log:

Stress Level	Low	Medium	High
bogo ops	160040	48525	29225
bogo ops/s (real time)	2635.88	788.86	334.05
bogo ops/s (usr+sys time)	2050.59	968.53	497.41
Real time (s)	60.72	61.51	87.49
Usr time (s)	1.59	1.97	2.36
Sys time (s)	76.46	48.13	56.40

ApacheBench Log:

Stress Level	Low	Medium	High
Throughput (Requests per second)	18.53	9.02	4.18
Latency (Time per request)	539.652	1108.42	2392.575

Grafana Result:

Stress Level	Low	Medium	High
File operation latency (ms)	18	4	20
Disk throughput (MB/s)	5	20	7.5
Disk I/O utilization (%)	36	21	14
Average queue size	0.3	1.61	2.29

Analysis:

The table above shows the result for file I/O stress testing using the stress-ng --filename tool. By increasing (4, 16, 64) workers gradually, we can obtain a detailed result for each stress level.

Based on the stress-ng log, bogo operations decrease as we test from low to high concurrency (160,040 to 48,525 to 29,225). This means that the file system can handle small numbers of concurrent file operations efficiently but is hard to handle higher concurrency (more than16 workers) as the bogo operation reduce rapidly.

Moreover, system time is not keeping increase, sys time is reduce at medium stress test (76.46s drop to 48.13s) before rising again at high stress test (56.40s), while real time jumps from 61.51s to 87.49s. This is shows the bottleneck is not caused by each operation becoming slower. Instead, as the workload increases, more threads have to wait for CPU scheduling and for XFS inode to become available. As a result, fewer operations can only be completed within the same time period.

In addition, the ApacheBench log also supports this explanation, as requests per second dropped from 18.53(low) to 9.02(medium) to 4.18(high), while latency per request increase from 539.652ms(low) to 1108.42ms(medium) to 2392.575ms(high). This confirms that the file system contention created by stress-ng has directly affect the Apache's ability to handle requests, both the stress test and the web service compete for the same disk and metadata resources, causing Apache performance to decrease.

For the Grafana result:

At low load test, the OS handled the stress level efficiently due to low concurrency (4 workers). This can be proved by the low disk utilization (36%) and low average queue size (0.3). This is because most file operations were handled by Linux's in-memory page cache in stead of access to physical disk. As a result, most operations can be completed quickly without waiting for the disk itself.

At medium load test, the system achieves to its best performance: disk throughput (20 MB/s) while latency drops to its lowest (4ms) and utilization falls (21%). This shows that higher workload does not always cause worse performance. With 16 concurrent processes, write operations were

combined efficiently, allowing the XFS to process them together and which improves throughput and response time.

At high load test, we can see that the OS performance drop down again, disk utilization drops further (14%), while average queue size increase to its highest point (2.29) and latency rises back to 20ms. high utilization does not always mean the disk is fully overloaded. At high concurrency, processes spend more time waiting for CPU scheduling and available resources instead of performing I/O operations. This makes disk utilization lower, even though the queue size becomes the highest. The ApacheBench results prove this, it shows the lowest throughput (4.18 req/s) and highest latency (239.258 ms) happen in the high stress test, proving that the system was most overloaded in this stage.

Hypothesis evaluation:

Low stress: Accepted

All three predictions are fine, throughput (~5 MB/s) stay within the predicted 4–6 MB/s range, latency (~18ms) stay under the predicted 20ms ceiling, and disk utilisation (36%) stay below the 40%. This confirms the Linux I/O scheduler managed 4 concurrent file operation processes without any performance drop.

Medium stress: Partially Rejected

Disk utilisation (21%) fell within the predicted area below 25%, it works fine. However, throughput did not reduce by 35% as predicted, instead of reducing, it increase to 20 MB/s. Latency also did not increase by 50% as predicted, instead of increase, it reduces to 4ms. Both results are rejected, the prediction is wrong. The actual result shows that XFS was able to group together and process them efficiently, temporarily improving performance.

High stress: Partially Rejected

Disk utilization (14%) fell within the predicted area below 15%, it works fine. However, throughput did not reduce by ~70% as predicted, it remained at 7.5 MB/s, still higher than the low stress. Latency also did not increase by 200% as predicted, it only increases to 20ms, roughly 11% over low stress. Both results are rejected, showing that although throughput and latency did not change as much as predicted, the system was still overloaded because the other evidence shows that test exceeded the 60-second timeout by 27.49 seconds, and Bogo Ops/s dropped by 87.3% compared to low stress.

7.5 Large metadata uploads + log churn

Stress-ng Log:

Stress Level	Low	Medium	High
Bogo ops	352	319	399
Real time (s)	60.00	60.00	60.01
Usr time (s)	0.08	0.09	0.09
Sys time (s)	1.33	1.33	1.37

ApacheBench Logs:

Stress Level	Low	Medium	High
Throughput (Requests per second)	2832.64	7265.38	7521.72
Latency (Time per request)	3.530	6.882	13.295

Grafana Result:

Stress Level	Low	Medium	High
Disk iops (io/s)	38.01	38.64	39.80
Disk throughput (MB/s)	0.071	0.062	0.073
Disk Max wait time (ms)	11.93	11.79	11.44
Disk I/O utilization (peak)	3.96%	4.03%	4.14%
Average queue size (peak)	0.56	0.56	0.47
Root File Nodes Free (Million)	18.93 Million		

Root Filesystem used (GiB)	11 GiB

Analysis:

The result shows that the combined fallocate and file workload increased write requests only slightly. Write requests increased from 38.01 w/s at low stress to 39.80 w/s at high stress. Disk utilization also increased only from 3.96% to 4.14%. This shows that the storage system received more write activity, but the increase was very small.

Write throughput changed from 72.54 kB/s at low stress to 63.51 kB/s at medium stress and then increased to 74.73 kB/s at high stress. This result did not show a clear upward or downward trend. The change may be related to the different way the system processed the file allocation and file operations at each stress level. It does not show a serious performance drop because all tests finished in about 60 seconds.

Write wait time stayed close to 12 ms at all three levels. It decreased slightly from 11.93 ms to 11.44 ms. This is different from the hypothesis, which expected write latency to increase as the workload became higher. Bogo operations decreased at medium stress and increased again at high stress. This shows that the amount of completed work changed during the tests, but the system did not stop working or reach a serious I/O bottleneck.

Overall, the combined workload did not create the high disk pressure expected in the hypothesis. Disk utilization stayed very low, write wait time stayed stable, and all tests finished successfully. The system remained stable even when fallocate and file workloads ran together.

Hypothesis Evaluation:

Low stress: Partially Accepted

The hypothesis is partially accepted. Disk utilization was 3.96%, which was below the predicted 30%. This shows that the system was under low storage pressure. However, write wait time was 11.93 ms instead of below 5 ms. The system was stable, but the latency prediction was not correct.

Medium stress: Rejected

The hypothesis is rejected. It predicted that disk utilization would increase to 40%–60% and that write latency would increase by 10%–20%. The actual disk utilization was only 4.03% and write wait time stayed close to the low-stress result. The expected increase in I/O waiting did not appear.

High stress: Rejected

The hypothesis is rejected. It predicted more than 80% disk utilization, a 30%–60% increase in write latency and high I/O wait. The actual disk utilization was only 4.14% and write wait time decreased to 11.44 ms. The tests also finished within about 60 seconds. The system did not reach the expected bottleneck condition.

7.6 I/O latency + persistence guarantees

Stress-ng Log:

Stress Level	Prometheus query result	sync-file (16 workers)
Bogo ops	1204889	25528
Bogo Ops/s (usr+sys)	8159.89	1844.51
Bogo Ops/s (real time)	19598.68	413.38
Real Time (s)	61.48	61.75
Usr Time (s)	5.67	2.65
Sys Time (s)	141.99	11.19

ApacheBench Logs:

Stress Level	Low	Medium	High
Throughput (Requests per second)	5658.73	6772.19	17624.50
Latency (Time per request)	1.767	7.383	5.674

Grafana Result:

Metric	Prometheus query result	sync-file (16 workers)
Disk Read Throughput	Mean 43.3 kB/s / Max 75.7 kB/s (combined sda)	_____
Disk Write Throughput	Mean 7.45 MB/s / Max 19.6 MB/s (combined sda)	_____
Disk I/O Utilization (%)	Mean 36.6% / Max 85.3% (combined sda)	_____
Write Amplification Factor	N/A (Not Measured, requires comparison between the number of bytes written by the application	N/A

	layer and the actual number of bytes written to the disk)	
Avg / p99 Latency Peak (ms)	Background value ~3.5ms → During the test, the peak value stabilized at ~5.90-5.91ms → After the test, it dropped back to ~2.3-2.5ms (Note: Node_exporter does not expose the delay histogram, so the true p99 cannot be calculated. Here, the approximate peak value of the average delay within the test window is used as a substitute.)	————
Transaction Packet Loss Rate (%)	N/A (This test did not cover any network-related matters.)	N/A

Analysis

The actual measurement data shows that under a medium load where both the "file" and "sync-file" stress items are running in parallel with 16 worker processes, the disk write latency significantly increased from the background value of approximately 3.5ms and remained stable at around 5.90-5.91ms (approximately 1.7 times the background value) during the test run. After the test, the latency quickly dropped back to 2.3-2.5ms, even slightly lower than the background level before the test - this is significantly lower than the expectation in 4.0.5.2 where it was assumed that "the fsync latency should increase to 20-40ms" under medium load. The actual increase was far less drastic than the assumption. In terms of disk throughput, the average Disk I/O Utilization was 36.6% (peak 85.3%), and the average Disk Write Throughput was 7.45 MB/s (peak 19.6 MB/s). From the IOps/Throughput panel, it can be seen that the read and write volumes continuously climbed throughout the test window rather than quickly reaching full capacity, indicating that the system still had considerable capacity under the medium concurrent read-write mixed stress and did not exhibit the significant degradation as expected in the assumption - "the

system total throughput decreased by approximately 20%-30%". The "filename (--file)" stress item contributed the vast majority of Bogo Ops (1,204,889 vs sync-file's 25,528) and Sys Time (141.99s vs 11.19s), indicating that the pressure of ordinary file read and write operations on the system was much greater than the fsync synchronous call itself in this test.

Hypothesis evaluation:

Partially Accepted: The directional trend that the delay indeed increases with the rise in load is consistent with the assumption (this is different from the result in 4.0.5.2, where the delay hardly changed with the load). However, the actual increase (from ~3.5ms to ~5.9ms) is much smaller than the 20-40ms expected by the assumption. The decrease in disk utilization and throughput also did not reach the extent envisioned by the assumption, indicating that the direction of the assumption is correct but the estimation of the impact extent was too high.

7.7 Metadata heavy workloads

Result for --dir stress test:

Stress Level	Low	Medium	High
bogo ops	239571	196623	139902
Real time (s)	60.66	85.48	82.11
Usr time (s)	4.71	9.88	10.56
Sys time (s)	63.59	121.88	123.12

ApacheBench Log:

Stress Level	Low	Medium	High
Throughput (Requests per second)	13.74	9.33	8.28
Latency (Time per request)	727.552	1071.950	1207.202

Result for --fallocate stress test:

Stress Level	Low	Medium	High
bogo ops	135	348	522
Real time (s)	60.04	60.06	65.74
Usr time (s)	0.61	0.49	0.31
Sys time (s)	26.32	20.05	15.56

Stress Level	Low	Medium	High
Disk iops (io/s)	1.20k	1.40k	4.0k
Disk throughput (MB/s)	600	400	200
Disk Max wait time	5ms	5ms	80ms

Disk I/O utilization (peak)	80%	85%	70%
Average queue size (peak)	6	8	10
Root Filesystem Space Available (GiB)	Mean = 31.2 GiB		
Root Filesystem used (GiB)	Mean = 8.07GiB		
Root File Nodes Free (Million)	Mean = 20.4 Million		

The table above shows the result for directory stress testing using the stress-ng --dir and --fallocate tool. By gradually increasing workers for dir (4, 24, 80) and fallocate (4, 24, 60) simulatneously, we can observe how the filesystem behaves under different concurrency levels.

From stress-ng log, we can observe that:

Bogo operations decrease steadily from low to high concurrency. This indicates the filesystem can efficiently handle small numbers of directory operations but struggles as concurrency rises beyond 20 workers.

System time increases significantly at medium and high loads, showing the kernel spends more cycles managing locks and metadata.

Lock contention:

At medium concurrency, threads compete for directory locks and metadata structures, reducing throughput and increasing latency.

Grafana metrics:

Low load (4 workers): Accepted

Cache-dominated. This situation shows that OS have low disk I/O, short wait times, and minimal queue depth prove the OS can efficiently schedule requests using in-memory caches. The result matches the exact situation described in hypothesis. Therefore, hypothesis is accepted.

Medium load (24 workers): Partially accepted

Bad contention period. This situation shows that cache misses rise, throughput increases, wait times spike, disk utilization grows, and queue depth expands. Hence, threads spend more time waiting than working. The result shows 85% peak in disk utilization which is far from the predicted disk utilization (below 70%). However, the result still matches the trends of high stress level behaviour. Therefore, hypothesis is still partially accepted.

High load (80 workers): Partially accepted

Saturation point. Disk IOPS spike, but throughput and utilization increase slowly. Queue depth shrinks as the kernel throttles to stabilize performance. The result is a bit off from the prediction in hypothesis, but it still shows saturation point (bogo ops for directory decrease as workers increase). Therefore, hypothesis is partially accepted.

7.8 Everything is combined

ApacheBench Log:

Stress Level	Low	Medium	High
Throughput (Requests per second)	6.46	3.32	2.20
Latency (Time per request)	1548.82	3010.589	4536.552

Stress-ng Log:

Result for --fallocate stress test:

Stress Level	Low	Medium	High
bogo ops	115	280	126
bogo ops/s (real time)	1.91	4.62	1.86
bogo ops/s (usr+sys time)	7.62	28.46	68.25
Real time (s)	60.25	60.55	67.61
Usr time (s)	0.19	0.19	0.10
Sys time (s)	14.90	9.65	1.75

Result for --file stress test:

Stress Level	Low	Medium	High
bogo ops	20251	7415	4577
bogo ops/s (real time)	336.59	116.81	64.97
bogo ops/s (usr+sys time)	1640.59	972.09	517.62
Real time (s)	60.17	63.48	70.45
Usr time (s)	0.55	0.40	0.28
Sys time (s)	11.79	7.22	8.56

Result for --fsync stress test:

Stress Level	Low	Medium	High
bogo ops	4972	9690	2274
bogo ops/s (real time)	82.75	160.41	33.87
bogo ops/s (usr+sys time)	410.20	1325.19	2146.17
Real time (s)	60.09	60.41	67.14
Usr time (s)	1.85	1.08	0.09
Sys time (s)	10.27	6.23	0.97

Result for --dir stress test:

Stress Level	Low	Medium	High
bogo ops	119109	109944	110270
bogo ops/s (real time)	1958.24	1146.82	1414.15
bogo ops/s (usr+sys time)	5927.70	1543.80	1008.79
Real time (s)	60.82	95.87	77.98
Usr time (s)	1.37	3.55	6.56
Sys time (s)	18.73	67.67	102.75

Grafana Result:

Stress Level	Low	Medium	High
File operation latency (ms)	3	31.3	514
Disk throughput (MB/s)	469	279	38
Disk I/O utilization (%)	87.5	88	53.5

Average queue size	4.6	11.5	42.2

Analysis

The table above shows the result for the everything combined stress test, running --fallocate, --file, --fsync, and --dir simultaneously using stress-ng. By increasing (4, 16, 64) workers gradually across all four stressors, we can obtain a detailed result for each stress level.

Based on the stress-ng log, the four stressors do not behave similarly under combined pressure. --file and --dir behave as expected, --file's bogo ops is keep falling (336.59 to 116.81 to 64.97) as concurrency increases, while --dir's sys time keep (18.73s to 67.67s to 102.75s). This shows that directory operations require more kernel processing as more processes compete for the same resource.

However, --fallocate and --fsync shows in different way, their highest ops/s actually at medium stress, for both fallocate (1.91 to 4.62 to 1.86) and fsync (82.75 to 160.41 to 33.87) and their sys time decreases instead of increases for both fallocate (14.90s to 9.65s to 1.75s) and fsync (10.27s to 6.23s to 0.97s). This means that at high stress, --file and --dir operations reduce the processing time available for --fallocate and --fsync. The file system handles more requests from –file and –dir first and delays space allocation and sync operations, causing these operations to receive less kernel processing time.

The ApacheBench log also confirm that this contention has impact beyond the disk itself. Requests per second keep drop from 6.46 (low) to 3.32 (medium) to 2.20 (high), while latency increase a lot from 1548.82ms (low) to 3010.589ms (medium) to 4536.552ms (high). It cause large performance drop compare to other scenario, it shows that combining four stress workloads creates greater pressure on Apache's ability to handle requests.

Then, for the Grafana result:

At low stress test, the OS handled the combined workload efficiently even with four stressors running at the same time. This can be proved by the low file operation latency (3ms) and low average queue size (4.6), even though disk utilisation was already high (87.5%). This shows that

most operations in low stress tests are still managed efficiently by the page cache and XFS, preventing any delays from disk queueing.

At medium stress test, the OS handle more stress, as all four stressors compete directly. This can be proven by the large increase in latency (3ms to 31.3ms) and average queue size (4.6 to 11.5), even the disk throughput fall (469 MB/s to 279 MB/s).

At high stress test, the OS performance dropped significantly. Disk utilization has large decreased from 88% to 53.5%, latency increased sharply from 31.3 ms to 514 ms and the average queue size have large increase from 11.5 to 42.2. This shows that the system has reached saturation. At this point, processes spent more time waiting for CPU scheduling and kernel resources than performing I/O operations. As a result, disk utilization is lower even though the queue became much larger. The ApacheBench results also prove this, by showing the lowest throughput (2.20 req/s) and highest latency (453.655 ms), confirm that the system was most overloaded during the high stress test.

Hypothesis evaluation:

Low stress: Partially Accepted

Average queue size (4.6) is almost near the predicted of queue size 5. However, disk throughput is 469MB/s which is more than 450, disk utilization (87.5%) is more than predicted range 60–75%, and latency (3ms) is below the predicted 4–5ms range. Although there are three of four predictions that are wrong, but the hypothesis correctly predicted that the system was not under heavy load at low concurrency.

Medium stress: Rejected

Disk utilization (88%) is more than predicted range 70–80%, disk throughput is 279MB/s which is reduce more than 35% compared to low stress test, latency (31.3ms) is higher than the predicted 10–12ms, and average queue size (11.5) is lower than predicted 15. All four predictions are rejected, showing the combined workload created more latency pressure than expected at medium concurrency.

High stress: Rejected

Average queue size (42.2) is more than predicted 30 of queue size, which can confirm that the file system reached saturation. Disk throughput is 38MB/s which is reduce more than 55% compare to low stress test, latency (514ms) was more than 20 times higher than the predicted 20–24ms, and disk utilization (53.5%) was much lower than the predicted value of nearly 100%. All four predictions are rejected, and the test shows that the main bottleneck is the I/O queue, as the file system became overloaded even though disk utilization is lower.

8.0 Limitations and Recommendations

Limitations:

- The monitoring infrastructure itself is unstable, causing data loss or the need to re-run tests multiple times. In this project, Prometheus/Grafana was deployed within Minikube, while VMs used DHCP to dynamically allocate IP addresses. During the actual testing process, operations such as Minikube restarts and Pods being rebuilt due to configuration updates repeatedly resulted in VM IP not being consistent with the target address in the Prometheus scrape configuration. Moreover, Prometheus did not have persistent storage (PVC) configured, and once a Pod was deleted and rebuilt, the previously collected historical metric data was completely lost. Grafana also lost the previously imported dashboard configuration upon container restart due to the lack of persistent storage. Both 5.0.2 and 5.0.6 tests experienced at least one instance of "the test has been completed but the monitoring data is unavailable" during the testing process, forcing the re-execution of the test script.
- Each load level was tested only once, without repeated trials to obtain averages and standard deviations. The results of each sub-experiment may be affected by occasional systematic noise from a single measurement, making it impossible to assess the statistical significance or variability of the results.
- The test was conducted only under a single virtual machine hardware configuration (fixed CPU/memory/disk specifications), and did not explore the impact of different storage media (such as NVMe SSD and traditional HDD) or different hardware resource quotas on the results. Nor did it test for higher concurrency levels (such as when the number of fsync threads exceeds 64). Therefore, it is impossible to confirm the true performance inflection point and saturation limit of the system.

Recommendations:

- Configure Persistent Volume Claim for Prometheus to prevent loss of historical monitoring data due to Pod reconfiguration; at the same time, configure static IP for the VM (or use a resolvable hostname instead of hard-coded IP in the Prometheus scrape configuration) to fundamentally avoid monitoring interruption caused by DHCP address reassignment.

- In the subsequent research, it is recommended to conduct at least 3-5 tests for each load level and calculate the mean and standard deviation to quantify the fluctuation range of the results and enhance the statistical credibility of the conclusions.
- It is recommended to repeat this experiment on different storage media (such as SSD, NVMe, HDD), and gradually expand the concurrent testing range until a significant performance inflection point is observed in the system, in order to more comprehensively evaluate the impact of hardware differences on the I/O performance of the file system, and more accurately determine the system saturation threshold.

9.0 Conclusion

This project studied how Oracle Linux manages file system operations and disk I/O under different workloads. Apache and MariaDB created normal server activity. Stress-ng created file allocation, file, fsync and directory workloads. Prometheus and Grafana collected and displayed the system data. This setup helped us observe how different services use the same storage resources.

The results show that Oracle Linux performed well under simple workloads. In the fallocate test, write requests increased from 38.76 w/s to 42.49 w/s. Write throughput increased from 367.74 kB/s to 480.15 kB/s. Disk utilization only changed from 4.40% to 4.60%. The large media upload and log churn test showed a similar result. Write activity increased, but write wait time stayed close to 12 ms. These results support the original hypotheses. The Linux file system and XFS could manage normal file allocation and continuous writing without a clear bottleneck.

The metadata-heavy workload showed a different situation. The test increased dir workers from 4 to 80 and fallocate workers from 4 to 60. Bogo operations decreased as the number of workers increased. This means that the file system could handle a small number of operations, but it had more difficulty at higher concurrency. System time also increased at medium and high loads. This shows that the kernel spent more time managing locks and metadata. At medium load, cache misses increased and threads competed for the same resources. Peak disk utilization reached about 85%, which was higher than the predicted value. At high load, IOPS increased, but throughput and disk utilization increased slowly. The queue depth also became smaller because the kernel started to control the requests. This result shows that the system had reached a saturation point. The low-load hypothesis was accepted. The medium- and high-load hypotheses were partially accepted.

The fsync and file tests showed that system performance is not always directly related to the number of threads. In the fsync test, the average latency stayed around 2.24 to 2.29 ms when the number of threads increased from 8 to 64. This rejected our prediction that the latency would become higher than 50 ms. The single disk queue limited the result. Adding more fsync threads did not create a large change in the average latency. In the mixed file and sync-file test, latency increased from about 3.5 ms to 5.9 ms. This was still much lower than the predicted 20 to 40 ms. Ordinary file operations also created much more Bogo Ops and system time than sync-file. This shows that normal file reading and writing created more pressure than fsync in this test.

The file test gave another interesting result. The medium workload had higher throughput and lower latency than the low workload. This may be because XFS grouped some requests and processed them more efficiently. At high stress, Bogo Ops/s became much lower. The test also took 87.49 seconds instead of 60 seconds. Disk utilization became lower, but the system was not working better. Many processes were waiting for CPU time, queue space or disk completion. The file system also had to handle more inode and metadata updates. This shows that low disk utilization can still happen when the system has a serious waiting problem.

The combined workload gave the clearest result. When fallocate, file, fsync and dir ran together, the average I/O queue increased from about 4.6 to 42.2. File operation latency increased from 3 ms to 514 ms. The Linux kernel had more work to do. It had to manage fsync, file creation, directory changes, inode updates, metadata changes and I/O scheduling at the same time. These operations competed for the same disk. The high workload reached the limit of the XFS file system. Queue size and latency showed this problem more clearly than disk utilization.

This project changed our understanding of operating system performance. Before the tests, we expected higher workloads to always cause higher disk utilization and higher latency. The results were more complex. Some single workloads stayed stable. Some medium workloads improved for a short time. The combined workload caused a sudden rise in waiting time. The metadata test also showed that cache misses, lock contention and kernel control can change the system result. This means that performance depends on the type of workload and how different operations work together. The rejected hypotheses helped us compare our expectations with the real system behaviour.

The project also showed the importance of reliable monitoring. Prometheus and Grafana sometimes lost data because Minikube was restarted, Pods were rebuilt and VM IP addresses changed. Each load level was tested only once. The results are useful observations, but they are not final statistical proof. Repeating each test, using persistent monitoring storage and testing different disks would make the results more reliable. Overall, Oracle Linux and XFS handled normal workloads well. High concurrency and mixed I/O caused a serious queue bottleneck. To judge system performance correctly, we need to study throughput, latency, queue size, disk utilization and system time together.

References

A, H. a. R. S. H. H. A. (2024, September 2). *Understanding the Linux Filesystem: An In-Depth Guide for DevOps engineers*. DEV Community. https://dev.to/prodevopsguytech/understanding-the-linux-filesystem-an-in-depth-guide-for-devops-engineers-ona

Carrigan, T. (2020, June 9). *Inodes and the Linux filesystem*. Redhat.com. https://www.redhat.com/en/blog/inodes-linux-filesystem

GeeksforGeeks. (2026, April 25). *File systems in operating system*. GeeksforGeeks. https://www.geeksforgeeks.org/operating-systems/file-systems-in-operating-system/

Molnar, I. (2025). Completely Fair Scheduler design. The Linux Kernel Archives. https://www.kernel.org/doc/html/next/scheduler/sched-design-CFS.html

Performance Scaling - Apache HTTP Server Version 2.5. (n.d.). https://httpd.apache.org/docs/trunk/misc/perf-scaling.html

Rowjee, A. (2026, April 9). Linux IO Subsystem Walkthrough. Rowjee.com. https://rowjee.com/blog/linux_io_subsystem_walkthrough